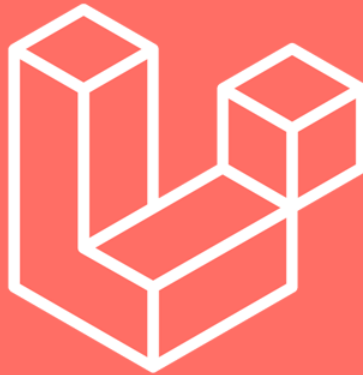




{ REST API }

STUDI KASUS: Sistem Peminjaman Alat

API-LARAVEL

Kata Pengantar

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas berkat dan rahmat-Nya, modul ajar "**Pengembangan REST API: Aplikasi Peminjaman Alat**" ini dapat diselesaikan. Modul ini disusun sebagai panduan komprehensif bagi para pembelajar untuk memahami peran strategis seorang pemrogram di perusahaan *software developer* dalam membangun sistem yang modern dan skalabel.

Di era digital saat ini, tren industri telah beralih dari arsitektur monolitik menuju **Decoupled Architecture** (Arsitektur Terpisah). Dalam ekosistem ini, **Application Programming Interface (API)** memegang peranan krusial sebagai jembatan komunikasi yang memungkinkan berbagai aplikasi—baik itu mobile, web frontend, maupun layanan pihak ketiga—untuk saling bertukar data secara efisien. Melalui modul ini, pembelajar akan diajak untuk mendalami bagaimana Laravel bertindak sebagai mesin di balik layar yang menyediakan data mentah demi mendukung fleksibilitas sistem tersebut.

Fokus utama modul ini adalah implementasi **REST API**, sebuah "aturan main" komunikasi stateless yang populer di industri. Pembelajar akan dibimbing secara langkah demi langkah untuk membangun fitur-fitur utama, mulai dari sistem autentikasi, pengelolaan data master (User, Alat, Kategori), hingga transaksi kompleks seperti Peminjaman dan Pengembalian alat. Materi ini juga mencakup aspek-aspek penting lainnya, seperti:

- Penerapan metode HTTP standar (**GET, POST, PUT/PATCH, DELETE**) sesuai pola CRUD.
- Pemanfaatan format **JSON** sebagai standar pertukaran data yang ringan dan universal.
- Pemahaman terhadap **HTTP Status Codes** untuk memastikan respons server yang informatif dan profesional.
- Standardisasi data melalui **API Resources** serta perlindungan keamanan melalui **Sanctum** dan middleware berbasis peran (**RBAC**).

Modul ini dirancang sedemikian rupa dengan menyertakan skema database dan tabel fitur untuk tiga level pengguna (**Admin, Petugas, Peminjam**) guna memberikan gambaran nyata mengenai kebutuhan industri. Kami berharap modul ini dapat membantu pembelajar menguasai kompetensi pengembangan API yang bersih (*clean code*) dan terstruktur.

Selamat belajar dan berkarya!

Bandung, Juli 2026
Tim Penyusun

Daftar Isi

Kata Pengantar.....	
Daftar Isi	ii
Pengembangan Aplikasi Peminjaman Alat	1
Konsep Dasar API & Arsitektur REST	2
1. Apa itu API? (Analogi Pelayan Restoran).....	2
2. Mengapa API Penting di Era Modern?	2
3. Mengenal REST API	2
4. Metode Utama (HTTP Methods)	3
5. Format Data: JSON.....	4
HTTP Response	6
1. HTTP Status Codes (Kode Status).....	6
1.a 2xx Success (Berhasil):	6
1.b 4xx Client Errors (Kesalahan Klien):	6
1.c 5xx Server Errors (Kesalahan Server):	6
2. Response Body & Standarisasi (API Resources)	6
Instalasi dan Konfigurasi Tools	8
Konfigurasi Laravel	16
Penyesuaian pada file .env	16
Install Package Sanctum via Composer	16
Instalasi Ekosistem API.....	16
Instalasi library PDF	17
Storage Link Laravel	17
Migrations	18
Materi Singkat	18
Kegiatan Praktikum	20
1. Update file create_users_table.php.....	20
2. 02_create_kategori_table.php.....	21
3. 03_create_alat_table.php.....	21
4. 04_create_peminjaman_table.php	22

5.	05_create_detail_pinjam_table.php.....	22
6.	06_create_pengembalian_table.php	23
7.	07_create_log_aktivitas_table.php	23
8.	Jalankan File Migrasi	24
9.	Pengujian	25
Models.....		26
Materi Singkat		26
Kegiatan Praktikum		27
1.	Model User.....	27
2.	Model Kategori	29
3.	Model Alat	29
4.	Model Peminjaman	30
5.	Model DetilPinjam	31
6.	Model Pengembalian	32
7.	Model LogAktivitas	33
Seeders		35
Materi Singkat		35
Kegiatan Praktikum		35
1.	Seeder User	35
2.	Seeder Kategori	38
3.	Seeder Alat	38
4.	Seeder Peminjaman.....	40
5.	Seeder DetailPinjam	41
6.	Seeder Pengembalian.....	42
7.	Seeder LogAktivitas	43
8.	DatabaseSeeder	44
9.	Db:seed & Pengecekan.....	45
Sistem Autentikasi API (Login, Register, Logout)		46
Kegiatan Praktikum		46
Persiapan Instalasi Postman		46
Persiapan Form Request.....		46
Persiapan Resource		48
Persiapan Middleware Role Check.....		49

Registrasi Middleware bootstrap/app.php.....	51
Membuat AuthController.....	52
Route API	57
Pengujian Sistem Autentikasi API.....	59
Membuat Fitur Untuk Mengelola Kategori.....	64
1. Implementasi API Resources (Kategori)	64
2. Implementasi Form Requests	64
StoreKategori	64
UpdateKategori.....	65
Implementasi Controllers	66
Update route/api.php.....	69
Pengujian Fitur Kategori.....	70
Membuat Fitur Untuk Mengelola Alat.....	75
Implementasi API Resource (Alat)	75
Implementasi Form Requests	76
StoreAlat.....	76
UpdateAlat.....	78
Implementasi Controllers	79
Update Model User	83
Update route/api.php.....	83
Pengujian Fitur Alat.....	83
Tugas Pengujian	85
Membuat Fitur User	86
Implementasi API Resource (User)	86
Implementasi Form Requests	86
StoreUser	86
UpdateUser	88
Implementasi Controllers	90
Update model User	93
Update route/api.php.....	93
Tugas Pengujian	94
Membuat Fitur Peminjaman.....	95
Implementasi API Resource.....	95

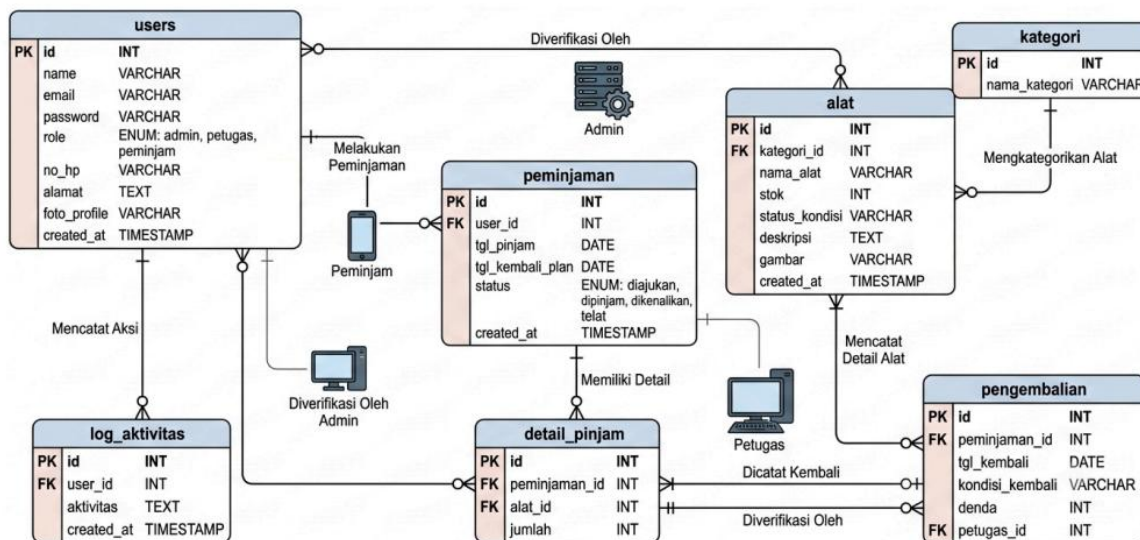
Implementasi Form Request Peminjaman	96
Implementasi Controllers	98
Update route/api.php.....	105
Tugas Pengujian	106
Membuat Fitur Pengembalian.....	107
Implementasi Form Request	107
StorePengembalianRequest.....	107
UpdatePengembalianRequest.....	108
Implementasi Controllers	109
Update route/api.php.....	114
Tugas Pengujian	115
Log Aktifitas	116
Materi Singkat	116
Kegiatan Praktikum	117
1. Membuat File Observer Alat.....	117
2. Membuat File Observer Peminjaman	119
3. Membuat File Observer Pengembalian	120
Membuat API Resource untuk Log	121
Membuat LogAktivitasController	122
Update Route/api.php.....	123
Tugas Pengujian	123
Membuat Fitur Laporan	124
Kegiatan Praktikum	124
Update Route/api.php.....	126
Tugas Pengujian.....	126

Pengembangan Aplikasi Peminjaman Alat

Sebagai seorang Pemrogram pada sebuah Perusahaan Software Developer, anda ditugaskan sebagai programmer, anda diminta untuk membuat sebuah Aplikasi Peminjaman Alat. Tugas Anda adalah membuat sebuah aplikasi Peminjaman Alat dengan 3 level pengguna, dengan fitur sebagai berikut:

Tabel Fitur Aplikasi Peminjaman Alat

Fitur	Admin	Petugas	Peminjam
Login	V	V	V
Logout	V	V	V
CRUD user	V		
CRUD alat	V		
CRUD kategori	V		
CRUD data peminjaman	V		
CRUD pengembalian	V		
Log Aktifitas	V		
Menyetujui Peminjaman		V	
Memantau Pengembalian		V	
Mencetak Laporan		V	
Melihat daftar alat			V
Mengajukan peminjaman			V
Mengembalikan alat			V



Skema Database Aplikasi Peminjaman Alat

Pada modul ini kita akan fokus membahas dan mengerjakan REST API dengan Laravel.

Konsep Dasar API & Arsitektur REST

1. Apa itu API? (Analogi Pelayan Restoran)

Bayangkan **API** seperti seorang **pelayan di sebuah restoran**. Kamu sebagai tamu (klien) ingin memesan makanan dari dapur (server). Kamu tidak langsung masuk ke dapur dan memasak sendiri, melainkan memberikan pesananmu kepada pelayan. Pelayan tersebutlah yang menyampaikan pesananmu ke dapur, lalu mengantarkan makananmu kembali ke meja.

Secara teknis, API adalah **jembatan komunikasi** antar sistem aplikasi yang berbeda. Dengan API, aplikasi yang kamu buat bisa "berbicara" dan bertukar data dengan aplikasi lain.

- **Kamu sebagai Tamu (Klien):** Bayangkan kamu adalah sebuah aplikasi (misalnya aplikasi mobile di HP). Kamu duduk di meja restoran dan ingin memesan makanan. Kamu memiliki kebutuhan (permintaan data), tetapi kamu tidak punya akses langsung untuk masuk ke dapur.
- **Dapur (Server):** Dapur adalah server atau pusat data tempat semua "makanan" (data dan logika program) disiapkan. Dapur memiliki semua bahan yang kamu butuhkan, namun sistemnya tertutup untuk orang luar demi keamanan dan keteraturan.
- **Pelayan (API):** Di sinilah peran **API**. Pelayan berfungsi sebagai **jembatan komunikasi**. Kamu memberikan pesananmu kepada pelayan. Pelayan kemudian membawa catatan pesanan tersebut ke dapur, lalu kembali lagi ke mejamu untuk mengantarkan makanan yang sudah jadi.

2. Mengapa API Penting di Era Modern?

Dahulu, aplikasi web bersifat **monolitik**, di mana tampilan (HTML/CSS) dan logika data menjadi satu kesatuan utuh. Namun, sekarang tren industri beralih ke **Decoupled Architecture** (Arsitektur Terpisah).

Dalam ekosistem modern, **Laravel bertindak sebagai mesin di balik layar** yang hanya mengirimkan **data**, bukan lagi halaman web utuh. Data ini kemudian digunakan oleh berbagai "klien" seperti:

- **Aplikasi Mobile:** Android atau iOS mengambil data yang sama dari API yang kamu buat.
- **Frontend Framework:** Teknologi seperti Vue.js atau React yang fokus membuat tampilan cantik.
- **Layanan Pihak Ketiga:** Aplikasi lain yang ingin berintegrasi dengan sistemmu.

3. Mengenal REST API

REST (Representational State Transfer) adalah "aturan main" agar komunikasi API bekerja secara teratur menggunakan protokol HTTP. Sistem yang menggunakan REST API sangat

populer karena **stateless** (setiap permintaan harus membawa informasi lengkap) dan mudah dikelola.

4. Metode Utama (HTTP Methods)

Metode-metode HTTP seperti GET, POST, PUT, dan DELETE adalah "cara" atau instruksi yang kita berikan kepada pelayan (API) untuk melakukan sesuatu terhadap data di dapur (server). Dalam standar industri, metode ini mengikuti pola **CRUD** (Create, Read, Update, Delete).

Berikut adalah penjelasan rinci mengenai cara kerja masing-masing metode berdasarkan sumber:

1. GET 📄 (Read - Membaca Data)

Metode ini digunakan untuk **mengambil atau meminta data** dari server tanpa mengubah apa pun di dalamnya.

- **Cara Kerja:** Klien mengirimkan permintaan ke URL tertentu, dan server merespons dengan mengirimkan data yang diminta (biasanya dalam format JSON).
- **Contoh Kasus:** Menampilkan daftar semua artikel blog atau mencari detail produk tertentu berdasarkan ID.
- **Karakteristik:** Ini adalah metode "normal" yang dijalankan saat kamu mengetik alamat web langsung di browser atau mengklik sebuah link.

2. POST 📤 (Create - Membuat Data Baru)

Metode ini digunakan untuk **mengirimkan data baru** ke server untuk disimpan di basis data.

- **Cara Kerja:** Data dikirimkan melalui "body" dari permintaan (bukan di URL) agar lebih aman. Di Laravel, penggunaan POST pada form web memerlukan **CSRF Token** sebagai pelindung keamanan agar permintaan tidak ditolak (Error 419).
- **Contoh Kasus:** Mendaftarkan akun pengguna baru atau menambah data produk baru ke toko online.

3. PUT atau PATCH 🔄 (Update - Mengubah Data)

Keduanya digunakan untuk **memperbarui atau mengubah data** yang sudah ada di server.

- **Cara Kerja:**
 - **PUT:** Biasanya digunakan untuk mengganti seluruh data sumber daya dengan data yang baru dikirimkan.
 - **PATCH:** Digunakan jika kita hanya ingin mengubah sebagian kecil data saja (misalnya hanya mengubah harga produk tanpa mengubah namanya).
- **Catatan Laravel:** Karena formulir HTML standar tidak mendukung metode PUT secara langsung, Laravel menggunakan "trik" dengan tetap menggunakan metode POST di form namun menambahkan arahan `@method('PUT')` di dalam kodenya.

4. DELETE 🗑️ (Delete - Menghapus Data)

Metode ini digunakan untuk **menghapus data** dari server.

- **Cara Kerja:** Klien mengirimkan permintaan penghapusan yang ditujukan pada ID data yang spesifik.
- **Keamanan & Integritas:** Dalam sistem yang profesional, penghapusan sering kali dibatasi. Misalnya, sebuah produk tidak boleh dihapus jika sudah pernah dibeli untuk menjaga riwayat transaksi (integritas data).
- **Soft Delete:** Laravel juga menyediakan fitur *Soft Delete*, di mana data tidak benar-benar dihapus dari database, melainkan hanya ditandai sebagai "terhapus" agar bisa dikembalikan (restore) jika terjadi kesalahan.

Ringkasan Pemetaan Metode HTTP ke Operasi Database

Metode HTTP	Operasi CRUD	Perintah SQL Raw
GET	Read	SELECT
POST	Create	INSERT
PUT/PATCH	Update	UPDATE
DELETE	Delete	DELETE

Untuk mencoba metode selain GET (seperti POST, PUT, dan DELETE), kamu sangat disarankan menggunakan aplikasi seperti **Postman**, karena browser biasa secara default hanya bisa melakukan GET dengan mudah melalui bar alamat.

5. Format Data: JSON

API biasanya mengirimkan data dalam format **JSON (JavaScript Object Notation)**. JSON dipilih karena formatnya ringan, berbentuk teks sederhana, dan mudah dimengerti baik oleh manusia maupun mesin. Contoh datanya akan terlihat seperti kumpulan pasangan "kunci" dan "nilai".

JSON (JavaScript Object Notation) adalah sebuah format penulisan data yang terinspirasi dari cara penulisan objek dalam bahasa pemrograman JavaScript. Secara teknis, JSON merupakan format pertukaran data berbasis teks yang ringan dan mudah dibaca baik oleh manusia maupun mesin. Dalam pengembangan dengan Laravel, array PHP yang di-*echo* atau dikembalikan (*return*) dari sebuah rute atau *controller* akan secara otomatis dikonversi oleh sistem menjadi format JSON.

Berikut adalah beberapa alasan utama mengapa JSON sangat sering digunakan dalam pengembangan API:

- **Standar Pertukaran Data Universal:** JSON telah menjadi standar industri sebagai sarana pertukaran data antar sistem yang berbeda, seperti antara server (backend) dan berbagai perangkat klien.
- **Kompatibilitas dengan Teknologi Modern:** Format JSON dimengerti oleh hampir semua teknologi sisi klien saat ini, termasuk aplikasi **mobile (Android/iOS)** serta framework **frontend** modern seperti React, Vue.js, dan Next.js.
- **Mendukung Arsitektur Terpisah (*Decoupled Architecture*):** Tren industri saat ini bergerak menuju arsitektur di mana backend hanya bertindak sebagai penyedia data mentah (format JSON), sementara urusan tampilan atau antarmuka dikelola sepenuhnya oleh aplikasi klien secara terpisah.

- **Ringan dan Performa Tinggi:** Dibandingkan dengan protokol lama seperti SOAP yang menggunakan XML, JSON jauh lebih **ringan, hemat bandwidth, dan memiliki performa yang lebih cepat**.
- **Konsistensi dan Struktur:** Penggunaan JSON memastikan bahwa aplikasi klien menerima data dalam struktur yang **konsisten dan dapat diprediksi**, meskipun terjadi perubahan pada skema basis data di sisi server.
- **Dukungan Penuh Framework:** Laravel 12 menyediakan dukungan bawaan yang sangat kuat untuk respon JSON, memudahkan pengembang untuk mengirimkan data secara terstruktur dengan kode yang minimalis.

HTTP Response

Dalam pengembangan API, HTTP Response adalah pesan yang dikirimkan oleh peladen (server) kembali ke klien setelah menerima dan memproses sebuah permintaan (request). Memahami struktur dan jenis respons ini sangat krusial karena API bertindak sebagai "kontrak" antara sistem, di mana peladen menjamin data dikembalikan dalam format terstruktur, biasanya JSON.

1. HTTP Status Codes (Kode Status)

Kode status memberi tahu klien tentang hasil dari permintaan mereka secara instan. Berikut adalah kategori yang paling sering ditemui dalam API:

1.a 2xx Success (Berhasil):

200 OK: Permintaan berhasil diproses. Digunakan untuk operasi GET (ambil data) atau PUT/PATCH (update data).

201 Created: Sumber daya baru berhasil dibuat. Sangat umum dikembalikan setelah proses registrasi atau menambah data baru (POST).

1.b 4xx Client Errors (Kesalahan Klien):

401 Unauthorized: Klien tidak terautentikasi. Terjadi jika token (Bearer Token) tidak ada, kedaluwarsa, atau tidak valid.

403 Forbidden: Klien terautentikasi tetapi tidak memiliki izin (role) untuk mengakses resource tersebut.

404 Not Found: Resource atau endpoint yang diminta tidak ditemukan di database atau server.

422 Unprocessable Entity: Terjadi kesalahan validasi data. Peladen memahami permintaan tersebut, tetapi input data tidak memenuhi aturan (misal: email sudah terdaftar).

429 Too Many Requests: Klien telah melampaui batas permintaan yang ditentukan oleh sistem Rate Limiting.

1.c 5xx Server Errors (Kesalahan Server):

500 Internal Server Error: Terjadi kesalahan umum pada sisi peladen, seperti kode yang rusak atau database yang mati.

2. Response Body & Standarisasi (API Resources)

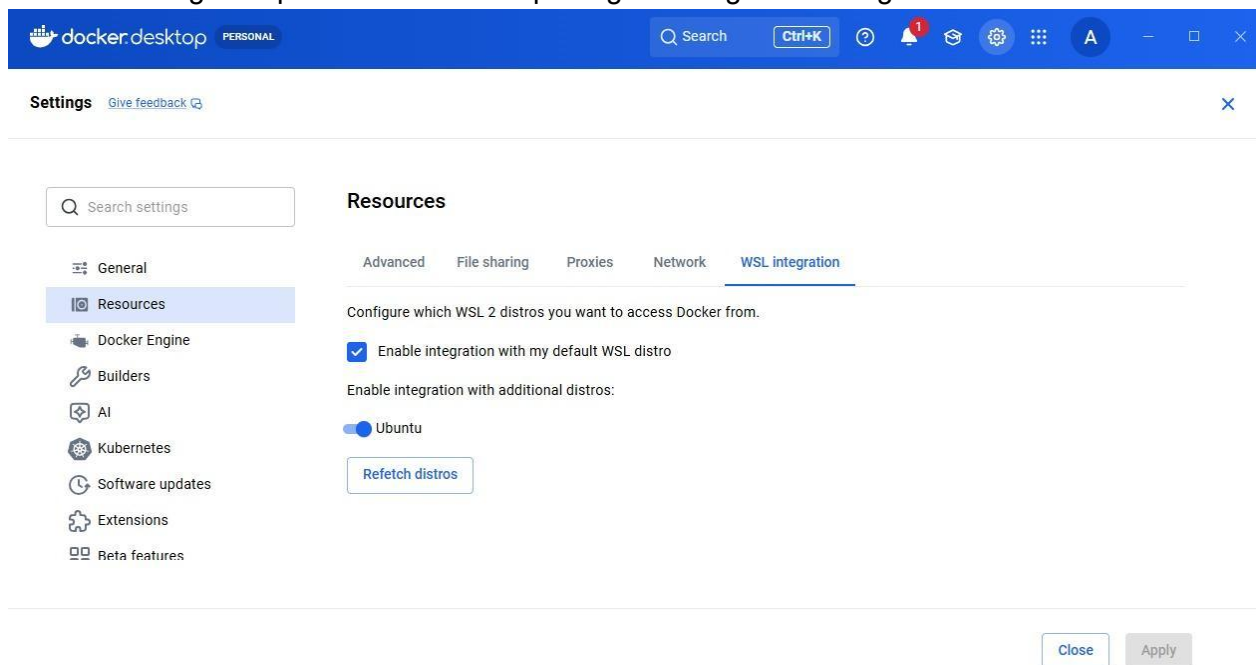
Dalam API modern, isi pesan (body) harus konsisten agar mudah dibaca oleh aplikasi frontend atau mobile.

- **Format JSON:** Data dikirimkan dalam pasangan key-value. Misalnya, sebuah respons sukses akan berisi objek data, sedangkan respons error akan menyertakan pesan kesalahan dan daftar field yang bermasalah.

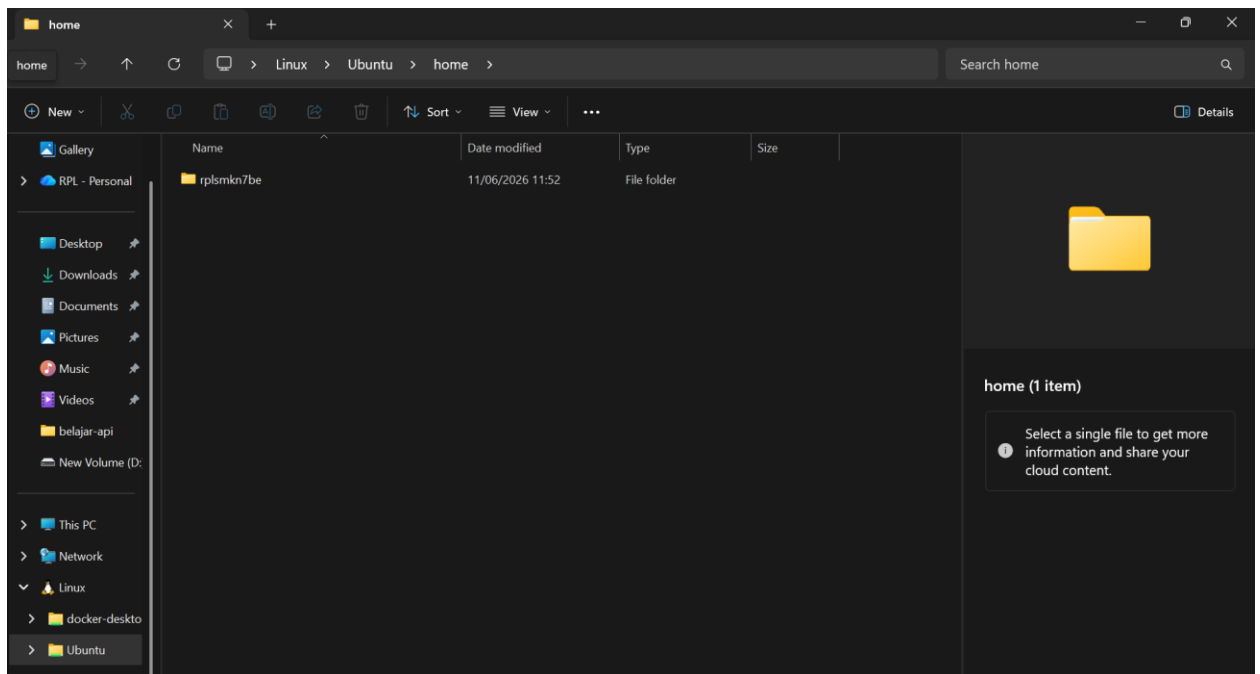
- **API Resources** (JsonResource): Dalam Laravel, komponen ini digunakan untuk melakukan transformasi data sebelum dikirim. Gunanya adalah untuk memastikan struktur output seragam, menyembunyikan data sensitif (seperti password), dan menangani relasi antar data secara efisien.
- **Struktur Error yang Seragam**: Disarankan untuk memiliki skema error yang konsisten, misalnya: {"status": "error", "message": "Failed validation.", "errors": {...}}.

Instalasi dan Konfigurasi Tools

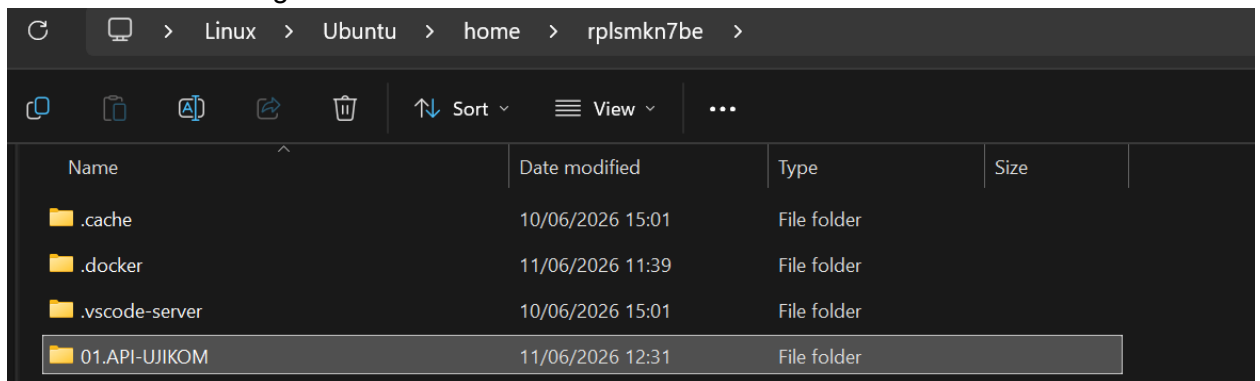
1. Pastikan Docker Desktop sudah terinstall dengan baik pada komputer masing-masing, atau dapat melakukan pengunduhan serta instalasi pada link : <https://docs.docker.com/desktop/setup/install/windows-install/>
2. Docker memerlukan WSL 2 untuk berjalan optimal. Buka Command Prompt (CMD) atau PowerShell sebagai Administrator, lalu jalankan perintah berikut:
wsl --install
3. Lakukan penginstalan ubuntu yang dapat diunduh di microsoft store, lakukan username dan password untuk di ubuntu.
4. Lakukan konfigurasi pada docker desktop dengan konfigurasi sebagai berikut:



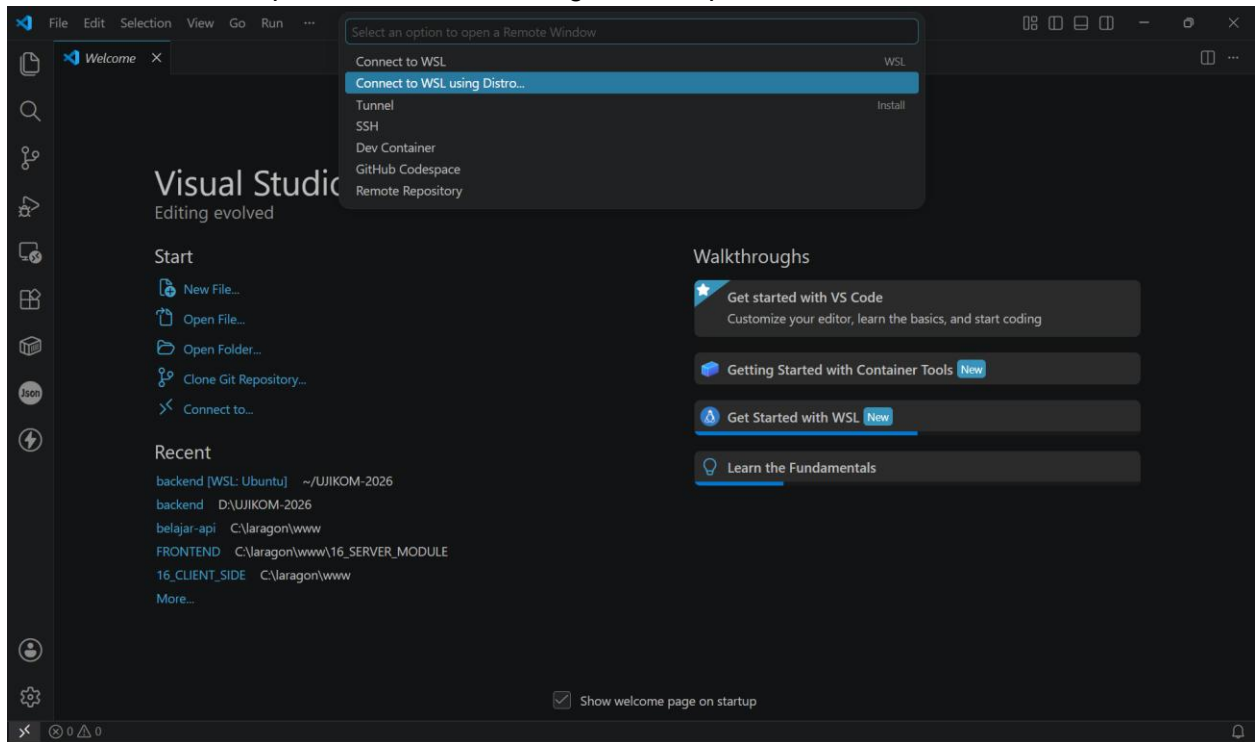
5. Masuk kedalam direktori ubuntu dan user masing-masing seperti:
\\wsl.localhost\Ubuntu\home



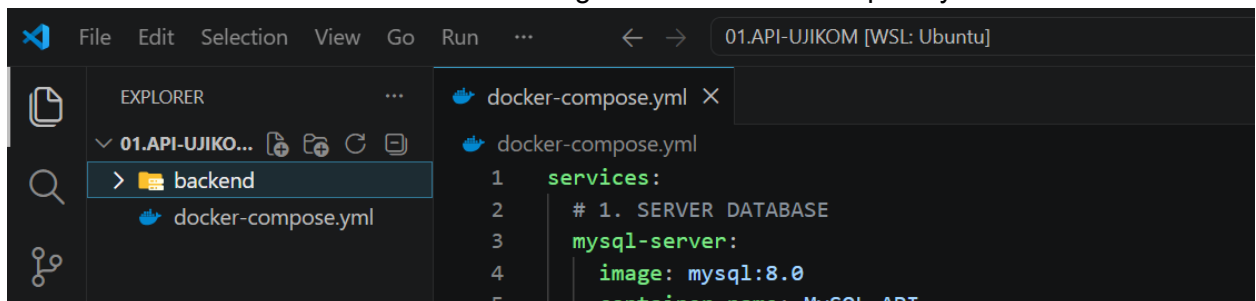
6. Buat folder baru dengan contoh 01.API-UJIKOM



7. Buka VS Code kemudian integrasikan dengan WSL dengan cara memilih icon pojok kiri bawah -> kemudian pilih connect WSL using distro -> pilih Ubuntu:



8. Buka folder 01.API-UJIKOM -> buat file dengan nama docker-compose.yml



9. Masukkan baris kode berikut pada docker-compose.yml

```
services:
  # 1. SERVER DATABASE
  mysql-server:
    image: mysql:8.0
    container_name: MySQL-API
    environment:
      MYSQL_DATABASE: API_ujikom
      MYSQL_USER: api_ujikom
      MYSQL_PASSWORD: api_ujikom
      MYSQL_ROOT_PASSWORD: passwordroot
    ports:
      - "3306:3306"
```

```

networks:
  - API-UJIKOM
volumes:
  - db_data:/var/lib/mysql

# 2. SERVER BACKEND (LARAVEL)
app-laravel:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: laravel-api
  working_dir: /var/www
  ports:
    - "8000:8000"
  volumes:
    - ./backend:/var/www
  command: php artisan serve --host=0.0.0.0 --port=8000
  depends_on:
    - mysql-server
  networks:
    - API-UJIKOM

# 3. SERVER UI DATABASE (PHPMYADMIN)
phpMyadmin:
  image: phpmyadmin
  container_name: PhpMyadmin-API
  environment:
    PMA_HOST: mysql-server
  ports:
    - "8081:80"
  depends_on:
    - mysql-server
  networks:
    - API-UJIKOM

volumes:
  db_data:
networks:
  API-UJIKOM:
    driver: bridge

```

Penjelasan File/Baris Kode Program:

File/Baris Code	Fungsi
Docker-Compose.yml	sebagai cetak biru (blueprint) berbasis YAML untuk mendefinisikan dan menjalankan aplikasi multi-kontainer Docker. File ini menyatukan pengaturan seluruh layanan, jaringan (network), dan volume penyimpanan dalam satu tempat sehingga mempermudah proses konfigurasi dan manajemen aplikasi
services:	Blok utama yang memberi tahu Docker Compose bahwa baris-baris di bawahnya adalah daftar kontainer (layanan) yang akan dibuat
mysql-server:	Nama layanan internal di dalam Docker Compose. Nama ini bebas Anda tentukan dan berfungsi sebagai nama host jika kontainer lain ingin terhubung ke database ini.
image: mysql:8.0	Mengunduh dan menggunakan image resmi MySQL versi 8.0 dari Docker Hub sebagai dasar pembuatan kontainer.
container_name: MySQL-API	Memberikan nama spesifik MySQL-API pada kontainer saat berjalan. Jika tidak diatur, Docker akan memberikan nama acak secara otomatis.
environment:	Blok untuk mengisi variabel lingkungan yang dibutuhkan oleh image MySQL untuk konfigurasi awal saat kontainer pertama kali dinyalakan.
ports:	Blok untuk memetakan jalur komunikasi (port) antara komputer fisik Anda (host) dan kontainer
networks:	Blok untuk menentukan jaringan isolasi tempat kontainer ini berada.
volumes:	Blok untuk mengatur penyimpanan data persisten agar data tidak hilang saat kontainer dimatikan atau dihapus
build:	Memberi tahu Docker Compose untuk membuat (<i>build</i>) <i>image</i> kustom sendiri menggunakan Dockerfile lokal
context: ./backend	Menentukan lokasi folder utama (konteks) tempat kode sumber Laravel Anda berada, yaitu di dalam folder bernama backend di direktori yang sama dengan file docker-compose ini
dockerfile: Dockerfile	Menentukan nama file instruksi Docker yang akan dibaca di dalam folder ./backend tersebut, yaitu file bernama Dockerfile
working_dir: /var/www	Menetapkan folder kerja utama di dalam kontainer. Semua perintah terminal (seperti perintah di baris command) akan dieksekusi di dalam folder /var/www
command: php artisan serve --host=0.0.0.0 --port=8000	Perintah utama yang otomatis dijalankan saat kontainer menyala. Perintah ini mengaktifkan server bawaan Laravel pada port 8000. Parameter --host=0.0.0.0 wajib digunakan agar server Laravel mau menerima koneksi dari luar kontainer

depends_on:	Blok untuk mengatur urutan jalannya kontainer berdasarkan ketergantungan antar layanan.
-------------	---

10. Buka terminal pada VS Code kemudian ketikkan perintah **docker run --rm -u "\$(id -u):\$(id -g)" -v "\${PWD}:/app" -w /app composer create-project laravel/laravel:^12 backend** untuk melakukan instalasi laravel12 pada folder backend.

The screenshot shows the VS Code interface with a file explorer on the left showing a project structure with a 'backend' folder. The main editor displays a 'docker-compose.yml' file with the following content:

```

1 services:
2   # 1. SERVER DATABASE
3   mysql-server:
4     image: mysql:8.0
5     container_name: MySQL-API
6     environment:
7       MYSQL_DATABASE: API-UJIKOM

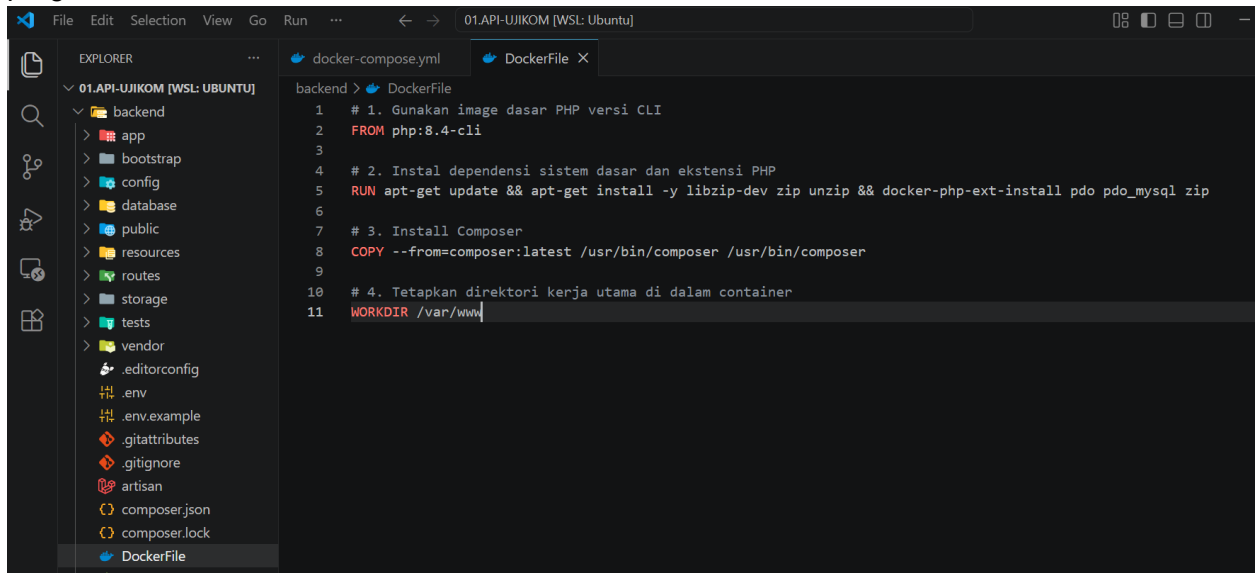
```

The terminal window at the bottom shows the command being executed: `docker run --rm -u "$(id -u):$(id -g)" -v "${PWD}:/app" -w /app composer create-project laravel/laravel:^12 backend`. The output shows the command running successfully, with a message "Discovering packages." and a "DONE" status.

Penjelasan Baris Kode Program

docker run	Instruksi utama untuk membuat dan menjalankan sebuah kontainer Docker baru berdasarkan image tertentu.
--rm	Menghapus kontainer secara otomatis setelah perintah selesai dijalankan. Ini menjaga komputer Anda tetap bersih dari kontainer bekas yang sudah tidak terpakai
-u "\$(id -u):\$(id -g)"	Mengatur hak akses (user dan group) di dalam kontainer agar sama dengan pengguna komputer Anda. Ini sangat penting agar folder dan file proyek Laravel yang terbentuk nanti tidak terkunci sebagai milik root, sehingga Anda bisa bebas mengedit atau menghapusnya tanpa masalah izin (permission denied).
-v "\${PWD}:/app"	Melakukan volume binding (sinkronisasi folder). Bagian ini menghubungkan folder aktif saat ini di komputer Anda (\${PWD}) ke dalam folder /app di dalam kontainer. Apa pun yang dibuat di folder /app kontainer akan langsung muncul di komputer Anda.
-w /app	Menentukan working directory (direktori kerja). Perintah ini memberi tahu Docker untuk langsung masuk dan mengeksekusi perintah selanjutnya di dalam folder /app.
composer	Nama image Docker resmi yang digunakan, yang sudah berisi PHP dan tools Composer di dalamnya.
create-project laravel/laravel:^12 backend	Perintah internal Composer yang dijalankan di dalam kontainer untuk mengunduh skrip utama Laravel versi 12 dan menyimpannya ke dalam folder baru bernama backend.

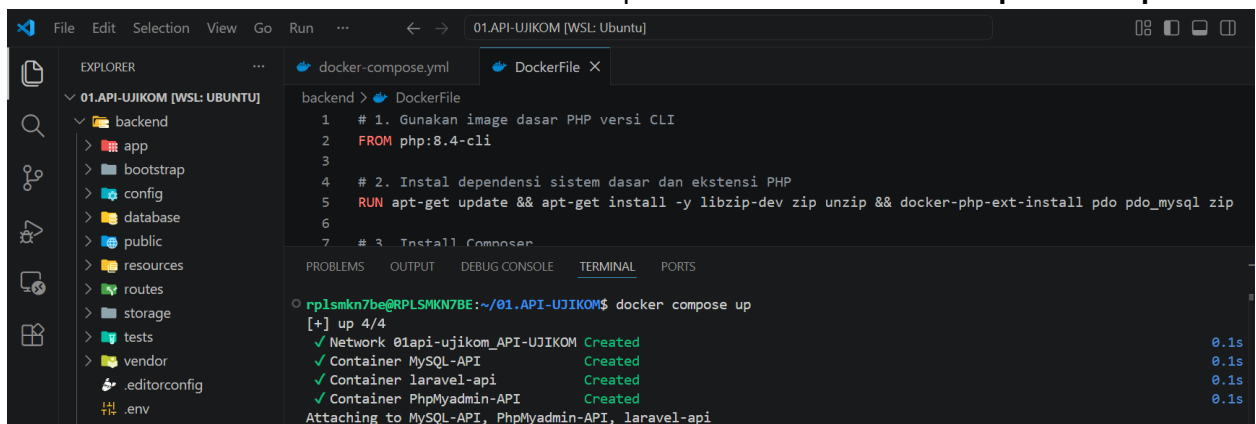
11. Buat file baru dengan nama **Dockerfile** pada folder backend kemudian ketikan baris program berikut:



The screenshot shows the Visual Studio Code interface with the Explorer on the left showing the file structure of a project named '01.API-UJIKOM [WSL: UBUNTU]'. The 'backend' folder is expanded, and a new file named 'DockerFile' has been created. The Dockerfile content is as follows:

```
1 # 1. Gunakan image dasar PHP versi CLI
2 FROM php:8.4-cli
3
4 # 2. Instal dependensi sistem dasar dan ekstensi PHP
5 RUN apt-get update && apt-get install -y libzip-dev zip unzip && docker-php-ext-install pdo pdo_mysql zip
6
7 # 3. Install Composer
8 COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
9
10 # 4. Tetapkan direktori kerja utama di dalam container
11 WORKDIR /var/www
```

12. Buka terminal kemudian ketikan perintah **docker compose up**



The screenshot shows the Visual Studio Code interface with the Dockerfile content in the backend folder. The terminal window at the bottom shows the command 'docker compose up' being executed, resulting in the following output:

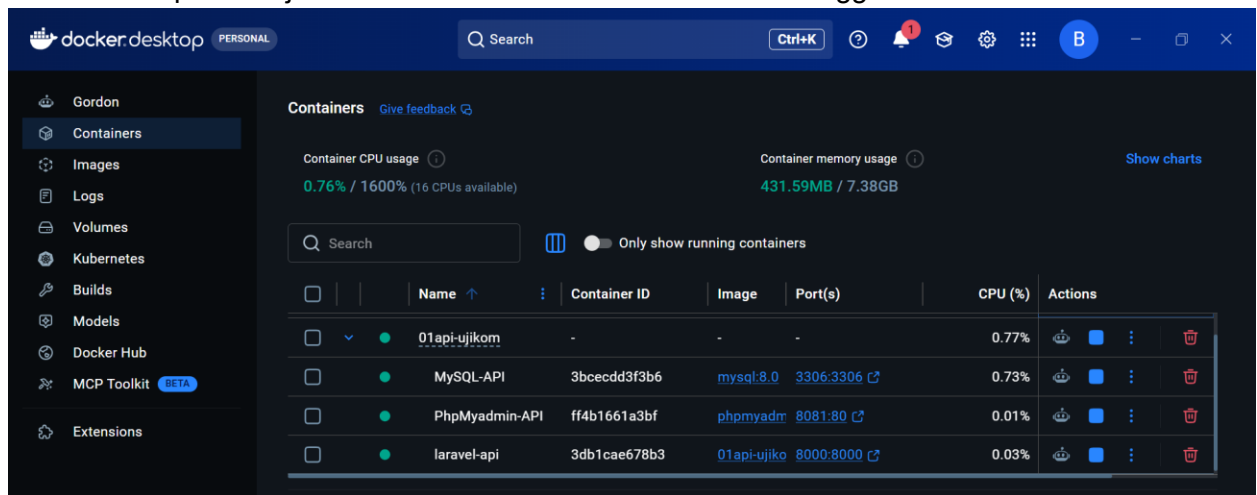
```
rp1smkn7be@RPLSMKN7BE:~/01.API-UJIKOM$ docker compose up
[+] up 4/4
✓ Network 01api-ujikom_API-UJIKOM Created 0.1s
✓ Container MySQL-API Created 0.1s
✓ Container laravel-api Created 0.1s
✓ Container PhpMyadmin-API Created 0.1s
Attaching to MySQL-API, PhpMyadmin-API, laravel-api
```

Penjelasan File / Baris Kode Program

File/Baris Program	Fungsi
DockerFile	sebuah file teks tanpa ekstensi yang berfungsi sebagai skrip otomatisasi. File ini berisi kumpulan instruksi langkah-demi-langkah bagi Docker Engine untuk membangun (build) sebuah Docker Image, mulai dari sistem operasi dasar, dependensi, hingga kode aplikasi itu sendiri.
FROM:	Menentukan base image atau sistem operasi awal (misalnya, FROM php:8.4-cli atau FROM node:18).
RUN:	Menjalankan perintah terminal saat image sedang dibuat (seperti RUN npm install atau RUN apt-get update
COPY & ADD:	Menyalin file atau folder dari komputer lokal ke dalam container.

WORKDIR:	Menentukan direktori kerja utama di dalam container.
docker compose up	Menginstruksikan Docker untuk membaca file docker-compose.yml, lalu mengunduh image, membuat kontainer, membuat jaringan (network), dan menjalankan semua layanan yang tertulis di dalamnya.

13. Tunggu sampai proses selesai kemudian periksa pada Docker Desktop, jika tampilan seperti gambar dibawah maka kalian sudah berhasil melakukan instalasi dan konfigurasi untuk pembelajaran API Laravel menggunakan Docker.



Konfigurasi Laravel

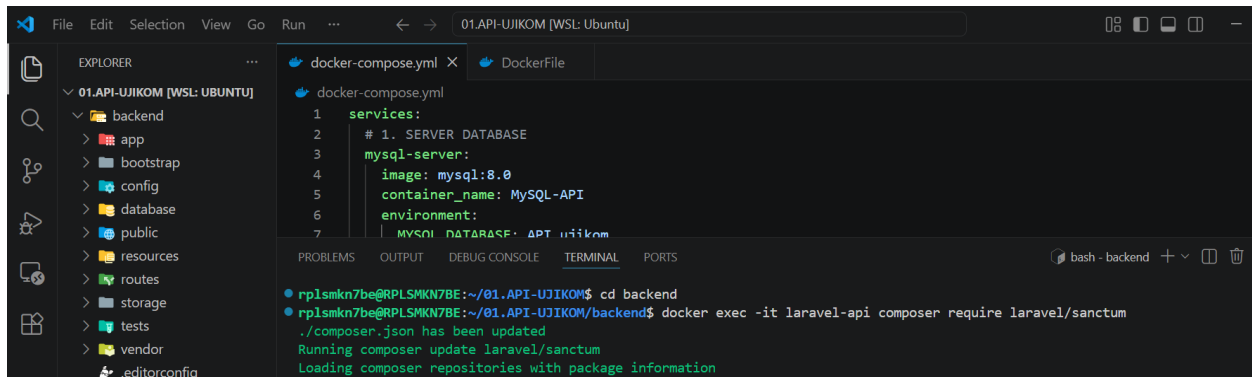
Penyesuaian pada file .env

Buka file .env kemudian lakukan perubahan pada koneksi database (sesuaikan pada isi pada file docker-compose.yml) dan session seperti berikut:

```
DB_CONNECTION=mysql
DB_HOST=mysql-server
DB_PORT=3306
DB_DATABASE=API_ujikom
DB_USERNAME=api_ujikom
DB_PASSWORD=api_ujikom
```

Install Package Sanctum via Composer

Masuk kedalam direktori backend kemudian jalankan perintah ini untuk mengunduh Sanctum ke dalam project dengan mengetikan perintah: **docker exec -it laravel-api composer require laravel/sanctum**



```
File Edit Selection View Go Run ... 01.API-UJIKOM [WSL: UBUNTU]
EXPLORER
01.API-UJIKOM [WSL: UBUNTU]
  backend
    app
    bootstrap
    config
    database
    public
    resources
    routes
    storage
    tests
    vendor
    .editorconfig
docker-compose.yml X DockerFile
docker-compose.yml
1 services:
2 # 1. SERVER DATABASE
3 mysql-server:
4 image: mysql:8.0
5 container_name: MySQL-API
6 environment:
7 MYSQL_DATABASE= API_ujikom
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - backend + -
rplsmkn7be@RPLSMKN7BE:~/01.API-UJIKOM$ cd backend
rplsmkn7be@RPLSMKN7BE:~/01.API-UJIKOM/backend$ docker exec -it laravel-api composer require laravel/sanctum
./composer.json has been updated
Running composer update laravel/sanctum
Loading composer repositories with package information
```

Instalasi Ekosistem API

Perintah ini akan mempublikasikan file konfigurasi Sanctum dan membuat file routes/api.php secara otomatis. (Jika muncul prompt konfirmasi di terminal, ketik yes atau tekan Enter). Dengan

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - backend + ▢ ▢ ▢
rplsmkn7be@RPLSMKN7BE:~/01.API-UJIKOM/backend$ docker exec -it laravel-api php artisan install:api
INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.

INFO Published API routes file.

One new database migration has been published. Would you like to run all pending database migrations? (yes/no) [yes]:
> yes

INFO Preparing database.

Creating migration table ..... 80.71ms DONE
```

Package ini memungkinkan aplikasi Anda mengonversi kode HTML/Blade menjadi dokumen PDF dengan mudah, yang sangat berguna untuk fitur cetak laporan atau export data. Dengan mengetikkan perintah **docker exec -it laravel-api composer require barryvdh/laravel-dompdf** pada terminal

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - backend + v [ ] [X] [ ] [X] [X]
```

```
rpismkn7be@RPLSMKN7BE:/~/API-UJIKOM/backend$ docker exec -it laravel-api composer require barryvdh/laravel-dompdf  
Running composer update barryvdh/laravel-dompdf  
Loading composer repositories with package information  
Updating dependencies  
Lock file operations: 7 installs, 0 updates, 0 removals  
- Locking barryvdh/laravel-dompdf (v3.1.2)  
- Locking dompdf/dompdf (v3.1.5)
```

Secara default, file yang diunggah menggunakan sistem penyimpanan lokal Laravel tersimpan di dalam folder *storage/app/public* yang bersifat privat. Agar file tersebut dapat ditampilkan di website melalui URL, Laravel harus dapat mengaksesnya melalui folder public.

Karena kita menggunakan docker maka perintah yang harus diketikkan pada terminal adalah **docker exec -it laravel-api php artisan storage:link**

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - backend + - [ ] [ ] ...
rplsmkn7be@RPLSMKN7BE:~/01.API-UJIKOM/backend$ docker exec -it laravel-api php artisan storage:link
INFO The [public/storage] link has been connected to [storage/app/public].
rplsmkn7be@RPLSMKN7BE:~/01.API-UJIKOM/backend$
```

Migrations

Materi Singkat

Migration adalah fitur Laravel untuk membangun dan mengelola tabel database dengan cepat menggunakan kode PHP, yang bertindak seperti **version control** untuk database kamu.

Perintah Dasar Membuat Migration : Untuk membuat "cetak biru" sebuah tabel, kamu harus menjalankan perintah Artisan di terminal.

- **Format:** php artisan make:migration create_nama_table_table.
- **Contoh:** Untuk membuat tabel produk, jalankan php artisan make:migration create_produk_table. Jika menggunakan **Docker** perintahnya menjadi **docker exec -it laravel-api php artisan make:migration create_produk_table**.
- Hasilnya, file baru akan muncul di folder **database/migrations/** dengan nama yang diawali tanggal (timestamp) pembuatan.

Struktur File Migration

Setiap file migration memiliki dua bagian utama di dalam class-nya:

- **Method up():** Berisi perintah untuk **membuat** atau mengubah struktur tabel.
- **Method down():** Berisi perintah kebalikan dari up(), biasanya untuk **menghapus** tabel jika terjadi kesalahan (rollback).

Cara Menulis Kolom Tabel

Di dalam method up(), kamu akan menggunakan objek \$table untuk mendefinisikan kolom. Berikut adalah aturan penulisan berdasarkan contoh modul:

- **Tipe Data String:** \$table->string('nama_kolom', panjang).
 - Contoh: \$table->string('id_produk', 5)->primary(); membuat kolom VARCHAR(5) sekaligus menjadi **Primary Key**.
- **Nullable (Boleh Kosong):** Jika di SQL tertulis DEFAULT NULL, di Laravel gunakan ->nullable().
 - Contoh: \$table->string('nm_produk', 30)->nullable();.
- **Tipe Angka & Desimal:**
 - \$table->integer('nama'); untuk angka bulat.
 - \$table->decimal('nama', total_digit, digit_belakang_koma); untuk harga atau uang.
 - Contoh: \$table->decimal('harga', 10, 0)->nullable();.
- **Otomatisasi Waktu:** \$table->timestamps(); akan otomatis membuat kolom created_at dan updated_at untuk mencatat waktu data dibuat/diubah.

Membuat Relasi (Foreign Key)

Pada tabel pesan dan detail_pesan, kamu harus menghubungkan satu tabel dengan tabel lainnya.

- **Contoh Tabel Pesan:** Kamu perlu mencatat id_pelanggan agar tahu siapa yang memesan.

- **Kodenya:** `$table->foreign('id_pelanggan')->references('id_pelanggan')->on('pelanggan')->onDelete('cascade');`
- Artinya, kolom `id_pelanggan` di tabel ini merujuk ke tabel pelanggan. Jika data pelanggan dihapus, maka data pesanan milik pelanggan tersebut juga otomatis terhapus (cascade)

Composite Key (Kunci Gabungan)

Khusus pada tabel `detil_pesan`, tidak ada satu ID tunggal. Kunci utamanya adalah gabungan dari `id_pesan` dan `id_produk`.

- **Kodenya:** `$table->primary(['id_pesan', 'id_produk']);`

Menjalankan Rencana (Migrate)

Setelah semua file migration selesai kamu edit dan simpan, jalankan perintah ini agar tabel benar-benar muncul di database (MySQL/MariaDB):

- `php artisan migrate.`

Tips Tambahan:

- Jika kamu melakukan kesalahan ketik pada kolom dan ingin mengulang dari nol, gunakan perintah `php artisan migrate:fresh` (Hati-hati: ini akan menghapus semua tabel dan membuatnya kembali dari awal).

Perbedaan antara migration up() dan migration down()

1. Method up() (Menjalankan Perubahan)

Method ini berisi perintah untuk membuat atau mengubah struktur tabel di database kamu.

- Fungsi: Digunakan saat kamu menjalankan perintah `php artisan migrate`. Di sinilah kamu mendefinisikan nama tabel, nama kolom, serta tipe datanya.
- Contoh Isi: Perintah `Schema::create('users', ...)` untuk membuat tabel baru atau `Schema::table(..., ...)` untuk memodifikasi tabel yang sudah ada.

2. Method down() (Membatalkan Perubahan)

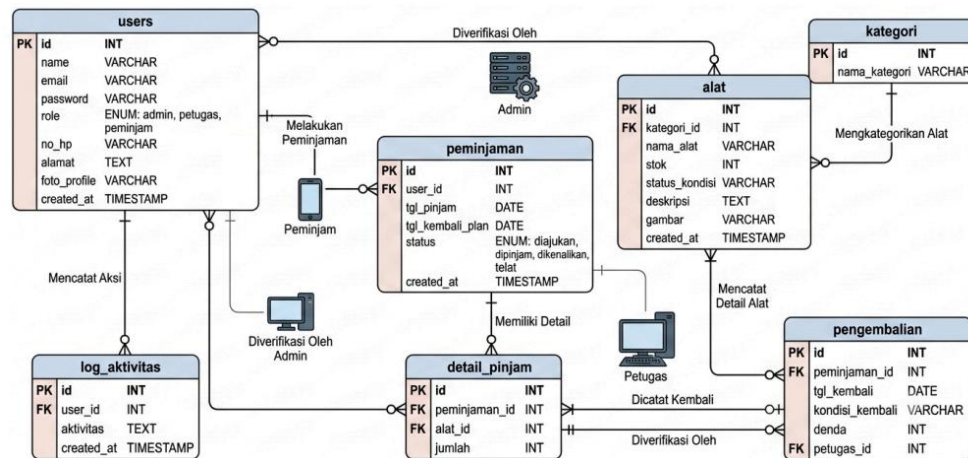
Method ini berisi perintah yang merupakan kebalikan (reverse) dari apa yang dilakukan oleh method `up()`.

- Fungsi: Digunakan saat kamu melakukan *rollback* (menarik mundur perubahan) melalui perintah seperti `php artisan migrate:rollback`. Tujuannya adalah untuk mengembalikan kondisi database ke posisi sebelum migration tersebut dijalankan.
- Contoh Isi: Jika di method `up()` kamu membuat tabel 'users', maka di method `down()` kamu menulis perintah `Schema::dropIfExists('users')` untuk menghapus tabel tersebut.

Mengapa Harus Ada Keduanya?

Laravel menggunakan sistem ini agar migration dapat berfungsi sebagai version control untuk database kamu. Dengan adanya pasangan `up()` dan `down()`, tim pengembang bisa dengan mudah "maju-mundur" dalam sejarah struktur database tanpa perlu menghapus data atau tabel secara manual di phpMyAdmin.

Kegiatan Praktikum



Skema Database Aplikasi Peminjaman Alat

1. Update file create_users_table.php

File migration **create_users_table.php** biasanya sudah terbuat secara otomatis ketika kita melakukan migrasi pertama kali jadi kita akan mengupdate file tersebut dengan mengubahnya menjadi seperti ini:

```
0001_01_01_000000_create_users_table.php X
database > migrations > 0001_01_01_000000_create_users_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7
8  return new class extends Migration
9  {
10     public function up(): void {
11         Schema::create('users', function (Blueprint $table) {
12             $table->id();
13             $table->string('name');
14             $table->string('email')->unique();
15             $table->string('password');
16             $table->enum('role', ['admin', 'petugas', 'peminjam'])->default('peminjam');
17             $table->string('no_hp')->nullable();
18             $table->text('alamat')->nullable();
19             $table->string('foto_profile')->nullable();
20             $table->timestamps();
21         });
22     }
23
24     public function down(): void {
25         Schema::dropIfExists('users');
26     }
27 };
```

2. 02_create_kategori_table.php

Selanjutnya kita akan membuat struktur tabel kategori dengan mengetikkan perintah pada terminal : **docker exec -it laravel-api php artisan make:migration**

02_create_kategori_table.php

Kemudian lakukan pengisian baris kode program seperti ini:

```
2026_06_10_051545_02_create_kategori_table.php X
database > migrations > 2026_06_10_051545_02_create_kategori_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void {
10         Schema::create('kategori', function (Blueprint $table) {
11             $table->id();
12             $table->string('nama_kategori');
13             $table->timestamps();
14         });
15     }
16
17     public function down(): void {
18         Schema::dropIfExists('kategori');
19     }
20 };
```

3. 03_create_alat_table.php

Selanjutnya kita akan membuat struktur tabel alat dengan mengetikkan perintah pada terminal : **docker exec -it laravel-api php artisan make:migration 03_create_alat_table.php**

Lakukan perubahan baris kode pada blok schema menjadi :

```
public function up(): void {
    Schema::create('alat', function (Blueprint $table) {
        $table->id();
        $table->foreignId('kategori_id')->constrained('kategori')->cascadeOnDelete();
        $table->string('nama_alat');
        $table->integer('stok')->default(0);
        $table->string('status_kondisi');
        $table->text('deskripsi')->nullable();
        $table->string('gambar')->nullable();
        $table->timestamps();
    });
}
```

4. 04_create_peminjaman_table.php

Selanjutnya kita akan membuat struktur tabel peminjaman dengan mengetikkan perintah pada terminal : **docker exec -it laravel-api php artisan make:migration**

04_create_peminjaman_table.php

Lakukan perubahan baris kode pada blok schema menjadi :

```
public function up(): void {
    Schema::create('peminjaman', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained('users')->cascadeOnDelete();
        $table->date('tgl_pinjam');
        $table->date('tgl_kembali_plan');
        $table->enum('status', ['diajukan', 'dipinjam', 'dikembalikan', 'telat'])->default('diajukan');
        $table->timestamps();
    });
}
```

5. 05_create_detail_pinjam_table.php

Selanjutnya kita akan membuat struktur tabel detail peminjaman dengan mengetikkan perintah pada terminal : **docker exec -it laravel-api php artisan make:migration**

05_create_detail_pinjam_table.php

Lakukan perubahan baris kode pada blok schema menjadi :

```
public function up(): void {
    Schema::create('detail_pinjam', function (Blueprint $table) {
        $table->id();
        $table->foreignId('peminjaman_id')->constrained('peminjaman')->cascadeOnDelete();
        $table->foreignId('alat_id')->constrained('alat')->cascadeOnDelete();
        $table->integer('jumlah')->default(1);
        $table->timestamps();
    });
}
```

6. 06_create_pengembalian_table.php

Selanjutnya kita akan membuat struktur tabel pengembalian dengan mengetikkan perintah pada terminal : **docker exec -it laravel-api php artisan make:migration**

06_create_pengembalian_table.php

Lakukan perubahan baris kode pada blok schema menjadi :

```
public function up(): void {
    Schema::create('pengembalian', function (Blueprint $table) {
        $table->id();
        $table->foreignId('peminjaman_id')->constrained('peminjaman')->cascadeOnDelete();
        $table->date('tgl_kembali');
        $table->string('kondisi_kembali');
        $table->integer('denda')->default(0);
        $table->foreignId('petugas_id')->constrained('users')->cascadeOnDelete();
        $table->timestamps();
    });
}
```

7. 07_create_log_aktivitas_table.php

Selanjutnya kita akan membuat struktur tabel pengembalian dengan mengetikkan perintah pada terminal : **docker exec -it laravel-api php artisan make:migration**

07_create_log_aktivitas_table.php

Lakukan perubahan baris kode pada blok schema menjadi :

```
public function up(): void {
    Schema::create('log_aktivitas', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained('users')->cascadeOnDelete();
        $table->text('aktivitas');
        $table->timestamps();
    });
}
```

Penjelasan File/Baris Program

\$table->string('nama_kategori')	untuk membuat sebuah kolom baru bernama nama_kategori dengan tipe data teks (VARCHAR) di dalam tabel database.
\$table	Sebuah objek (blueprint) yang digunakan Laravel untuk mendefinisikan struktur tabel baru atau mengubah tabel yang sudah ada.
->string(...)	Metode bawaan Laravel untuk menentukan tipe data kolom sebagai VARCHAR. Secara default, Laravel akan mengatur panjang maksimal kolom ini sebesar 255 karakter
'nama_kategori'	Nama dari kolom yang akan dibuat di dalam tabel database

\$table->foreignId('kategori_id')->constrained('kategori')->cascadeOnDelete();	berfungsi untuk membuat kolom kunci tamu (foreign key) yang menghubungkan tabel saat ini dengan tabel kategori, serta mengatur penghapusan data secara otomatis
\$table->foreignId('kategori_id')	Metode foreignId secara otomatis menetapkan tipe data BIGINT UNSIGNED, yang merupakan standar Laravel untuk menghubungkan ID antar tabel
->constrained('kategori')	Menegaskan bahwa kolom kategori_id adalah sebuah foreign key yang wajib merujuk (terkoneksi) ke kolom id di dalam tabel kategori Laravel Foreign Key Relations
->cascadeOnDelete()	Mengaktifkan fitur ON DELETE CASCADE. Jika sebuah data kategori di tabel kategori dihapus, maka seluruh data di tabel saat ini yang terhubung dengan kategori_id tersebut akan ikut terhapus otomatis oleh database demi menjaga konsistensi data
\$table->integer('stok')->default(0);	berfungsi untuk membuat sebuah kolom baru bernama stok dengan tipe data angka bulat (INTEGER) dan memberikan nilai awal otomatis sebesar 0 jika data baru dimasukkan tanpa mengisi kolom tersebut
->default(0)	Jika sistem menyimpan data produk baru ke database tetapi Anda tidak mengisi jumlah stoknya, database akan otomatis mengisinya dengan angka 0, bukan kosong (NULL)
\$table->text('deskripsi')->nullable();	berfungsi untuk membuat sebuah kolom baru bernama deskripsi dengan tipe data teks panjang (TEXT) yang sifatnya opsional (boleh dikosongkan)
\$table->enum('status', ['diajukan', 'dipinjam', 'dikembalikan', 'telat'])->default('diajukan');	berfungsi untuk membuat kolom bernama status dengan tipe data ENUM, di mana pilihan nilainya dibatasi hanya pada 4 opsi yang telah ditentukan, dan otomatis bernilai 'diajukan' jika tidak diisi.

8. Jalankan File Migrasi

Setelah kita membuat semua struktur tabel dengan benar, langkah selanjutnya adalah menjalankan file migrasi dengan perintah : **docker exec -it laravel-api php artisan Migrate**

Lakukan pengecekan pada container PhpMyadmin, apakah struktur database/tabel yang telah kita buat sudah sesuai dengan soal.



Models

Materi Singkat

Model adalah komponen yang bertanggung jawab penuh atas pengelolaan data dan interaksi dengan basis data. Di dalam Laravel, Model bertindak sebagai "**Gudang Data**" yang mengetahui cara mengambil, menyimpan, dan menghapus barang (data) dari rak-rak gudang (tabel MySQL).

Dasar Pembuatan Model

Secara standar, Laravel menggunakan aturan penamaan bahasa Inggris (Model tunggal, Tabel jamak). Namun, karena tabel dalam proyek ini menggunakan bahasa Indonesia dan Primary Key yang spesifik, kita harus melakukan penyesuaian manual di dalam file Model yang terletak di folder **app/Models/**.

Untuk membuat file Model, jalankan perintah berikut di terminal:

- **Perintah:** `php artisan make:model NamaModel`. Jika menggunakan Docker perintahnya menjadi : `docker exec -it laravel-api php artisan make:model NamaModel`

Relasi antar tabel dalam Laravel

Relasi antar tabel dalam Laravel dikelola menggunakan fitur yang disebut **Eloquent Relationships**. Fitur ini memungkinkan kamu menghubungkan data antar tabel melalui model PHP tanpa harus menulis query SQL JOIN yang rumit secara manual.

Berikut adalah cara melakukan relasi **One-to-Many** dan **Many-to-Many** pada model Laravel :

1. Relasi One-to-Many (Satu-ke-Banyak)

Relasi ini terjadi jika **satu record** di tabel induk (misalnya Pelanggan) bisa memiliki **banyak record** di tabel anak (misalnya Pesanan).

- **Langkah di Model Induk (Parent):** Gunakan method `hasMany()`.
 - *Contoh:* Pada model Pelanggan, kamu mendefinisikan bahwa satu pelanggan memiliki banyak pesanan.
- **Langkah di Model Anak (Child):** Gunakan method `belongsTo()`.
 - *Contoh:* Pada model Pesan, kamu mendefinisikan bahwa satu pesanan dimiliki oleh satu pelanggan.

2. Relasi Many-to-Many (Banyak-ke-Banyak)

Relasi ini terjadi jika **banyak record** di tabel A bisa memiliki **banyak record** di tabel B. Relasi jenis ini membutuhkan sebuah **tabel perantara (pivot table)** untuk menghubungkan keduanya.

- **Metode yang Digunakan:** Gunakan method `belongsToMany()` di **kedua model** yang bersangkutan.

Kegiatan Praktikum

1. Model User

Untuk model user kita tidak akan membuat model baru, tapi kita akan mengupdate isi dari file `app\model\user.php` menjadi seperti ini:

```
<?php

namespace App\Models;

use Laravel\Sanctum\HasApiTokens;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Illuminate\Database\Eloquent\Relations\HasMany;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;

    protected $table = 'users';

    protected $fillable = [
        'name', 'email', 'password', 'role', 'no_hp', 'alamat',
        'foto_profile'
    ];

    protected $hidden = [
        'password', 'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'password' => 'hashed',
        ];
    }

    public function peminjaman(): HasMany {
        return $this->hasMany(Peminjaman::class);
    }
}
```

```

public function logAktivitas(): HasMany {
    return $this->hasMany(LogAktivitas::class);
}
}

```

Penjelasan Baris Kode Program

namespace App\Models;	berfungsi sebagai alamat pengenalan digital (ruang nama) yang menentukan lokasi pengelompokan file model tersebut di dalam struktur aplikasi Laravel.
use Laravel\Sanctum\HasApiTokens;	digunakan untuk mengimpor trait otentikasi API yang disediakan oleh Laravel Sanctum
use Illuminate\Foundation\Auth\User as Authenticatable;	berfungsi untuk mengimpor sistem keamanan dan otentikasi bawaan Laravel ke dalam file model
use Illuminate\Notifications\Notifiable;	berfungsi untuk mengimpor trait sistem notifikasi bawaan Laravel ke dalam file model
use Illuminate\Database\Eloquent\Relations\HasMany;	berfungsi untuk mengimpor kelas relasi satu-ke-banyak (One-to-Many) bawaan Laravel
protected \$table = 'users';	berfungsi untuk menentukan nama tabel basis data (database) secara manual yang terhubung dengan model tersebut
protected \$fillable = [...]	berfungsi untuk mendaftarkan nama-nama kolom tabel yang boleh diisi secara massal (Mass Assignment) di dalam aplikasi Laravel.
protected \$hidden = [...]	berfungsi untuk menyembunyikan kolom tertentu agar tidak ikut tampil saat data model diubah menjadi format JSON atau Array.
protected function casts(): array{...}	berfungsi untuk mengonversi tipe data kolom secara otomatis (Attribute Casting) saat data disimpan ke atau diambil dari basis data.
'password' => 'hashed'	fungsi ini bertugas mengamankan kata sandi secara otomatis dengan cara menyandikannya (hashing) menggunakan algoritma aman (seperti Bcrypt) sesaat sebelum data disimpan ke database
public function peminjaman(): HasMany { ... }	digunakan untuk mendefinisikan relasi satu-ke-banyak (One-to-Many) antar model di Laravel
return \$this->hasMany(...)	Perintah untuk mengaktifkan hubungan "saya memiliki banyak data di...".

2. Model Kategori

Selanjutnya kita akan membuat file model kategori dengan mengetikkan perintah pada terminal :
docker exec -it laravel-api php artisan make:model Kategori.php

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
abdulkarim@DESKTOP-D043LCG:~/UJIKOM-2026/backend$ docker exec -it laravel-ujikom php artisan make:model Kategori.php
```

Kemudian isikan baris kode berikut:

```
Kategori.php X
app > Models > Kategori.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\HasMany;
7
8  class Kategori extends Model
9  {
10     protected $table = 'kategori';
11     protected $fillable = ['nama_kategori'];
12
13     public function alat(): HasMany {
14         return $this->hasMany(Alat::class);
15     }
16 }
```

3. Model Alat

Selanjutnya kita akan membuat file model Alat dengan mengetikkan perintah pada terminal :

docker exec -it laravel-api php artisan make:model Alat.php

Lakukan perubahan baris kode program menjadi seperti berikut:

```

Alat.php X
app > Models > Alat.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\BelongsTo;
7  use Illuminate\Database\Eloquent\Relations\HasMany;
8
9  class Alat extends Model
10 {
11     protected $table = 'alat';
12
13     protected $fillable = [
14         'kategori_id', 'nama_alat', 'stok', 'status_kondisi', 'deskripsi', 'gambar'
15     ];
16
17     protected function casts(): array {
18         return [
19             'stok' => 'integer',
20         ];
21     }
22
23     public function kategori(): BelongsTo {
24         return $this->belongsTo(Kategori::class);
25     }
26
27     public function detailPinjam(): HasMany {
28         return $this->hasMany(DetailPinjam::class);
29     }
30 }

```

4. Model Peminjaman

Selanjutnya kita akan membuat file model Peminjaman dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:model Peminjaman.php

Lakukan impor trait/kelas yang dipergunakan pada model peminjaman dan perubahan baris kode program pada blok class peminjaman menjadi seperti berikut:

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\HasOne;

```

```

class Peminjaman extends Model
{
    protected $table = 'peminjaman';

    protected $fillable = [
        'user_id', 'tgl_pinjam', 'tgl_kembali_plan', 'status'
    ];

    protected function casts(): array {
        return [
            'tgl_pinjam' => 'date:Y-m-d',
            'tgl_kembali_plan' => 'date:Y-m-d',
        ];
    }

    public function user(): BelongsTo {
        return $this->belongsTo(User::class);
    }

    public function detailPinjam(): HasMany {
        return $this->hasMany(DetailPinjam::class);
    }

    public function pengembalian(): HasOne {
        return $this->hasOne(Pengembalian::class);
    }
}

```

5. Model DetilPinjam

Selanjutnya kita akan membuat file model DetilPinjam dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:model DetilPinjam.php

Lakukan impor trait/kelas yang dipergunakan pada model DetilPinjam dan perubahan baris kode program pada blok class DetilPinjam menjadi seperti berikut:

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

```

```

use Illuminate\Database\Eloquent\Relations\BelongsTo;

class DetailPinjam extends Model
{
    protected $table = 'detail_pinjam';

    protected $fillable = [
        'peminjaman_id', 'alat_id', 'jumlah'
    ];

    protected function casts(): array {
        return [
            'jumlah' => 'integer',
        ];
    }

    public function peminjaman(): BelongsTo {
        return $this->belongsTo(Peminjaman::class);
    }

    public function alat(): BelongsTo {
        return $this->belongsTo(Alat::class);
    }
}

```

6. Model Pengembalian

Selanjutnya kita akan membuat file model Pengembalian dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:model Pengembalian.php

Lakukan impor trait/kelas yang dipergunakan pada model Pengembalian dan perubahan baris kode program pada blok class Pengembalian menjadi seperti berikut:

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\HasOne;
class Peminjaman extends Model

```

```

{
    protected $table = 'peminjaman';

    protected $fillable = [
        'user_id', 'tgl_pinjam', 'tgl_kembali_plan', 'status'
    ];

    protected function casts(): array {
        return [
            'tgl_pinjam' => 'date:Y-m-d',
            'tgl_kembali_plan' => 'date:Y-m-d',
        ];
    }

    public function user(): BelongsTo {
        return $this->belongsTo(User::class);
    }

    public function detailPinjam(): HasMany {
        return $this->hasMany(DetailPinjam::class);
    }

    public function pengembalian(): HasOne {
        return $this->hasOne(Pengembalian::class);
    }
}

```

7. Model LogAktivitas

Selanjutnya kita akan membuat file model LogAktivitas dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:model LogAktivitas.php

Lakukan impor trait/kelas yang dipergunakan pada model LogAktivitas dan perubahan baris kode program pada blok class LogAktivitas menjadi seperti berikut:

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

```

```
class LogAktivitas extends Model
{
    protected $table = 'log_aktivitas';

    protected $fillable = [
        'user_id', 'aktivitas'
    ];

    public function user(): BelongsTo {
        return $this->belongsTo(User::class);
    }
}
```

Seeders

Materi Singkat

Untuk memahami **Seeding**, kita bisa membayangkan tabel database yang baru saja dibuat melalui *Migration* sebagai sebuah gudang yang masih kosong melompong. Agar kita bisa segera menguji "masakan" logika program (API) tanpa harus mengisi data satu per satu secara manual melalui aplikasi, Laravel menyediakan fitur **Seeder** untuk mengisi data contoh secara otomatis.

Cara Membuat File Seeder

Untuk membuat file pengisi data, jalankan perintah Artisan berikut di terminal: **php artisan make:seeder ProdukSeeder**. Jika menggunakan docker maka perintah nya menjadi **docker exec -it laravel-api php artisan make:seeder ProdukSeeder**. File ini akan muncul di folder `database/seeders/`.

Cara Menjalankan Seeder

Setelah selesai mengisi data di file seeder, kamu harus menjalankannya agar data masuk ke database:

- **Menjalankan satu file spesifik:** `php artisan db:seed --class=ProdukSeeder`.
- **Menjalankan secara masal (Multi Tabel):** Buka file `database/seeders/DatabaseSeeder.php`, lalu panggil semua seeder di dalam fungsi `run()` dengan perintah `$this->call([ProdukSeeder::class, PelangganSeeder::class]);`. Setelah itu, cukup jalankan perintah singkat: `php artisan db:seed`.

Kegiatan Praktikum

Pada kegiatan praktikum kita akan menjalankan seeder secara masal dengan perintah **db:seed** !

1. Seeder User

Sekarang kita akan membuat Seeder untuk tabel user dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:seeder UserSeeder

Lakukan pengisian baris kode seperti ini:

```
<?php
namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;
```

```

class UserSeeder extends Seeder
{
    public function run(): void
    {
        $users = [
            [
                'name' => 'Bagus Karim',
                'email' => 'admin@gmail.com',
                'password' => Hash::make('password123'),
                'role' => 'admin',
                'no_hp' => '081234567890',
                'alamat' => 'Bandung, West Java',
            ],
            [
                'name' => 'Arif Muhammad',
                'email' => 'petugas@gmail.com',
                'password' => Hash::make('password123'),
                'role' => 'petugas',
                'no_hp' => '082345678901',
                'alamat' => 'Baleendah, Bandung',
            ],
            [
                'name' => 'Rian Setiawan',
                'email' => 'rian@gmail.com',
                'password' => Hash::make('password123'),
                'role' => 'peminjam',
                'no_hp' => '083456789012',
                'alamat' => 'Ciparay, Bandung',
            ],
            [
                'name' => 'Siti Aminah',
                'email' => 'siti@gmail.com',
                'password' => Hash::make('password123'),
                'role' => 'peminjam',
                'no_hp' => '084567890123',
                'alamat' => 'Dayeuhkolot, Bandung',
            ],
            [
                'name' => 'Eka Pratama',
            ]
        ]
    }
}

```

```

        'email' => 'eka@gmail.com',
        'password' => Hash::make('password123'),
        'role' => 'peminjam',
        'no_hp' => '085678901234',
        'alamat' => 'Banjaran, Bandung',
    ],
];

foreach ($users as $user) {
    User::create($user);
}
}
}

```

Penjelasan Baris Kode Program

use App\Models\User	berfungsi untuk mengimpor (memanggil) model User agar bisa digunakan di dalam file kode yang sedang Anda buka
function run()	Nama fungsi wajib bawaan Laravel [Seeder]. Laravel sudah diprogram untuk selalu mengeksekusi fungsi bernama run sebagai gerbang utama saat seeding berjalan [Seeder]
: void	Penanda tipe kembalian (return type) [HasMany]. Tipe : void menegaskan bahwa fungsi ini tidak menghasilkan atau mengembalikan data apa pun (tidak menggunakan kata kunci return) setelah selesai bekerja, melainkan langsung melakukan aksi eksekusi ke database
\$users = [...]	Mendeklarasikan sebuah variabel bernama \$users yang menampung array utama (berfungsi sebagai daftar atau kumpulan dari banyak pengguna)
'name' => 'Bagus Karim'	Pasangan kunci (key) dan nilai (value). Mengatur agar kolom name di database kelak diisi dengan teks "Bagus Karim"
'password' => Hash::make('password123')	Mengamankan kata sandi dengan cara mengubah teks biasa "password123" menjadi kode acak terenkripsi menggunakan sistem Hash::make() bawaan Laravel. Kata sandi asli tidak akan terlihat di database.
foreach (\$users as \$user)	Perintah perulangan ini bertugas membaca variabel \$users (yang berisi daftar semua pengguna).
User::create(\$user)	Di dalam tubuh perulangan, baris ini mengeksekusi perintah pembuatan data ke basis data. Laravel akan mengambil data yang sedang aktif di dalam variabel \$user pada putaran tersebut, lalu

	langsung memasukkannya ke dalam tabel basis data
--	--

2. Seeder Kategori

Sekarang kita akan membuat Seeder untuk tabel kategori dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:seeder KategoriSeeder

Lakukan pengisian baris kode seperti ini:

```
<?php

namespace Database\Seeders;

use App\Models\Kategori;
use Illuminate\Database\Seeder;

class KategoriSeeder extends Seeder
{
    public function run(): void
    {
        $kategori = [
            ['nama_kategori' => 'Jaringan & Konektivitas'],
            ['nama_kategori' => 'Multimedia & Audio Visual'],
            ['nama_kategori' => 'Perangkat Pemrosesan'],
            ['nama_kategori' => 'Perkakas & Elektronik'],
            ['nama_kategori' => 'Suku Cadang & Aksesoris'],
        ];

        foreach ($kategori as $kat) {
            Kategori::create($kat);
        }
    }
}
```

3. Seeder Alat

Sekarang kita akan membuat Seeder untuk tabel Alat dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:seeder AlatSeeder

Lakukan pengisian baris kode seperti ini:

```
<?php

namespace Database\Seeders;
```

```

use App\Models\Alat;
use Illuminate\Database\Seeder;
class AlatSeeder extends Seeder
{
    public function run(): void
    {
        $alat = [
            [
                'kategori_id' => 1,
                'nama_alat' => 'Router Mikrotik RB941-2nD',
                'stok' => 15,
                'status_kondisi' => 'Baik',
                'deskripsi' => 'Router nirkabel rumahan yang cocok untuk
praktik jaringan dasar.',
                'gambar' => 'mikrotik_rb941.jpg',
            ],
            [
                'kategori_id' => 2,
                'nama_alat' => 'Kamera DSLR Canon EOS 3000D',
                'stok' => 5,
                'status_kondisi' => 'Baik',
                'deskripsi' => 'Kamera pemula untuk kebutuhan dokumentasi
dan pembuatan aset media.',
                'gambar' => 'canon_3000d.jpg',
            ],
            [
                'kategori_id' => 3,
                'nama_alat' => 'Mini PC Intel NUC 11',
                'stok' => 8,
                'status_kondisi' => 'Baik',
                'deskripsi' => 'Perangkat komputasi ringkas untuk server
lokal skala kecil.',
                'gambar' => 'intel_nuc.jpg',
            ],
            [
                'kategori_id' => 4,
                'nama_alat' => 'Tang Crimping RJ45/RJ11 Proskit',
                'stok' => 20,
                'status_kondisi' => 'Baik',
            ]
        ]
    }
}

```

```

                'deskripsi' => 'Alat potong dan pasang konektor kabel
UTP.',
                'gambar' => 'crimping_proskit.jpg',
            ],
            [
                'kategori_id' => 5,
                'nama_alat' => 'Adapter HDMI to VGA dengan Audio',
                'stok' => 25,
                'status_kondisi' => 'Baik',
                'deskripsi' => 'Konverter display untuk menyambungkan
perangkat modern ke proyektor lama.',
                'gambar' => 'hdmi_vga.jpg',
            ],
        ];

        foreach ($alat as $item) {
            Alat::create($item);
        }
    }
}

```

4. Seeder Peminjaman

Sekarang kita akan membuat Seeder untuk tabel Peminjaman dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:seeder PeminjamanSeeder

Lakukan pengisian baris kode seperti ini:

```

<?php

namespace Database\Seeders;

use App\Models\Peminjaman;
use Illuminate\Database\Seeder;

class PeminjamanSeeder extends Seeder
{
    public function run(): void
    {
        $peminjaman = [
            [
                'user_id' => 3, // Rian (Peminjam)
            ]
        ];
    }
}

```

```

        'tgl_pinjam' => '2026-06-01',
        'tgl_kembali_plan' => '2026-06-04',
        'status' => 'dikembalikan',
    ],
    [
        'user_id' => 4, // Siti (Peminjam)
        'tgl_pinjam' => '2026-06-02',
        'tgl_kembali_plan' => '2026-06-05',
        'status' => 'dikembalikan',
    ],
    [
        'user_id' => 5, // Eka (Peminjam)
        'tgl_pinjam' => '2026-06-03',
        'tgl_kembali_plan' => '2026-06-06',
        'status' => 'telat',
    ],
    [
        'user_id' => 3,
        'tgl_pinjam' => '2026-06-08',
        'tgl_kembali_plan' => '2026-06-11',
        'status' => 'dipinjam',
    ],
    [
        'user_id' => 4,
        'tgl_pinjam' => '2026-06-09',
        'tgl_kembali_plan' => '2026-06-12',
        'status' => 'diajukan',
    ],
],
];

foreach ($peminjaman as $pinjam) {
    Peminjaman::create($pinjam);
}
}
}

```

5. Seeder DetailPinjam

Sekarang kita akan membuat Seeder untuk tabel DetailPinjam dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:seeder DetailPinjamSeeder

Lakukan pengisian baris kode seperti ini:

```
<?php
namespace Database\Seeders;
use App\Models\DetailPinjam;
use Illuminate\Database\Seeder;

class DetailPinjamSeeder extends Seeder
{
    public function run(): void
    {
        $details = [
            ['peminjaman_id' => 1, 'alat_id' => 1, 'jumlah' => 2], //
Pinjam 2 Mikrotik
            ['peminjaman_id' => 2, 'alat_id' => 2, 'jumlah' => 1], //
Pinjam 1 Kamera
            ['peminjaman_id' => 3, 'alat_id' => 3, 'jumlah' => 1], //
Pinjam 1 Mini PC
            ['peminjaman_id' => 4, 'alat_id' => 4, 'jumlah' => 2], //
Pinjam 2 Tang Crimping
            ['peminjaman_id' => 5, 'alat_id' => 5, 'jumlah' => 3], //
Pinjam 3 Adapter
        ];

        foreach ($details as $detail) {
            DetailPinjam::create($detail);
        }
    }
}
```

6. Seeder Pengembalian

Sekarang kita akan membuat Seeder untuk tabel Pengembalian dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:seeder PengembalianSeeder

Lakukan pengisian baris kode seperti ini:

```
<?php

namespace Database\Seeders;

use App\Models\Pengembalian;
use Illuminate\Database\Seeder;
```

```

class PengembalianSeeder extends Seeder
{
    public function run(): void
    {
        $pengembalian = [
            [
                'peminjaman_id' => 1,
                'tgl_kembali' => '2026-06-04',
                'kondisi_kembali' => 'Lengkap dan Berfungsi Baik',
                'denda' => 0,
                'petugas_id' => 2, // Arif (Petugas)
            ],
            [
                'peminjaman_id' => 2,
                'tgl_kembali' => '2026-06-05',
                'kondisi_kembali' => 'Lengkap dan Berfungsi Baik',
                'denda' => 0,
                'petugas_id' => 2,
            ],
            [
                'peminjaman_id' => 3,
                'tgl_kembali' => '2026-06-09', // Telat 3 hari dari tgl 6
                'kondisi_kembali' => 'Lengkap, Casing Sedikit Tergores',
                'denda' => 30000, // Asumsi denda per hari 10rb
                'petugas_id' => 2,
            ],
        ];

        foreach ($pengembalian as $kembali) {
            Pengembalian::create($kembali);
        }
    }
}

```

7. Seeder LogAktivitas

Sekarang kita akan membuat Seeder untuk tabel LogAktivitas dengan mengetikkan perintah pada terminal:

docker exec -it laravel-api php artisan make:seeder LogAktivitasSeeder

Lakukan pengisian baris kode seperti ini:

```

<?php

namespace Database\Seeders;
use App\Models\LogAktivitas;
use Illuminate\Database\Seeder;

class LogAktivitasSeeder extends Seeder
{
    public function run(): void
    {
        $logs = [
            ['user_id' => 1, 'aktivitas' => 'Melakukan import data master
alat baru sebanyak 5 entitas.'],
            ['user_id' => 2, 'aktivitas' => 'Menyetujui permohonan
peminjaman ID #4.'],
            ['user_id' => 3, 'aktivitas' => 'Mengajukan peminjaman alat
baru untuk kebutuhan praktik kelompok.'],
            ['user_id' => 2, 'aktivitas' => 'Memproses pengembalian alat
telat untuk peminjaman ID #3 dan mengenakan denda.'],
            ['user_id' => 1, 'aktivitas' => 'Mengubah konfigurasi hak
akses aplikasi.'],
        ];

        foreach ($logs as $log) {
            LogAktivitas::create($log);
        }
    }
}

```

8. DatabaseSeeder

File ini berfungsi sebagai orchestrator untuk menjalankan semua seeder dengan urutan yang benar.

Ubah kode pada databaseSeeder.php menjadi seperti ini:

```

<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder

```

```

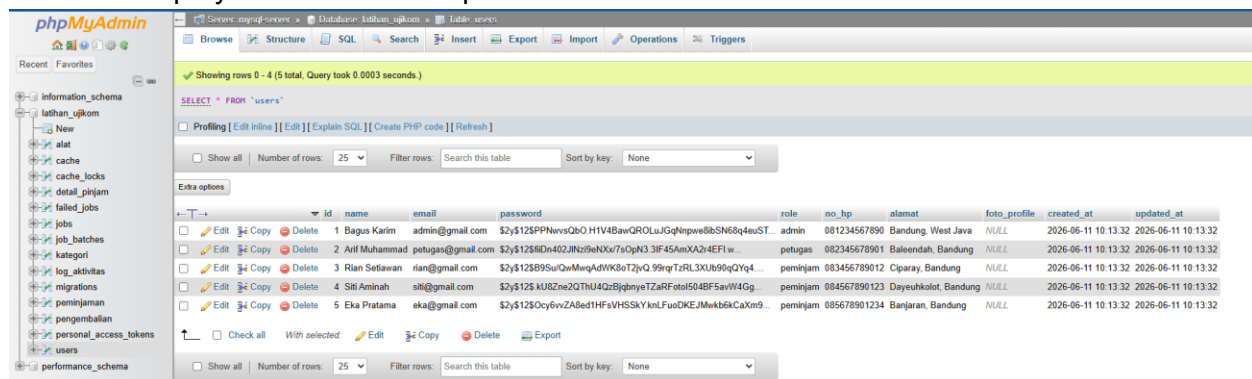
{
    public function run(): void
    {
        $this->call([
            UserSeeder::class,
            KategoriSeeder::class,
            AlatSeeder::class,
            PeminjamanSeeder::class,
            DetailPinjamSeeder::class,
            PengembalianSeeder::class,
            LogAktivitasSeeder::class,
        ]);
    }
}

```

9. Db:seed & Pengecekan

Ketikkan perintah pada terminal : **docker exec -it laravel-api php artisan db:seed**

Lakukan pemeriksaan pada setiap tabel yang kita lakukan seeding, jika tidak error maka data akan otomatis masuk pada container database. Kalian bisa melakukan pengecekan pada container PhpMyAdmin untuk mempermudah.



Showing rows 0 - 4 (5 total, Query took 0.0003 seconds)

SELECT * FROM `users`

Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	id	name	email	password	role	no_hp	alamat	foto_profile	created_at	updated_at
☐	1	Bagus Karim	admin@gmail.com	\$2y\$12\$PpNvrsQbO.H1V4BawQROLuJGqNpwe8bSN68q4euST...	admin	081234567890	Bandung, West Java	NULL	2026-06-11 10:13:32	2026-06-11 10:13:32
☐	2	Arlil Muhammad	petugas@gmail.com	\$2y\$12\$8Dn40ZJNz9eNXu7xOpN3.3iF45AmXA2z4EF1w...	petugas	082345678901	Baleendah, Bandung	NULL	2026-06-11 10:13:32	2026-06-11 10:13:32
☐	3	Rian Setiawan	rian@gmail.com	\$2y\$12\$89SuQwMwqADWk6t72pVg.99rqTzRL3XU696qQYq4...	peminjam	083456789012	Ciparay, Bandung	NULL	2026-06-11 10:13:32	2026-06-11 10:13:32
☐	4	Siti Aminah	siti@gmail.com	\$2y\$12\$KUSZne2QThU4GzBjpbnyeTZaRfotl504BF5avW4Gg...	peminjam	084567890123	Dayeuhkolot, Bandung	NULL	2026-06-11 10:13:32	2026-06-11 10:13:32
☐	5	Eka Pratama	eka@gmail.com	\$2y\$12\$Ocy6vzZA8ed1HFvVHSSKYknLFuoDKEJMukb6kCaXm9...	peminjam	085678901234	Banjaran, Bandung	NULL	2026-06-11 10:13:32	2026-06-11 10:13:32

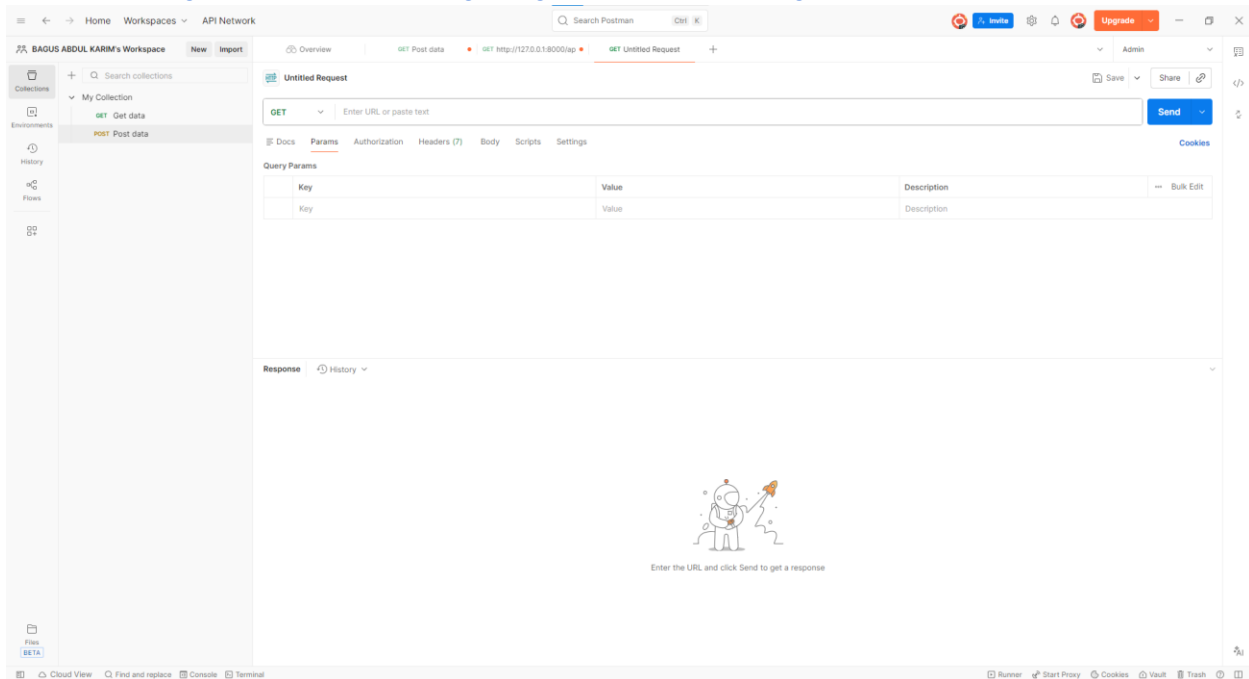
Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Sistem Autentikasi API (Login, Register, Logout)

Kegiatan Praktikum

Persiapan Instalasi Postman

Lakukan Instalasi Postman pada komputer masing masing, pastikan postman dapat terbuka dan berjalan sebagaimana mestinya. Jika belum melakukan penginstalan kalian dapat melakukan mengunduh dan melakukan penginstalan pada :
<https://learning.postman.com/docs/getting-started/first-steps/get-postman>



Persiapan Form Request

Form Request adalah class kustom dalam Laravel yang merangkum logika validasi dan otorisasi dari sebuah permintaan (request) data pengguna. Penggunaan komponen ini merupakan bagian dari penerapan prinsip **Separation of Concerns**, yang bertujuan memindahkan logika validasi keluar dari controller agar kode tetap bersih dan mudah dikelola, atau yang sering disebut dengan **konsep Skinny Controller**.

Form Request dapat dibuat dengan perintah Artisan **php artisan make:request NamaRequest** jika menggunakan docker maka perintahnya menjadi **docker exec -it php artisan make:request NamaRequest**. File yang dihasilkan akan tersimpan di direktori **app/Http/Requests** dan memiliki dua metode utama:

- **authorize()**: Digunakan untuk menentukan apakah pengguna yang terautentikasi memiliki izin untuk melakukan tindakan tersebut. Jika metode ini mengembalikan nilai false, Laravel otomatis mengirimkan respons HTTP 403 Forbidden.
- **rules()**: Berisi kumpulan aturan validasi yang harus dipenuhi oleh data yang dikirimkan, seperti required, unique, string, hingga validasi file.

1. RegisterRequest

Sekarang kita akan membuat file RegisterRequest dengan cara menjalankan perintah pada terminal ***docker exec -it laravel-api php artisan make:request Auth/RegisterRequest*** Lakukan perubahan baris kode pada file RegisterRequest.php menjadi seperti ini:

```
<?php
namespace App\Http\Requests\Auth;
use Illuminate\Foundation\Http\FormRequest;

class RegisterRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => 'required|string|min:8|confirmed',
            'no_hp' => 'nullable|string|max:15',
            'alamat' => 'nullable|string',
        ];
    }
}
```

2. LoginRequest

Sekarang kita akan membuat file LoginRequest dengan cara menjalankan perintah pada terminal ***docker exec -it laravel-api php artisan make:request Auth/LoginRequest*** Lakukan perubahan baris kode pada file LoginRequest.php menjadi seperti ini:

```
<?php
namespace App\Http\Requests\Auth;
```

```

use Illuminate\Foundation\Http\FormRequest;

class LoginRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'email' => 'required|email',
            'password' => 'required|string',
        ];
    }
}

```

Persiapan Resource

API Resources (JsonResource) adalah lapisan transformasi yang berfungsi sebagai perantara antara model database (Eloquent) dan respons JSON yang dikirimkan ke klien. Komponen ini sangat krusial untuk dipelajari karena memungkinkan pengembang untuk membangun API yang profesional dengan struktur data yang konsisten dan terprediksi.

API Resources digunakan untuk memisahkan logika presentasi data dari logika bisnis di controller. Beberapa tujuan utamanya meliputi:

- **Standarisasi Response:** Memastikan setiap endpoint mengembalikan struktur data yang seragam, sehingga memudahkan pengembang frontend dalam mengolah data.
- **Keamanan Data:** Memungkinkan Anda untuk menyaring informasi sensitif, seperti menyembunyikan kolom password atau remember_token, sehingga tidak sengaja terkirim ke publik.
- **Transformasi Data:** Anda dapat mengubah format data sebelum dikirim, misalnya mengubah format tanggal menggunakan library Carbon atau menggabungkan nama depan dan belakang menjadi satu kolom full_name.

Untuk membuat resource baru, Anda dapat menggunakan perintah Artisan berikut: **php artisan make:resource NamaResource** .

Setelah file dibuat di direktori **app/Http/Resources**, Anda perlu mendefinisikan array data di dalam metode toArray(). Metode ini menentukan pasangan key-value apa saja yang akan muncul dalam output JSON.

Adapun Fitur Utama API Resource sebagai berikut:

- **Penanganan Relasi** (Eager Loading): Resource dapat menangani data dari tabel terkait dengan efisien. Misalnya, saat menampilkan data Alat, Anda bisa menyertakan data Kategori di dalamnya untuk mencegah masalah performa N+1 Query.
- **Pengumpulan Data** (Resource Collections): Jika Anda ingin mengubah koleksi data (misalnya daftar banyak user), Anda bisa membuat Resource Collection atau cukup menggunakan metode statis `UserResource::collection($users)` pada controller.
- **Penerapan pada Controller**: Di dalam controller, Anda tidak lagi mengembalikan array mentah, melainkan mengembalikan instance dari Resource tersebut: `return new UserResource($user);`

UserResource

Sekarang kita akan membuat file `UserResource` dengan cara menjalankan perintah pada terminal **`docker exec -it laravel-api php artisan make:resource UserResource`**
Lakukan perubahan baris kode pada file `UserResource.php` menjadi seperti ini:

```
<?php
namespace App\Http\Resources;
use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class UserResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->email,
            'role' => $this->role,
            'no_hp' => $this->no_hp,
            'alamat' => $this->alamat,
            'foto_profile' => $this->foto_profile ? url('storage/' .
$this->foto_profile) : null,
            'bergabung_sejak' => $this->created_at->format('Y-m-d'),
        ];
    }
}
```

Persiapan Middleware Role Check

Middleware di Laravel adalah mekanisme yang berfungsi sebagai lapisan penyaring (filter) untuk setiap permintaan HTTP yang masuk ke aplikasi Anda. Analogi sederhananya adalah seperti petugas keamanan di depan pintu masuk sebuah gedung atau antrian di taman hiburan; ia akan

memeriksa apakah pengunjung memiliki tiket (token) atau izin akses yang sesuai sebelum diperbolehkan masuk ke area tertentu.

Dalam arsitektur Laravel Sanctum, middleware bukan sekadar penyaring biasa, melainkan komponen kritis yang menentukan bagaimana sebuah permintaan (request) divalidasi, apakah melalui sesi berbasis cookie atau token API.

Dalam arsitektur API modern, Role-Based Access Control (RBAC) melalui Middleware adalah lapisan pertahanan kedua setelah otentikasi. Jika otentikasi (Sanctum) bertugas menjawab pertanyaan "Siapa Anda?", maka RBAC Middleware menjawab pertanyaan "Apa yang boleh Anda lakukan?".

Keuntungan Menggunakan Middleware untuk RBAC

- **Keamanan Terpusat:** Logika pengecekan izin akses tidak tersebar di setiap fungsi Controller, sehingga meminimalisir risiko lupa menambahkan cek keamanan.
- **Skinny Controller:** Controller tetap fokus pada logika bisnis (seperti menghitung stok atau denda) tanpa dibebani pengecekan hak akses.
- **Fleksibilitas Tinggi:** Anda bisa dengan mudah mengubah aturan akses (misalnya memberikan akses Admin ke fitur Petugas) hanya dengan mengubah rute di satu file saja.
- **Stateless Compliance:** Sangat cocok untuk API karena setiap permintaan membawa token yang divalidasi identitasnya dan kemudian perannya secara real-time tanpa bergantung pada sesi server yang berat.

Middleware Admin

Sekarang kita akan membuat file middleware untuk admin dengan cara menjalankan perintah pada terminal **docker exec -it laravel-api php artisan make:middleware IsAdmin**

Lakukan perubahan baris kode pada file IsAdmin.php menjadi seperti ini:

```
<?php
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class IsAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        if ($request->user() && $request->user()->role === 'admin') {
            return $next($request);
        }

        return response()->json(['message' => 'Akses ditolak. Anda bukan Admin.'], 403);
    }
}
```

```
}
```

Middleware Petugas

Sekarang kita akan membuat file middleware untuk petugas dengan cara menjalankan perintah pada terminal **docker exec -it laravel-api php artisan make:middleware IsPetugas**

Pada file IsPetugas.php Lakukan hal yang persis sama dengan IsAdmin, namun ubah pengecekan menjadi role === 'petugas' dan pesan error menyesuaikan.

Middleware Peminjam

Sekarang kita akan membuat file middleware untuk peminjam dengan cara menjalankan perintah pada terminal **docker exec -it laravel-api php artisan make:middleware IsPeminjam**

Pada file IsPeminjam.php Lakukan hal yang persis sama dengan IsAdmin, namun ubah pengecekan menjadi role === 'petugas' dan pesan error menyesuaikan.

Registrasi Middleware bootstrap/app.php

Buka file bootstrap/app.php dan daftarkan alias middleware agar mudah dipanggil di routes/api.php:

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use App\Http\Middleware\IsAdmin;
use App\Http\Middleware\IsPetugas;
use App\Http\Middleware\IsPeminjam;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->alias([
            'role.admin' => IsAdmin::class,
            'role.petugas' => IsPetugas::class,
            'role.peminjam' => IsPeminjam::class,
```

```

    });
})
->withExceptions(function (Exceptions $exceptions): void {
    //
})->create();

```

Membuat AuthController

Controller bertugas untuk mengelompokkan logika penanganan permintaan HTTP yang terkait ke dalam satu kelas. Secara default, semua file controller disimpan di direktori **app/Http/Controllers**. Anda dapat membuatnya dengan cepat menggunakan perintah Artisan: `php artisan make:controller NamaController` jika menggunakan docker menjadi **docker exec -it nama_kontainer php artisan make:controller NamaController**.

Laravel mendukung beberapa struktur controller sesuai dengan kebutuhan kompleksitas fitur:

- **Basic Controllers:** Kelas standar dengan banyak metode publik (seperti index, show, store) yang masing-masing menangani tugas berbeda.
- **Single Action Controllers (Invokable):** Jika sebuah tindakan sangat kompleks, Anda dapat mendedikasikan satu kelas penuh untuk satu aksi menggunakan metode `__invoke()`.
- **Resource Controllers:** Digunakan untuk menangani operasi CRUD (Create, Read, Update, Delete) secara standar. Laravel menyediakan perintah `--resource` atau `--api` (untuk API tanpa rute HTML) yang otomatis membuat metode-metode seperti index, store, show, update, dan destroy.

Kita dapat menetapkan Middleware (seperti autentikasi Sanctum atau pengecekan peran/RBAC) langsung di dalam kelas controller. Dengan mengimplementasikan interface `HasMiddleware`, controller dapat menentukan middleware apa saja yang harus dilewati sebelum sebuah metode dieksekusi. Ini memberikan kontrol yang sangat halus atas keamanan setiap aksi.

Dalam arsitektur API modern dan Clean Code, sangat disarankan untuk menjaga agar controller tetap "tipis" (lean). Berikut adalah strategi yang diterapkan pada proyek Peminjaman Alat:

- **Delegasi Validasi:** Validasi data input dipindahkan ke Form Request.
- **Delegasi Pemformatan:** Transformasi data untuk respons API dipindahkan ke API Resources (`JsonResource`).
- **Keamanan Transaksi:** Untuk operasi database yang kompleks, controller membungkus logika tersebut di dalam `DB::transaction` untuk menjamin integritas data jika terjadi kegagalan di tengah jalan.
- **Audit Trail:** Menggunakan Observers untuk mencatat log aktivitas secara otomatis di balik layar, sehingga controller tidak perlu lagi melakukan pencatatan log secara manual.

AuthController

Sekarang kita akan membuat file AuthController untuk aplikasi peminjaman alat dengan cara menjalankan perintah pada terminal **docker exec -it laravel- api php artisan make:controller API/AuthController**

```
<?php
```

```

namespace App\Http\Controllers\API;
use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use App\Http\Requests\Auth\RegisterRequest;
use App\Http\Resources\UserResource;
use App\Models\User;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthController extends Controller
{
    public function register(RegisterRequest $request): JsonResponse
    {
        try {
            $user = DB::transaction(function () use ($request) {
                return User::create([
                    'name' => $request->name,
                    'email' => $request->email,
                    'password' => Hash::make($request->password),
                    'role' => 'peminjam',
                    'no_hp' => $request->no_hp,
                    'alamat' => $request->alamat,
                ]);
            });

            $token = $user->createToken('auth_token')->plainTextToken;
            return response()->json([
                'message' => 'Registrasi berhasil.',
                'data' => new UserResource($user),
                'access_token' => $token,
                'token_type' => 'Bearer',
            ], 201);
        } catch (\Exception $e) {
            return response()->json(['message' => 'Terjadi kesalahan saat
registrasi.', 'error' => $e->getMessage()], 500);
        }
    }
}

```

```

public function login(LoginRequest $request): JsonResponse
{
    $user = User::where('email', $request->email)->first();
    if (!$user || !Hash::check($request->password, $user->password)) {
        throw ValidationException::withMessages([
            'email' => ['Kredensial yang diberikan tidak cocok dengan
data kami.'],
        ]);
    }

    $token = $user->createToken('auth_token')->plainTextToken;
    return response()->json([
        'message' => 'Login berhasil.',
        'data' => new UserResource($user),
        'access_token' => $token,
        'token_type' => 'Bearer',
    ]);
}

public function me(): JsonResponse
{
    return response()->json([
        'message' => 'Data profil berhasil diambil.',
        'data' => new UserResource(auth()->user()),
    ]);
}

public function logout(): JsonResponse
{
    auth()->user()->currentAccessToken()->delete();
    return response()->json([
        'message' => 'Logout berhasil. Token telah dihapus.',
    ]);
}
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Fungsi
--------------------	--------

<code>namespace App\Http\Controllers\API;</code>	Menentukan lokasi fisik dan logis dari file controller ini di dalam folder <code>app/Http/Controllers/API</code>
<code>use App\Http\Controllers\Contr oller;</code>	Mengimpor base controller utama milik Laravel
<code>use App\Http\Requests\Auth\L oginRequest;</code>	Mengimpor berkas validasi khusus untuk proses masuk (login)
<code>use App\Http\Resources\User Resource;</code>	Mengimpor Laravel API Resource untuk memformat data pengguna (User) sebelum dikirimkan sebagai respons JSON (misal: menyembunyikan password atau mengubah format tanggal)
<code>use App\Models\User;</code>	Mengimpor model User agar controller bisa berinteraksi dengan tabel users di dalam database menggunakan Eloquent ORM.
<code>use Illuminate\Http\JsonRespo nse;</code>	Memberikan petunjuk tipe (type-hinting) bahwa fungsi di dalam controller ini akan mengembalikan respons dalam format JSON standar API
<code>use Illuminate\Support\Facade s\DB;</code>	Mengimpor facade DB untuk menjalankan query SQL manual atau mengamankan transaksi data menggunakan <code>DB::beginTransaction()</code> dan <code>DB::commit()</code>
<code>use Illuminate\Support\Facade s\Hash;</code>	Mengimpor facade enkripsi untuk mengamankan kata sandi (misalnya <code>Hash::make(\$request->password)</code>) menggunakan algoritma Bcrypt/Argon2.
<code>use Illuminate\Validation\Valid ationException;</code>	Mengimpor sistem pengecualian (exception) bawaan Laravel untuk menghentikan proses secara otomatis dan mengembalikan pesan error jika validasi input gagal
Function Register	
<code>RegisterRequest \$request</code>	Laravel secara otomatis memvalidasi data input (seperti nama, email, password) sebelum masuk ke fungsi ini berdasarkan aturan yang Anda tulis di dalam file <code>RegisterRequest</code>
<code>: JsonResponse</code>	Menegaskan secara eksplisit bahwa fungsi ini harus mengembalikan respons dalam format JSON standar API.
<code>try { ... }</code>	Laravel akan mencoba menjalankan semua kode utama di dalam blok ini.
<code>catch (\Exception \$e)</code>	Jika terjadi error tak terduga (misal: server database mati atau email duplikat lolos validasi), aplikasi tidak akan crash melainkan langsung melompat ke blok catch untuk mengembalikan respons error dengan status kode 500 (Internal Server Error).

<code>\$user = DB::transaction(function () use (\$request) { ... });</code>	Fungsi ini memastikan integritas data. Jika di dalam fungsi penutupan (closure) ini terdapat beberapa proses database dan salah satunya gagal, Laravel akan membatalkan semua perubahan (rollback). Data pengguna baru hanya akan disimpan jika seluruh proses di dalamnya sukses.
<code>User::create([...])</code>	Menyimpan data pengguna baru ke dalam tabel users menggunakan Eloquent ORM
<code>'role' => 'peminjam'</code>	Menetapkan nilai hak akses secara tegas langsung dari kode program.
<code>\$token = \$user->createToken('auth_token')->plainTextToken;</code>	Baris ini menggunakan Laravel Sanctum. Setelah pengguna berhasil dibuat, Laravel langsung membuat token akses khusus untuk pengguna tersebut. Token dalam bentuk teks biasa ini (plainTextToken) yang nantinya digunakan pengguna untuk mengakses API yang dikunci.
<code>return response()->json(..., 201);</code>	Mengirimkan data kembali ke client (seperti aplikasi Mobile atau Frontend React/Vue) dalam bentuk format JSON dengan status kode HTTP yang berarti 201 (Created)
Function Login	
<code>\$user = User::where('email', \$request->email)->first();</code>	Sistem melakukan pencarian ke tabel users di database menggunakan Eloquent ORM. Metode first() akan mengambil satu data pengguna pertama yang memiliki email yang cocok dengan input <code>\$request->email</code> . Jika tidak ditemukan, variabel <code>\$user</code> akan bernilai null
<code>!\$user</code>	Memeriksa apakah pengguna tidak ditemukan di database
<code>!Hash::check(...)</code>	Membandingkan teks password mentah dari input (<code>\$request->password</code>) dengan password terenkripsi yang ada di database (<code>\$user->password</code>).
<code>throw ValidationException::withMessages(...)</code>	Jika salah satu kondisi di atas terpenuhi (salah email atau salah password), Laravel akan sengaja melempar <i>error</i> validasi.
<code>\$token = \$user->createToken('auth_token')->plainTextToken;</code>	Setelah melewati pemeriksaan keamanan, pengguna dinyatakan sah. Laravel Sanctum kemudian membuatkan token akses baru untuk pengguna tersebut
Function Me	
<code>auth()->user()</code>	helper bawaan Laravel untuk mengakses sistem otentikasi secara

	otomatis mendeteksi dan mengambil objek model data pengguna (User) yang sedang aktif mengirimkan permintaan API berdasarkan token (Sanctum/Passport) yang dilampirkan pada HTTP Header Authorization.
<code>new UserResource(...)</code>	UserResource akan menyusun ulang struktur datanya. Fitur ini memastikan data sensitif seperti password, remember_token, atau data internal lainnya tersembunyi, dan hanya mengirimkan data yang memang dibutuhkan oleh halaman profil (misalnya: id, name, email, role, no_hp, dan alamat)
<code>return response()->json([...]);</code>	Menggabungkan pesan sukses (message) dan data profil yang sudah rapi (data) ke dalam satu objek JSON untuk dikirimkan kembali ke client.
Function Logout	
<code>auth()->user()</code>	Mengambil data objek pengguna (User) yang saat ini sedang melakukan request berdasarkan token yang dikirim di HTTP Header
<code>currentAccessToken()</code>	Fungsi bawaan Laravel Sanctum untuk mengambil baris data token spesifik yang sedang digunakan oleh pengguna tersebut pada perangkat yang ia pakai saat itu
<code>delete()</code>	Melakukan query DELETE ke tabel personal_access_tokens di database. Begitu baris data token tersebut dihapus, token tersebut langsung menjadi tidak sah (invalid). Jika client mencoba menggunakannya lagi, server akan mengembalikan error 401 Unauthorized.

Route API

Dalam pengembangan API menggunakan Laravel, Route API merupakan gerbang utama yang memetakan permintaan HTTP klien ke logika bisnis di dalam aplikasi. Berbeda dengan route web biasa, route API dirancang untuk komunikasi antar-sistem yang bersifat stateless (tanpa sesi) dan biasanya mengembalikan data dalam format JSON.

Laravel mendukung semua kata kerja (verb) HTTP standar untuk mendefinisikan rute API:

- `Route::get()`: Untuk mengambil data.
- `Route::post()`: Untuk mengirim atau membuat data baru.
- `Route::put()` / `Route::patch()`: Untuk memperbarui data yang sudah ada.
- `Route::delete()`: Untuk menghapus data.
- `Route::match()` / `Route::any()`: Digunakan jika satu endpoint harus melayani beberapa atau semua jenis metode HTTP.

Untuk mempercepat pengembangan, Laravel menyediakan fungsi `Route::apiResource()`. Fungsi ini secara otomatis mendaftarkan rute-rute standar untuk operasi CRUD (Create, Read, Update, Delete) tanpa menyertakan rute create dan edit yang berbasis HTML.

Verb	URI	Action (Controller)	Tujuan Arsitektural
GET	<code>/api/posts</code>	<code>index</code>	Mengambil daftar data
POST	<code>/api/posts</code>	<code>store</code>	Validasi dan simpan data baru
GET	<code>/api/posts/{post}</code>	<code>show</code>	Mengambil satu data spesifik
PUT/PATCH	<code>/api/posts/{post}</code>	<code>update</code>	Memperbarui data yang ada
DELETE	<code>/api/posts/{post}</code>	<code>destroy</code>	Menghapus data dari database

Rute API biasanya dilindungi agar hanya pengguna yang sah yang bisa mengaksesnya:

- **Autentikasi Sanctum:** Menggunakan middleware `auth:sanctum` untuk memastikan setiap permintaan membawa token yang valid.
- **Role-Based Access Control (RBAC):** Rute dapat dikelompokkan berdasarkan peran pengguna (misalnya: admin atau petugas) menggunakan middleware kustom untuk membatasi akses ke endpoint tertentu.
- **Rate Limiting (Throttling):** Digunakan untuk membatasi jumlah permintaan yang dapat dilakukan klien dalam jangka waktu tertentu (misalnya: 60 permintaan per menit) guna mencegah penyalahgunaan API.

Rute API dapat menerima parameter dinamis untuk mengidentifikasi resource:

- **Required Parameters:** Ditandai dengan kurung kurawal, misal `{id}`. Data ini akan diteruskan ke controller sebagai argumen fungsi.
- **Constraints:** Anda dapat membatasi format parameter menggunakan metode `where()` (misal: hanya angka) agar sistem memberikan respon 404 jika format tidak sesuai.

Update Route api.php

Lakukan update routes/api.php menjadi:

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\API\AuthController;

// Public Routes (Tidak perlu token)
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

// Protected Routes (Wajib membawa Bearer Token dari Sanctum)
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', [AuthController::class, 'me']);
    Route::post('/logout', [AuthController::class, 'logout']);

    Route::middleware('role.admin')->group(function () {
        // Route untuk hak akses admin
    });

    Route::middleware('role.petugas')->group(function () {
        // Route untuk hak akses petugas
    });

    Route::middleware('role.peminjam')->group(function () {
        // Route untuk hak akses peminjam
    });
});
```

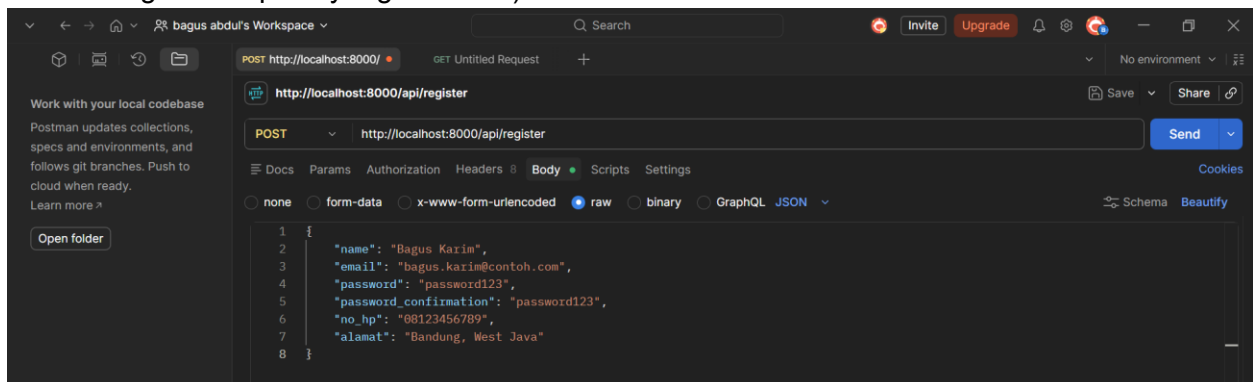
Pengujian Sistem Autentikasi API

Pastikan Container sudah berjalan dengan baik dan dapat difungsikan.

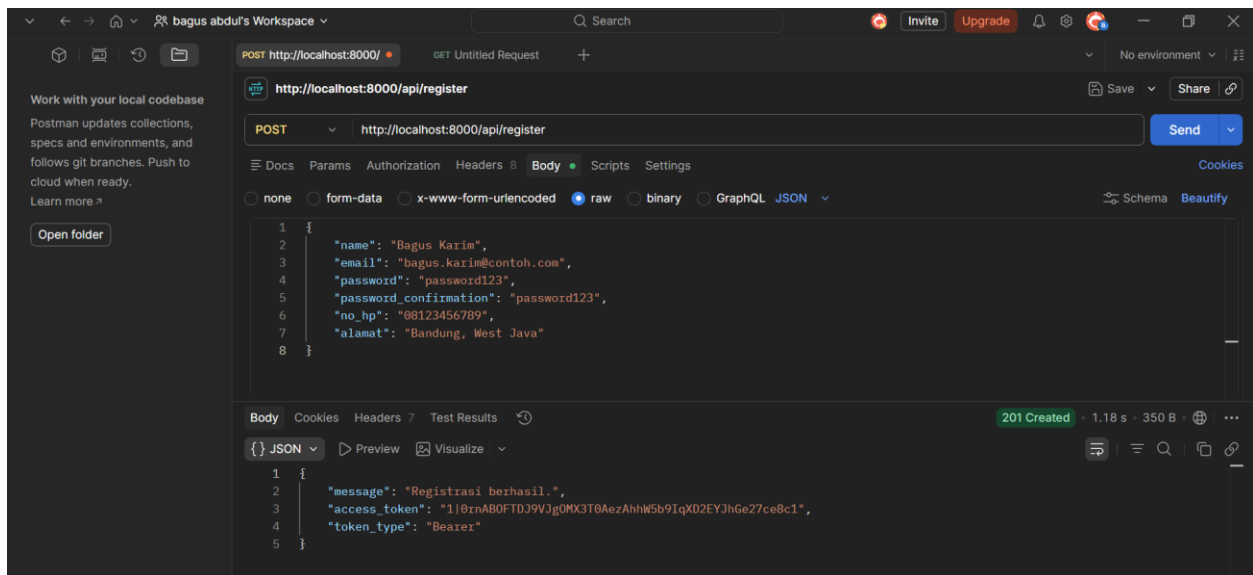
Pengujian Register (POST /api/register)



1. Buka aplikasi Postman, buat akun baru pada postman.
2. Ubah HTTP Method menjadi POST.
3. Masukkan URL: <http://127.0.0.1:8000/api/register>
4. Buka tab **Body**, pilih **raw**, lalu pada *dropdown* di sebelah kanan tulisan raw, ubah **Text** menjadi **JSON**.
5. Masukkan payload JSON berikut ke dalam kotak teks (sesuai dengan aturan RegisterRequest yang kita buat):



6. Klik Send. Hasil yang diharapkan: Anda akan menerima HTTP Status 201 Created dan response JSON yang berisi pesan sukses, data profil, dan sebuah access_token.

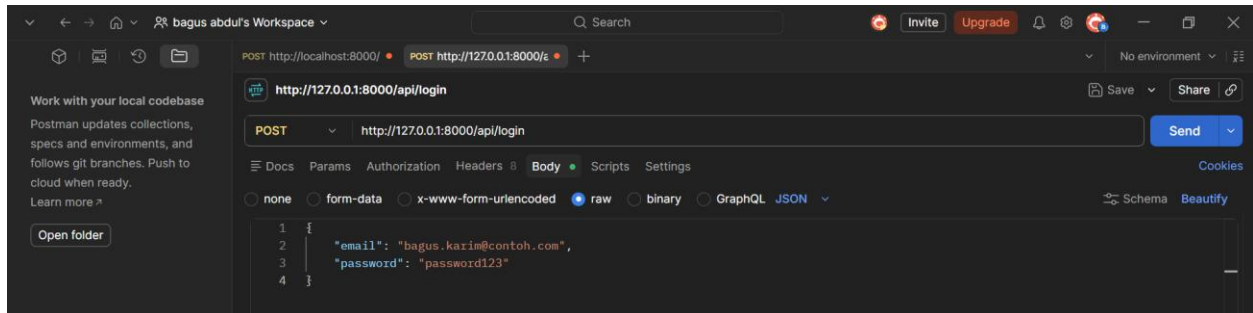


Pengujian Login (POST /api/login)

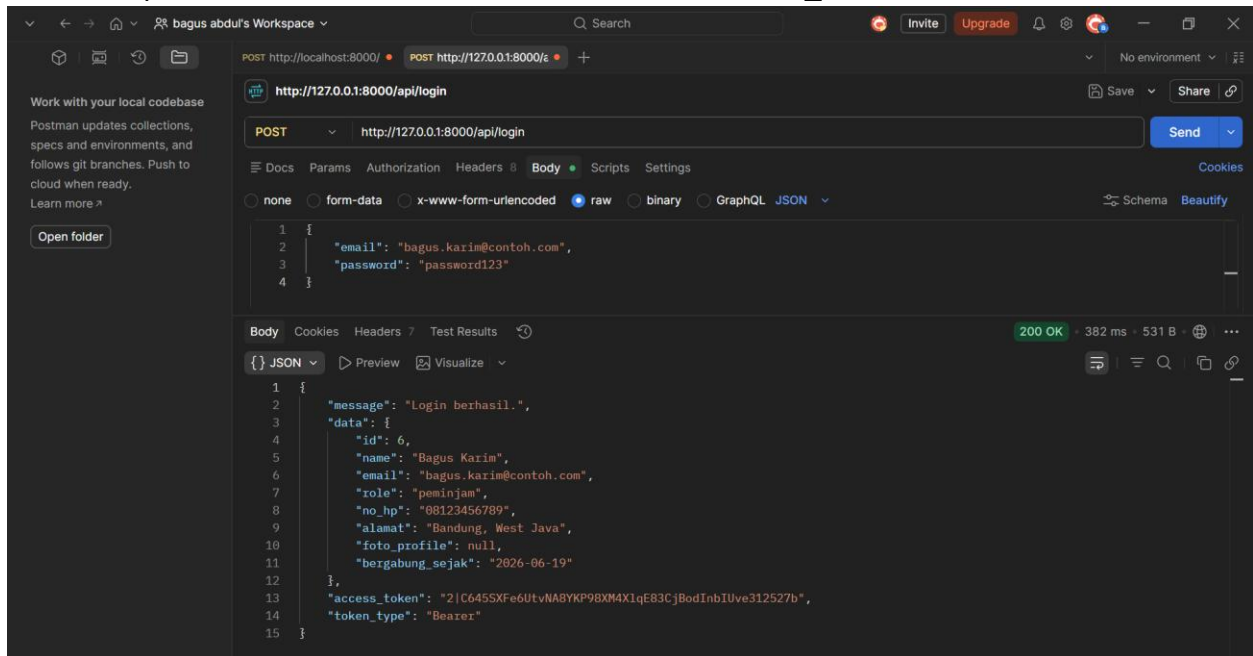


Sekarang mari kita tes *login* menggunakan akun yang baru saja dibuat.

1. Buka tab baru di Postman.
2. Pastikan tab Headers sudah memiliki Accept: application/json.
3. Ubah HTTP Method menjadi POST.
4. Masukkan URL: <http://127.0.0.1:8000/api/login>
5. Buka tab Body, pilih raw, dan ubah tipe menjadi JSON. Masukkan payload berikut:



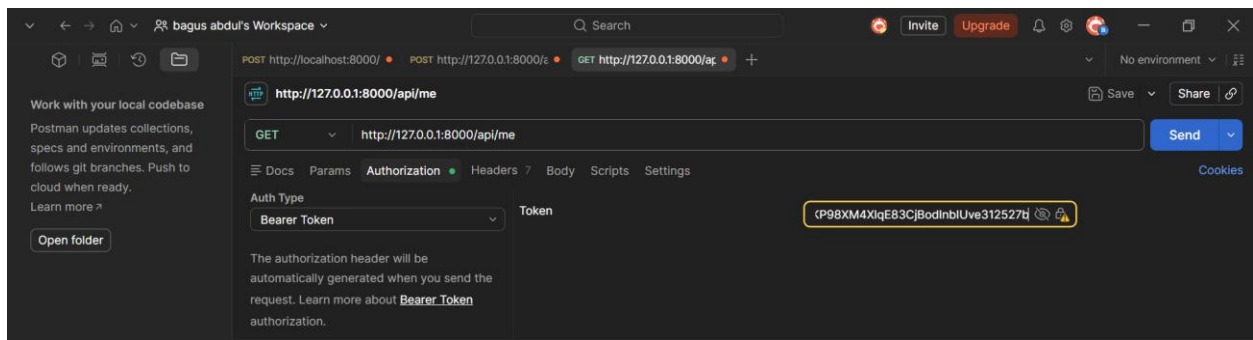
6. Klik Send. Hasil yang diharapkan: Anda menerima HTTP Status 200 OK. Di dalam response tersebut, Anda akan kembali melihat access_token.



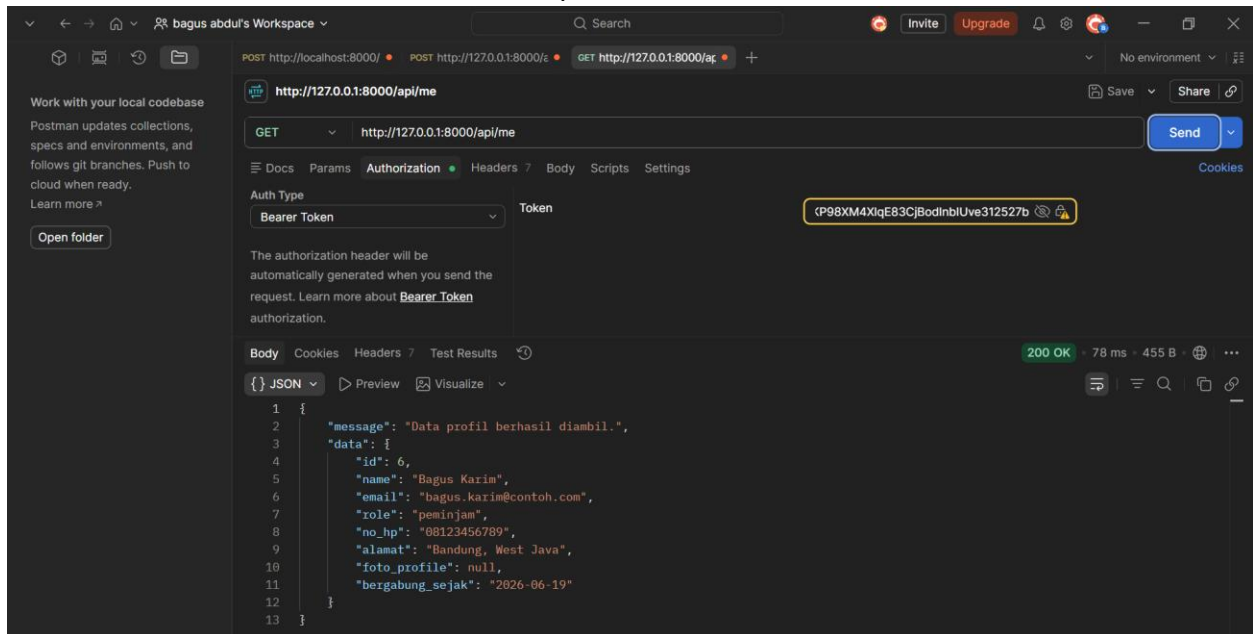
Cara Menggunakan Token untuk Endpoint yang Diproteksi

Setelah Anda mendapatkan access_token dari proses Register atau Login, Anda harus menyertakan token tersebut untuk mengakses endpoint yang diproteksi (seperti /api/me atau /api/alat).

1. Copy teks (nilai) dari access_token yang ada di response Login.
2. Buka tab baru untuk menguji endpoint profil (Method: GET, URL: <http://127.0.0.1:8000/api/me>).
3. Buka tab Authorization di Postman.
4. Pada menu dropdown Type, pilih Bearer Token.
5. Paste token yang sudah Anda copy ke dalam kolom Token.



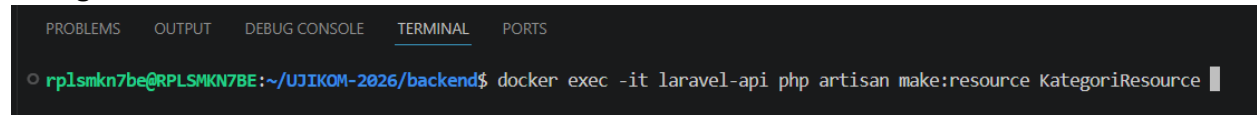
6. Klik Send. Anda akan melihat data profil Anda.



Membuat Fitur Untuk Mengelola Kategori

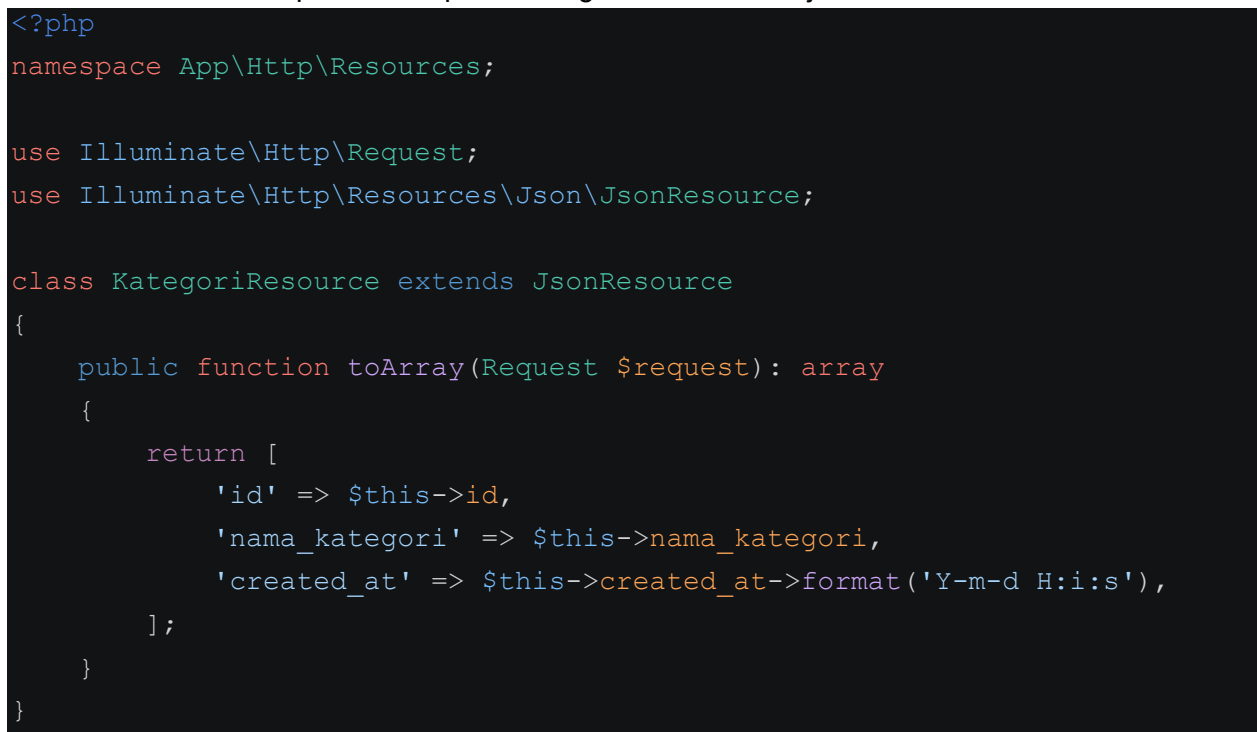
1. Implementasi API Resources (Kategori)

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:resource KategoriResource**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
○ rp1smkn7be@RPLSMKN7BE:~/UJIKOM-2026/backend$ docker exec -it laravel-api php artisan make:resource KategoriResource
```

Kemudian isi/lakukan perubahan pada KategoriResource menjadi:



```
<?php
namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class KategoriResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'nama_kategori' => $this->nama_kategori,
            'created_at' => $this->created_at->format('Y-m-d H:i:s'),
        ];
    }
}
```

2. Implementasi Form Requests

Form Requests bertugas sebagai tameng keamanan di layer HTTP yang memvalidasi data masukan (*input payload*) dari pengguna saat ingin menambahkan data alat baru ke sistem sebelum data tersebut diolah oleh Controller

StoreKategori

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request Kategori/StoreKategoriRequest**

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
rp1smkn7be@RPLSMKN7BE:~/UJIKOM-2026/backend$ docker exec -it laravel-api php artisan make:request Kategori/StoreKategoriRequest
```

Kemudian isi/lakukan perubahan pada KategoriResource menjadi:

```
<?php
namespace App\Http\Requests\Kategori;
use Illuminate\Foundation\Http\FormRequest;
class StoreKategoriRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'nama_kategori' =>
            'required|string|max:255|unique:kategori,nama_kategori',
        ];
    }
}
```

UpdateKategori

Buka terminal kemudian ketikan **docker exec -it laravel- api php artisan make:request Kategori/UpdateKategoriRequest**

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
rp1smkn7be@RPLSMKN7BE:~/UJIKOM-2026/backend$ docker exec -it laravel- api php artisan make:request Kategori/UpdateKategoriRequest
```

Kemudian isi/lakukan perubahan pada KategoriResource menjadi:

```
<?php

namespace App\Http\Requests\Kategori;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class UpdateKategoriRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // Pastikan bernilai true karena otorisasi sudah
        dihandle oleh Middleware di route
    }
}
```

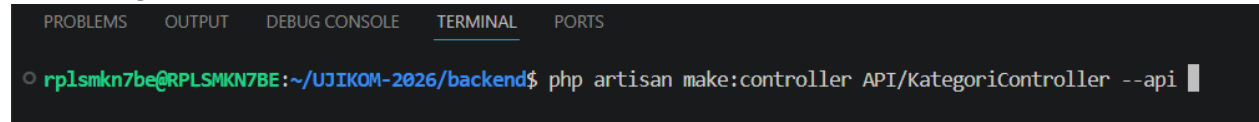
```

public function rules(): array
{
    $kategori = $this->route('kategori');
    return [
        'nama_kategori' => [
            'required',
            'string',
            'max:255',
            // Mengabaikan pengecekan unik untuk ID kategori yang
            sedang di-update
            Rule::unique('kategori', 'nama_kategori')-
            >ignore($kategori),
        ],
    ];
}
}

```

Implementasi Controllers

Buka terminal kemudian ketikkan **docker exec -it laravel- api php artisan make:controller API/KategoriController --api**



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
rp1smkn7be@RPLSMKN7BE:~/UJIKOM-2026/backend$ php artisan make:controller API/KategoriController --api

```

1. Impor Setiap File yang berhubungan dengan Kategori

```

use App\Http\Controllers\Controller;
use App\Http\Requests\Kategori\StoreKategoriRequest;
use App\Http\Requests\Kategori\UpdateKategoriRequest;
use App\Http\Resources\KategoriResource;
use App\Models\Kategori;
use Illuminate\Http\JsonResponse;

```

2. Lakukan perubahan baris kode pada blok function **index** menjadi seperti ini

```

public function index(): JsonResponse
{
    $kategori = Kategori::latest()->get();
    return response()->json([
        'message' => 'Daftar kategori berhasil diambil.',
        'data' => KategoriResource::collection($kategori)
    ]);
}

```

```

    });
}

```

Penjelasan Baris Program

Kode Program	Fungsi
<pre>\$kategori = Kategori::latest()->get();</pre>	Kategori: Model Eloquent yang terhubung dengan tabel data kategori (biasanya bernama tabel kategoris atau categories) di database. latest(): Query builder bawaan Laravel untuk mengurutkan data berdasarkan tanggal dibuat (created_at) secara menurun (descending). Artinya, kategori yang paling baru dibuat akan berada di urutan paling atas. get(): Perintah untuk mengeksekusi query tersebut ke database dan mengambil hasilnya dalam bentuk Collection (kumpulan data objek).
<pre>KategoriResource::collection(\$kategori)</pre>	KategoriResource: Lapisan penyaring (transformer) untuk mengatur struktur JSON satu data kategori. ::collection(...): Karena variabel \$kategori berisi kumpulan data (banyak kategori), kita wajib menggunakan metode collection(), bukan new KategoriResource(). Metode ini secara otomatis akan melakukan perulangan (looping) untuk menyaring setiap data kategori di dalam daftar tersebut agar formatnya seragam dan aman.

3. Lakukan perubahan baris kode pada blok function **store** menjadi seperti ini

```

public function store(StoreKategoriRequest $request): JsonResponse
{
    $kategori = Kategori::create($request->validated());
    return response()->json([
        'message' => 'Kategori berhasil ditambahkan.',
        'data' => new KategoriResource($kategori)
    ], 201);
}

```

Penjelasan Baris Kode Program

Baris kode Program	Penjelasan/Fungsi
<pre>public function store(StoreKategoriRequest \$request): JsonResponse</pre>	StoreKategoriRequest \$request: Laravel menggunakan Form Request kustom bernama StoreKategoriRequest untuk memeriksa data input dari pengguna (seperti nama kategori atau deskripsi). : JsonResponse: Menegaskan secara eksplisit bahwa fungsi

	ini wajib mengembalikan respons dalam format JSON standar API.
<code>\$kategori = Kategori::create(\$request->validated());</code>	<code>\$request->validated()</code> : Fungsi ini sangat penting untuk keamanan. Ia hanya mengambil data input yang lolos validasi saja berdasarkan aturan yang didefinisikan di <code>StoreKategoriRequest</code> . <code>Kategori::create(...)</code> : Memasukkan data aman tersebut ke dalam tabel kategori menggunakan Eloquent ORM, lalu menyimpan hasil data yang baru dibuat ke dalam variabel <code>\$kategori</code> .
<code>return response()->json([...], 201);</code>	<code>response()->json()</code> : mengembalikan data dengan format JSON. 201 : Status kode HTTP yang berarti "Created"
<code>new KategoriResource(\$kategori)</code>	Membungkus data objek kategori yang baru dibuat agar format JSON yang dikirimkan ke pengguna bersih dan sesuai standar yang ditentukan aplikasi Anda.

4. Lakukan perubahan baris kode pada blok function **show** menjadi seperti ini

```
public function show(Kategori $kategori): JsonResponse
{
    return response()->json([
        'data' => new KategoriResource($kategori)
    ]);
}
```

5. Lakukan perubahan baris kode pada blok function **update** menjadi seperti ini

```
public function update(UpdateKategoriRequest $request, Kategori $kategori): JsonResponse
{
    $kategori->update($request->validated());
    return response()->json([
        'message' => 'Kategori berhasil diperbarui.',
        'data' => new KategoriResource($kategori)
    ]);
}
```

Penjelasan Baris Kode Program

Basis Kode Program	Penjelasan/Fungsi
public function	UpdateKategoriRequest \$request : Laravel mengamankan

update(UpdateKategoriRequest \$request, Kategori \$kategori): JsonResponse	proses input menggunakan berkas validasi kustom UpdateKategoriRequest. Aturan di dalamnya bertugas memeriksa data baru yang dikirim (misalnya memastikan nama kategori tidak boleh kosong). Jika input tidak valid, Laravel langsung membatalkan proses dan mengembalikan pesan error validasi. Kategori \$kategori: Menggunakan fitur Route Model Binding. Laravel otomatis mencari data kategori yang ingin diubah ke database berdasarkan parameter ID yang dikirim lewat URL. Jika data tidak ditemukan, sistem langsung memunculkan error 404 Not Found. : JsonResponse: Menegaskan secara eksplisit bahwa fungsi ini akan mengembalikan data dalam format JSON standar API.
\$kategori->update(\$request->validated());	\$kategori->update(...): Menjalankan query UPDATE untuk mengganti data lama di database dengan data baru yang sudah aman dari inputan pengguna. \$request->validated(): Hanya mengambil data input yang lolos validasi saja. Fitur ini sangat penting untuk mencegah serangan Mass Assignment (pengubahan kolom database secara ilegal oleh pihak luar).

6. Lakukan perubahan baris kode pada blok function **destroy** menjadi seperti ini

```
public function destroy(Kategori $kategori): JsonResponse
{
    $kategori->delete();
    return response()->json([
        'message' => 'Kategori berhasil dihapus.'
    ]);
}
```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan/Fungsi
\$kategori->delete();	Baris ini memerintahkan Eloquent ORM untuk menjalankan perintah SQL DELETE ke database pada tabel kategori untuk baris data yang telah ditemukan sebelumnya.

Update route/api.php

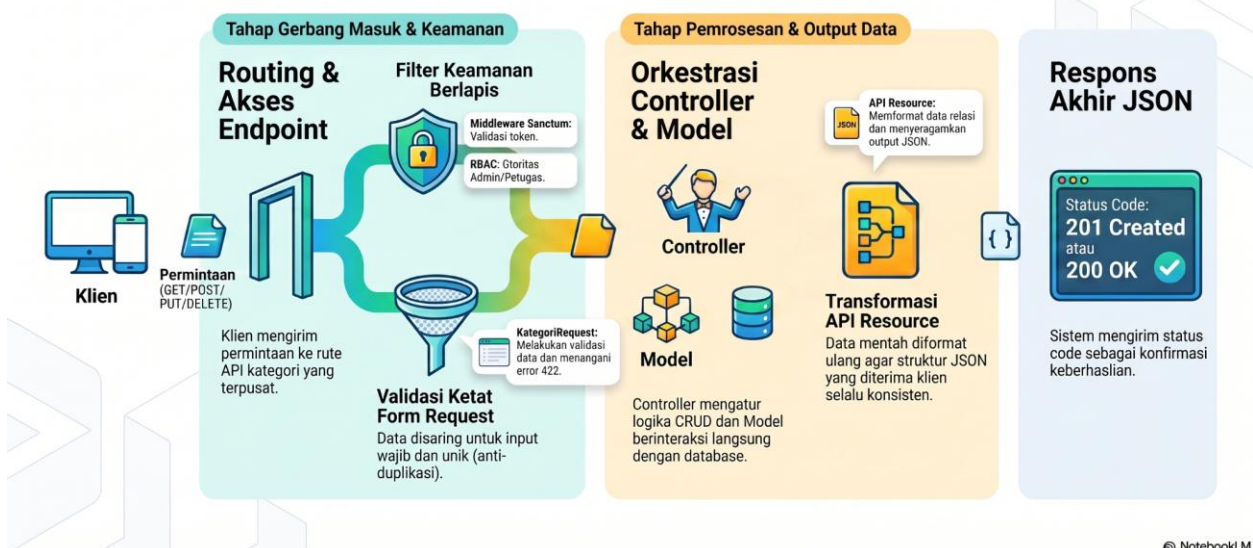
Tambahkan **Route::apiResource('kategori', KategoriController::class);** pada blok middleware dengan role admin.

```
// Hanya Admin
Route::middleware('role.admin')->group(function () {
    Route::apiResource('kategori', KategoriController::class);
});
```

Penjelasan Baris Kode Program

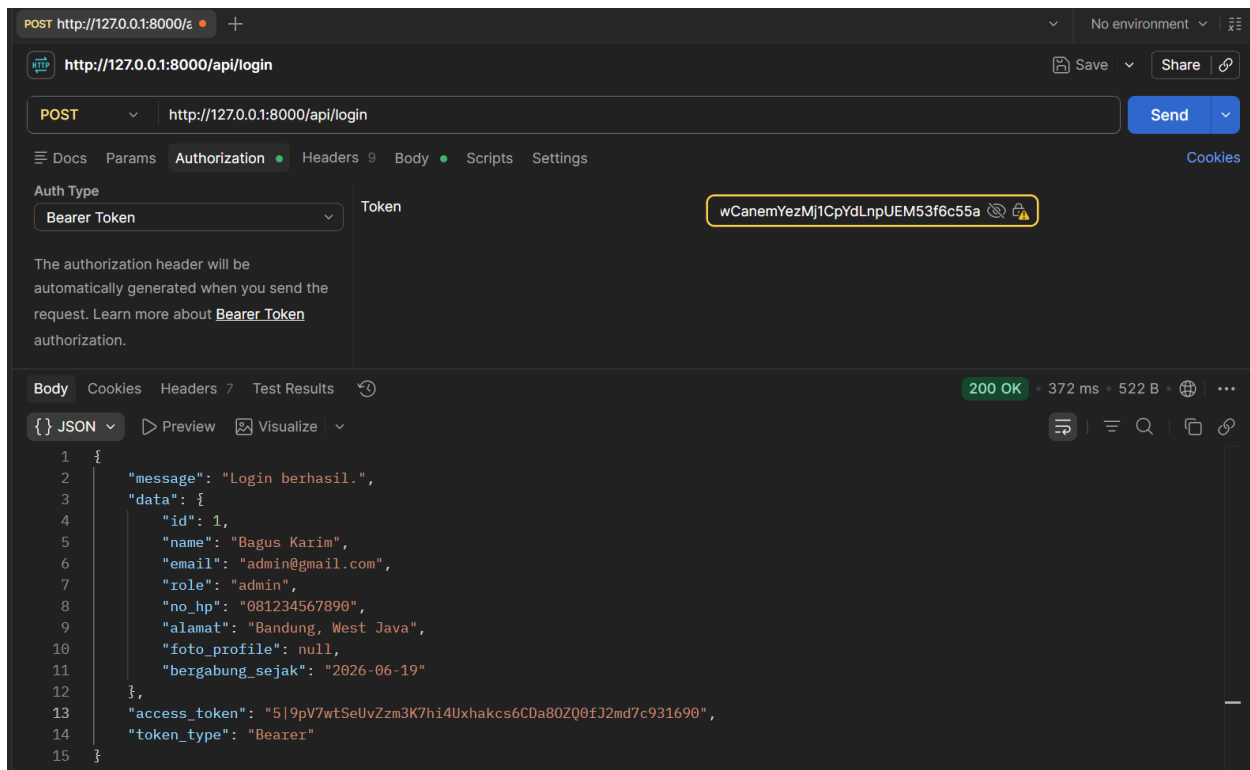
Route::	Memanggil Facade Route bawaan Laravel untuk mengatur jalur lalu lintas web/API.
apiResource	Metode khusus yang memerintahkan Laravel untuk menyediakan fungsi CRUD standar API secara otomatis.
'kategori'	Menentukan nama URL utama (endpoint). Jalur API Anda nantinya akan berawalan /kategori.
KategoriController::class	Menentukan Controller mana yang bertanggung jawab untuk memproses semua permintaan yang masuk ke URL tersebut.

Arsitektur & Alur Data: Pengelolaan Kategori Peminjaman Alat



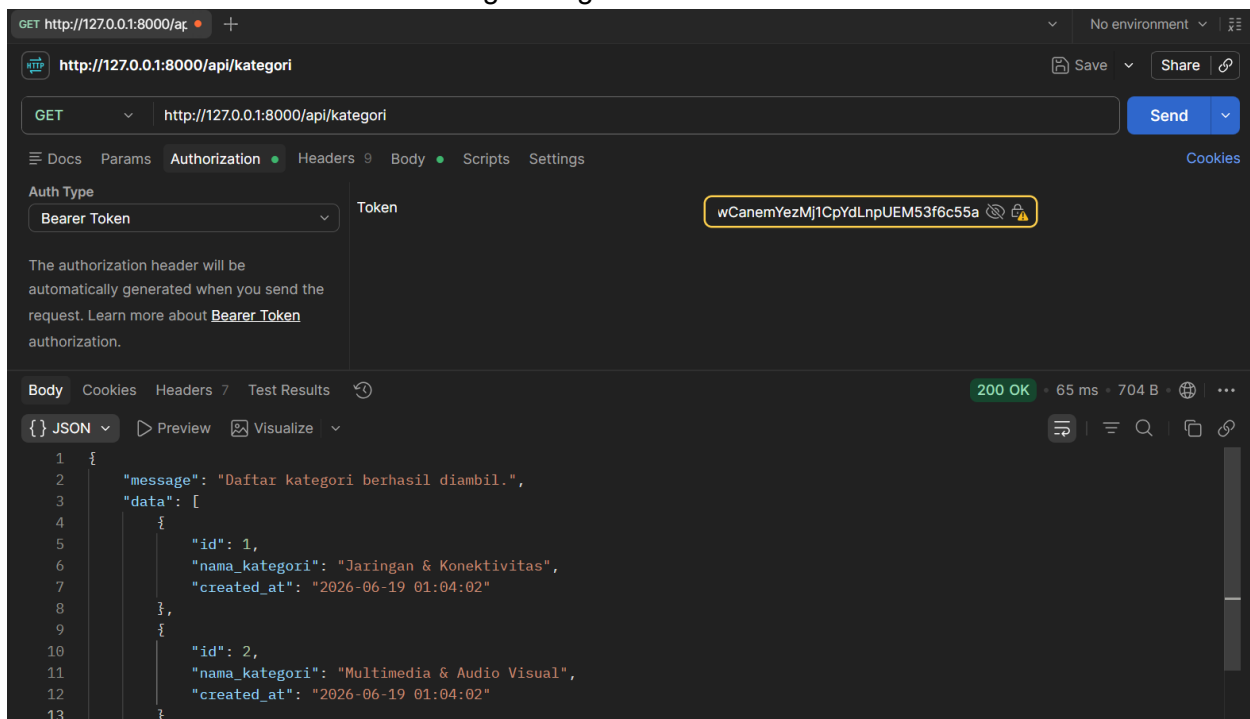
Pengujian Fitur Kategori

1. Pastikan bearer token sudah terisi dengan role admin pada postman.



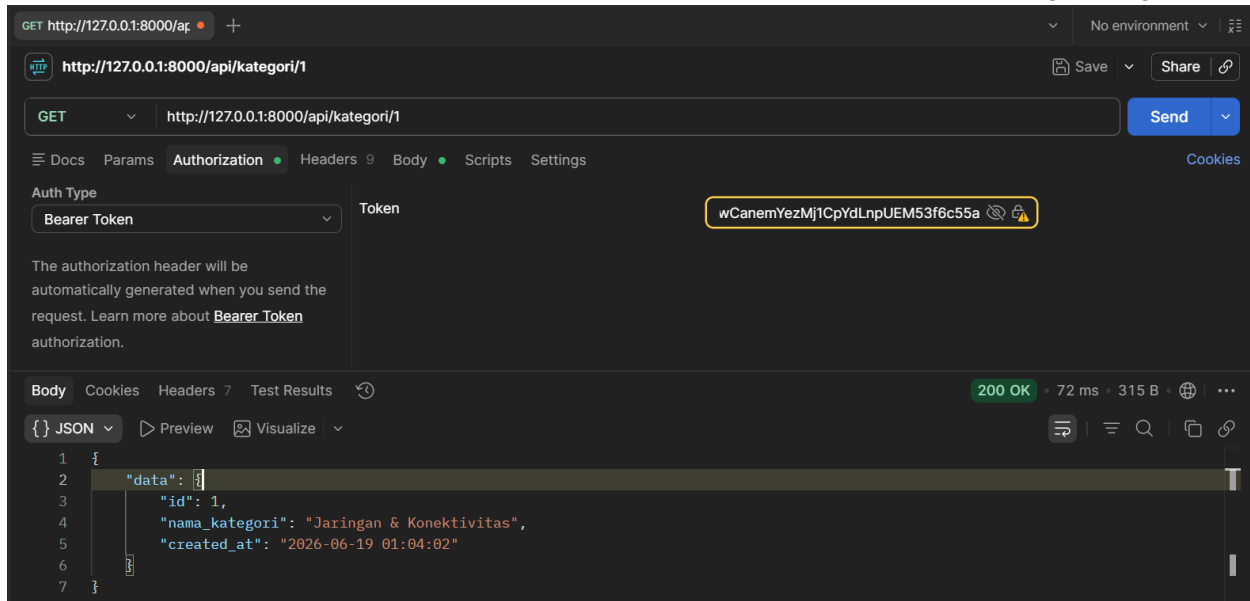
2. Pengujian Function Index

Ubah HTTP Method post menjadi **get** dan URL mengarah pada <http://127.0.0.1:8000/api/kategori> kemudian klik send. Jika tampil seperti berikut selamat **function index** sudah berfungsi dengan baik.



3. Pengujian Function Show

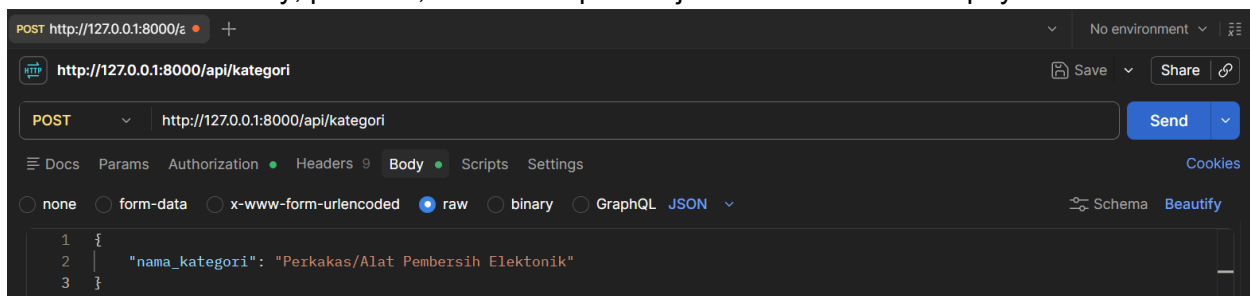
Pastikan HTTP Method masih pada **get** dan URL mengarah pada <http://127.0.0.1:8000/api/kategori/1> (mengambil data dengan id_kategori=1) kemudian klik send. Jika tampil seperti berikut selamat **function show** sudah berfungsi dengan baik.



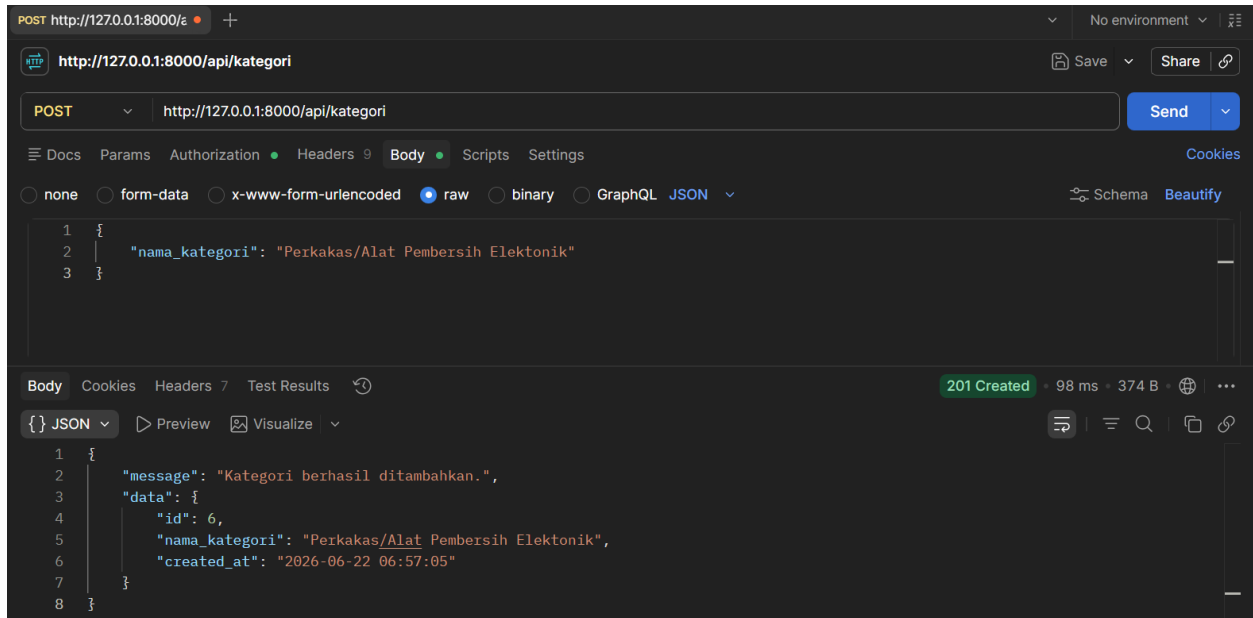
4. Pengujian Function Store

Ubah HTTP Method get menjadi **post** dan URL mengarah pada <http://127.0.0.1:8000/api/kategori>.

Buka tab Body, pilih raw, dan ubah tipe menjadi JSON. Masukkan payload berikut:

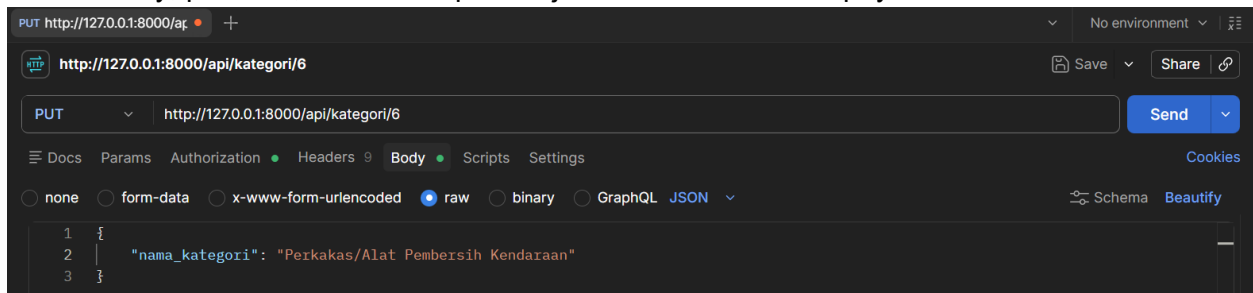


Klik Send. Hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

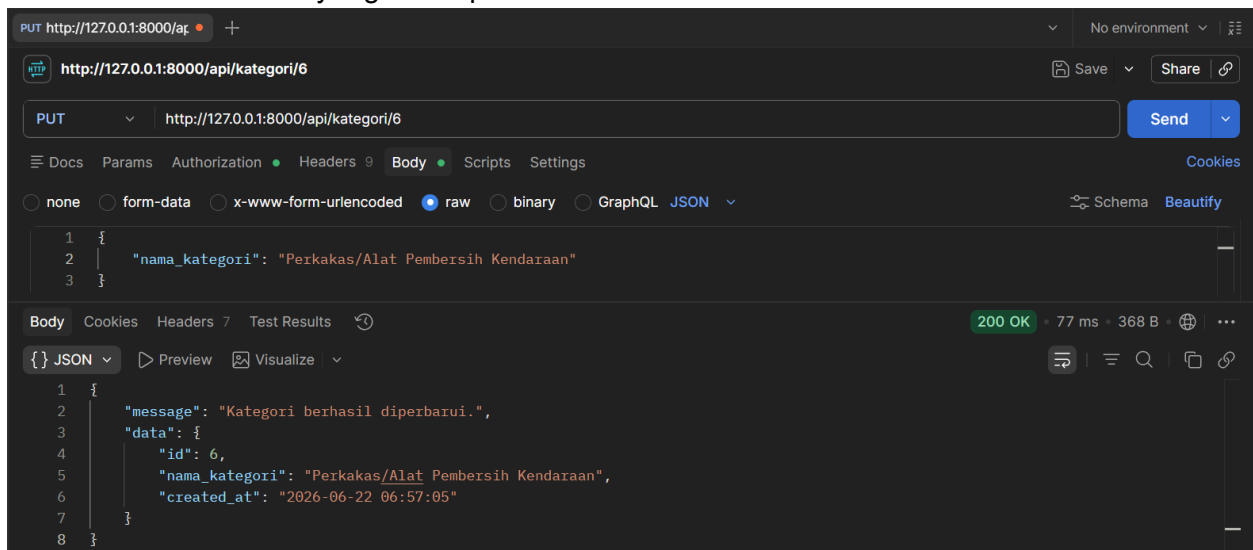


5. Pengujian Function Update

Ubah HTTP Method post menjadi **put** dan URL mengarah pada <http://127.0.0.1:8000/api/kategori/6> (mengupdate data dengan id_kategori=6). Buka tab Body, pilih raw, dan ubah tipe menjadi JSON. Masukkan payload berikut:

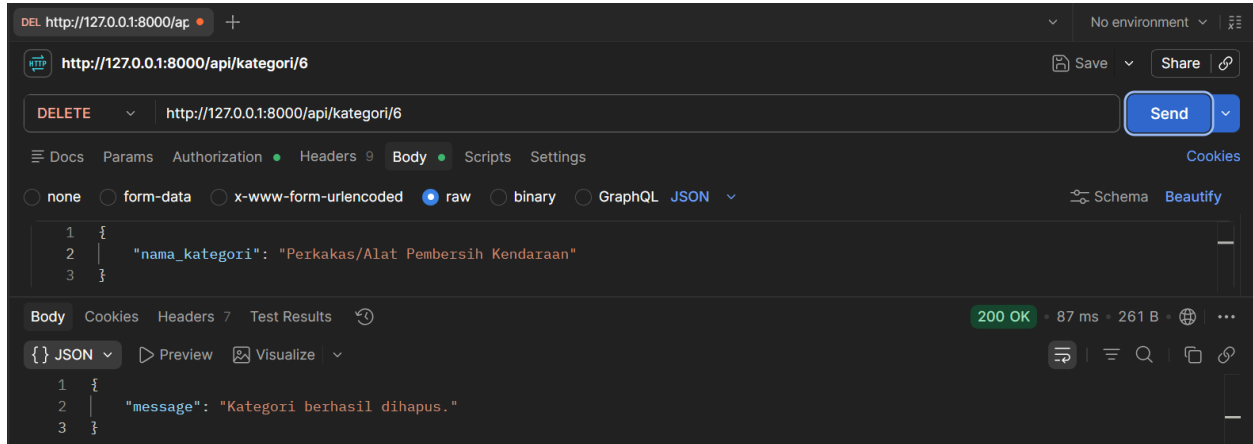


Klik Send. Hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**



6. Pengujian Function Destroy

Ubah HTTP Method put menjadi **destroy** dan URL mengarah pada <http://127.0.0.1:8000/api/kategori/6> (menghapus data dengan id_kategori=6). klik send. Jika tampil seperti berikut selamat **function show** sudah berfungsi dengan baik.



Membuat Fitur Untuk Mengelola Alat

Implementasi API Resource (Alat)

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:resource AlatResource**

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
o abdulkarim@DESKTOP-D043LCG:~/UJIKOM-2026/backend$ docker exec -it laravel-api php artisan make:resource KategoriResource
```

Kemudian isi/lakukan perubahan pada KategoriResource menjadi:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class AlatResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'nama_alat' => $this->nama_alat,
            'stok' => $this->stok,
            'status_kondisi' => $this->status_kondisi,
            'deskripsi' => $this->deskripsi,
            'gambar' => $this->gambar ? url('storage/' . $this->gambar) :
null,
            // Eager load relasi kategori jika tersedia
            'kategori' => new KategoriResource($this-
>whenLoaded('kategori')),
            'created_at' => $this->created_at->format('Y-m-d H:i:s'),
        ];
    }
}
```

Penjelasan Baris Program

Baris Kode Program	Fungsi/Pembahasan
<code>class AlatResource extends JsonResource</code>	Kelas ini akan mewarisi JsonResource yang akan memberikan kemampuan untuk membungkus (wrap) satu objek model Eloquent dan mengakses properti model tersebut secara langsung menggunakan keyword \$this .
<code>public function toArray(Request \$request): array</code>	Fungsi ini menerima objek Request saat ini (jika Anda butuh logika berbasis user role atau parameter URL) dan harus mengembalikan nilai berupa array yang nantinya otomatis dikonversi Laravel menjadi format JSON.
<code>'id' => \$this->id</code> dan properti lainnya	\$this merujuk langsung ke objek Model yang sedang ditransformasikan (misal: Model Alat). Laravel menggunakan magic method PHP sehingga \$this->nama_alat sebenarnya mengambil data dari <code>\$alat->nama_alat</code> di database.
<code>'kategori' => new KategoriResource(\$this->whenLoaded('kategori'))</code>	Fungsi \$this->whenLoaded('kategori') memberi tahu Laravel: "Hei, jangan muat data kategori KECUALI di Controller kita sudah melakukan eager loading (<code>Alat::with('kategori')</code>)". Jika di Controller tidak di-load, key kategori akan otomatis dihilangkan dari JSON, bukan bernilai null atau memicu query baru.
<code>created_at' => \$this->created_at?->format('Y-m-d H:i:s')</code>	objek created_at di Laravel adalah sebuah objek Carbon (pustaka manipulasi waktu). Kita menggunakan fungsi format() untuk menyeragamkan format waktu menjadi Tahun-Bulan-Tanggal Jam:Menit:Detik. Penggunaan tanda tanya (?->) disebut null-safe operator , berguna agar aplikasi tidak crash (Error 500) jika ada data yang kolom <code>created_at</code> -nya kosong.

Implementasi Form Requests

StoreAlat

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request Alat/StoreAlatRequest**

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
o abdulkarim@DESKTOP-D043LCG:~/UJIKOM-2026/backend$ docker exec -it laravel-api php artisan make:request Kategori/StoreAlatRequest

```

Kemudian isi/lakukan perubahan pada `AlatResource` menjadi:

```

<?php
namespace App\Http\Requests\Alat;
use Illuminate\Foundation\Http\FormRequest;

```

```

use Illuminate\Validation\Rule;

class StoreAlatRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'kategori_id' => ['required', 'integer',
Rule::exists('kategori', 'id')
            ],
            'nama_alat' => ['required', 'string', 'max:255'],
            'stok' => ['required', 'integer', 'min:0'],
            'status_kondisi' => ['required', 'string', 'max:255'],
            'deskripsi' => ['nullable', 'string'],
            'gambar' => ['nullable', 'image', 'mimes:jpeg,png,jpg',
'max:2048'], // Maksimal 2MB
        ];
    }

    //Opsional: Kustomisasi pesan error dalam bahasa Indonesia jika
validasi gagal
    public function messages(): array
    {
        return [
            'kategori_id.exists' => 'Kategori yang dipilih tidak valid
atau tidak terdaftar.',
            'stok.min' => 'Stok tidak boleh kurang dari 0.',
            'gambar.max' => 'Ukuran gambar maksimal adalah 2 MB.',
            'gambar.image' => 'File yang diunggah harus berupa gambar.',
        ];
    }
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Pembahasan
--------------------	------------

public function authorize(): bool	Metode ini mengecek apakah pengguna yang sedang mengakses rute ini memiliki hak akses (izin). Jika mengembalikan true, proses validasi lanjut ke tahap aturan (rules). Jika false, Laravel langsung melempar respons 403 Forbidden otomatis tanpa menjalankan Controller.
'kategori_id' => 'required exists:kategori,id'	'required' memastikan client wajib mengirim data ini. 'exists:kategori,id' bertindak sebagai pemeriksa integritas database (foreign key checker). Laravel akan mengecek ke tabel kategori pada kolom id, apakah ID yang dikirim oleh pengguna benar-benar ada di sana. Ini mencegah error <i>Integrity Constraint Violation</i> saat data disimpan.
'stok' => 'required integer min:0'	'integer' memastikan nilai yang masuk adalah angka bulat (bukan huruf atau desimal). 'min:0' adalah batas logika bisnis yang sangat penting; fungsi ini memastikan pengguna tidak bisa memasukkan nilai stok minus (misal: -5).
'deskripsi' => 'nullable string'	Aturan 'nullable' berarti kolom ini tidak wajib diisi oleh pengguna (boleh kosong). Namun, jika pengguna memutuskan untuk mengisinya, isinya harus berupa teks atau 'string'
'gambar' => 'nullable image mimes:jpeg,png,jpg max:2048'	'image' mendeteksi bahwa file tersebut benar-benar berkas gambar (melalui deteksi tipe MIME, bukan sekadar ekstensi file palsu). 'mimes:jpeg,png,jpg' membatasi jenis format gambar yang diizinkan agar server Anda tidak menerima format aneh. 'max:2048' membatasi ukuran file dalam satuan kilobita (KB), di mana 2048 KB setara dengan 2 Megabita (MB).

UpdateAlat

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request Alat/UpdateAlatRequest**

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
o abdulkarim@DESKTOP-D043LCG:~/UJIKOM-2026/backend$ docker exec -it laravel-api php artisan make:request Kategori/UpdateAlatRequest

```

Kemudian isi/lakukan perubahan pada KategoriResource menjadi:

```

<?php

namespace App\Http\Requests\Alat;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class UpdateAlatRequest extends FormRequest
{
    public function authorize(): bool

```

```

    {
        return true;
    }
    public function rules(): array
    {
        // Mengambil ID alat yang sedang di-update dari parameter rute URL
        $alatId = $this->route('alat');
        return [
            'kategori_id' => ['required', 'integer',
Rule::exists('kategori', 'id')],
            'nama_alat' => ['required', 'string', 'max:255'],
            'stok' => ['required', 'integer', 'min:0'],
            'status_kondisi' => ['required', 'string', 'max:255'],
            'deskripsi' => ['nullable', 'string'],
            'gambar' => ['nullable', 'image', 'mimes:jpeg,png,jpg',
'max:2048'],
        ];
    }
}

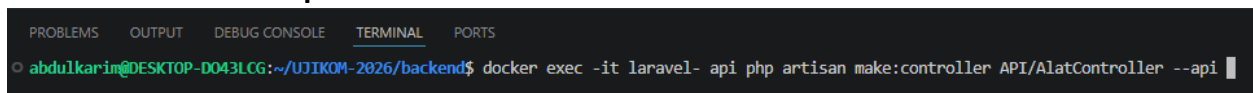
```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
<code>\$this->route('alat')</code>	Pada proses update, aplikasi perlu tahu data ID mana yang sedang diubah. Peta URL biasanya berbentuk <code>/alat/{id}</code> . Fungsi <code>\$this->route('parameter_name')</code> digunakan untuk menangkap ID tersebut langsung dari dalam kelas Request, yang sangat berguna jika kita perlu menerapkan validasi bersyarat seperti <code>Rule::unique()->ignore()</code> .
<code>'kategori_id' => ['required', 'integer', Rule::exists('kategori', 'id')]</code>	Saat mengubah data alat, user bisa saja memindahkan alat tersebut ke kategori lain. Aturan ini memastikan bahwa kategori tujuan yang baru diinput oleh user memang benar-benar eksis di database.

Implementasi Controllers

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:controller API/AlatController --api**



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
o abdulkarim@DESKTOP-D043LCG:~/UJIKOM-2026/backend$ docker exec -it laravel-api php artisan make:controller API/AlatController --api

```

Lakukan perubahan baris kode program menjadi:

```

<?php

namespace App\Http\Controllers\API;
use App\Http\Controllers\Controller;
use App\Http\Requests\Alat\StoreAlatRequest;
use App\Http\Requests\Alat\UpdateAlatRequest;
use App\Http\Resources\AlatResource;
use App\Models\Alat;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Storage;

class AlatController extends Controller
{
    public function index(): JsonResponse
    {
        // Tetap menggunakan eager loading untuk mencegah N+1 Query
        $alat = Alat::with('kategori')->latest()->get();
        return response()->json([
            'message' => 'Daftar alat berhasil diambil.',
            'data' => AlatResource::collection($alat)
        ]);
    }

    public function store(StoreAlatRequest $request): JsonResponse
    {
        $data = $request->validated();
        $alat = DB::transaction(function () use ($request, $data) {
            if ($request->hasFile('gambar')) {
                $data['gambar'] = $request->file('gambar')->store('alat',
'public');
            }
            return Alat::create($data);
        });
        return response()->json([
            'message' => 'Alat berhasil ditambahkan.',
            'data' => new AlatResource($alat->load('kategori'))
        ], 201);
    }

    public function show(Alat $alat): JsonResponse
    {

```

```

// Menggunakan Route Model Binding ($alat) ikombinasikan dengan load()
return response()->json([
    'data' => new AlatResource($alat->load('kategori'))
]);
}

public function update(UpdateAlatRequest $request, Alat $alat):
JsonResponse
{
    $data = $request->validated();
    $oldGambar = $alat->gambar;//Simpan path gambar lama lebih dahulu

    DB::transaction(function () use ($request, &$amp;$data, $alat,
$oldGambar) {
        if ($request->hasFile('gambar')) {
            $data['gambar'] = $request->file('gambar')->store('alat',
'public');

            // Hapus gambar lama dari server HANYA setelah gambar baru
sukses masuk database
            if ($oldGambar) {
                Storage::disk('public')->delete($oldGambar);
            }
        }
        $alat->update($data);
    });
    return response()->json([
        'message' => 'Alat berhasil diperbarui.',
        'data' => new AlatResource($alat->load('kategori'))
    ]);
}

public function destroy(Alat $alat): JsonResponse
{
    DB::transaction(function () use ($alat) {
        if ($alat->gambar) {
            Storage::disk('public')->delete($alat->gambar);
        }
        $alat->delete();
    });
    return response()->json([
        'message' => 'Alat berhasil dihapus.'
    ]);
}

```

```

    }

    public function katalog(): JsonResponse
    {
        $alat = Alat::with('kategori')->tersedia()->latest()->get();
        return response()->json([
            'message' => 'Katalog alat tersedia.',
            'data' => AlatResource::collection($alat)
        ]);
    }
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Pembahasan
\$alat = Alat::with('kategori')->latest()->get(); (index)	with('kategori') memerintahkan Laravel untuk melakukan Eager Loading. Laravel akan mengambil data alat dan data kategori sekaligus dalam 2 query utama, sehingga performa aplikasi tetap cepat meskipun data berjumlah ribuan. Fungsi latest() adalah singkatan dari orderBy('created_at', 'desc') untuk menampilkan data terbaru.
\$data = \$request->validated(); (store & update)	Kita tidak menggunakan <i>\$request->all()</i> . Fungsi validated() memastikan hanya data yang lolos dari aturan pengetatan di kelas Form Request saja yang boleh masuk ke database. Ini mengunci celah keamanan <i>Mass Assignment Vulnerability</i> .
\$request->file('gambar')->store('alat', 'public');	Metode store() bertugas mengunggah file. Parameter pertama ('alat') adalah nama folder tujuan di dalam direktori penyimpanan. Parameter kedua ('public') memberi tahu Laravel untuk menyimpannya di disk publik agar nantinya file tersebut bisa diakses melalui URL internet oleh aplikasi frontend.
public function show(Alat \$alat)	Ini disebut Route Model Binding . Di parameter fungsi, kita langsung mengetikkan Type-hint Modelnya (Alat \$alat). Laravel secara otomatis akan mencari ID yang dikirim melalui URL rute di database. Jika ID tidak ditemukan, Laravel langsung mengembalikan respons <i>404 Not Found</i> secara otomatis tanpa kita perlu menulis perintah <i>if-else</i> manual.
\$alat->load('kategori')	Berbeda dengan metode with() yang dipakai sebelum kueri dijalankan, metode load() digunakan untuk melakukan Lazy Eager Loading . Gunanya adalah memuat data relasi kategori pada objek model Eloquent yang sudah terlanjur diambil dari database.

Update Model User

Lakukan penambahan function scopeTersedia pada model user sebagai berikut:

```
public function scopeTersedia($query)
{
    return $query->where('stok', '>', 0)->where('status_kondisi',
'Baik');
}
```

Update route/api.php

Tambahkan **Route::apiResource('alat', AlatController::class);** dan **Route::get('/katalog', [AlatController::class, 'katalog']);** pada blok middleware dengan role admin.

Tambahkan **Route::get('/katalog', [AlatController::class, 'katalog']);** pada blok middleware dengan role peminjam.

Arsitektur Aliran Data API: Sistem Peminjaman Alat



Pengujian Fitur Alat

1. Pastikan bearer token sudah terisi dengan role admin pada postman.
2. Untuk pengujian function index dan pengujian function show sama seperti pengujian sebelumnya (alat)
3. Pengujian Function Store

Pastikan **Method:** POST | **URL:** <http://127.0.0.1:8000/api/alat>, kemudian Buka tab Body, lalu pilih form-data:

UJIKOM-2026 / Store Alat

Save Share

POST http://127.0.0.1:8000/api/alat Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

	Key		Value	Description	
☒	kategori_id	Text	1		
☒	nama_alat	Text	AP UniFi U6 Pro		
☒	stok	Text	5		
☒	status_kondisi	Text	Baru		
☒	deskripsi	Text	Access Point WiFi 6 dual-band dengan kec...		
☒	gambar	File	Ubiquiti.png		

Klik Send. Hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

Body Cookies Headers (7) Test Results 201 Created 74 ms 688 B Save Response

{ } JSON Preview Visualize

```

1 {
2   "message": "Alat berhasil ditambahkan.",
3   "data": {
4     "id": 7,
5     "nama_alat": "AP UniFi U6 Pro",
6     "stok": 5,
7     "status_kondisi": "Baru",
8     "deskripsi": "Access Point WiFi 6 dual-band dengan kecepatan hingga 5.3 Gbps, cocok untuk area dengan
9       trafik padat",
10    "gambar": "http://127.0.0.1:8000/storage/alat/QemyV78D8PqntIoPiD0La84i3svct4B2JxYHFSi4.png",
11    "kategori": {
12      "id": 1,
13      "nama_kategori": "Jaringan & Konektivitas",
14      "created_at": "2026-06-11 10:13:32"
15    },
16    "created_at": "2026-06-25 14:07:01"
17  }

```

4. Pengujian Function Update Metode Spoofing

Di Laravel (dan PHP pada umumnya), HTTP method PUT seringkali bermasalah dalam membaca multipart/form-data (file upload). Oleh karena itu, kita tetap menggunakan method POST, namun kita "tipu" Laravel dengan menambahkan parameter `_method`. Langkahnya sebagai berikut:
Pastikan **Method**: POST | **URL**: <http://127.0.0.1:8000/api/alat>, kemudian Buka tab Body, lalu pilih form-data:

Update Alat

Save

Share

POST

http://127.0.0.1:8000/api/alat/7

Send

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	Key		Value	Description		Bulk Edit
☒	_method	Text	PUT			
☒	kategori_id	Text	1			
☒	nama_alat	Text	AP UniFi U6 Pro XXX			
☒	stok	Text	5			
☒	status_kondisi	Text	Baru			
☒	deskripsi	Text	Access Point WiFi 6 dual-band dengan kec...			
☒	gambar	Text				
	Key	Text	Value	Description		

Klik Send. Hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

Body

Cookies

Headers (7)

Test Results

200 OK

30 ms

604 B

{ } JSON

Preview

Visualize

```

1  {
2    "message": "Alat berhasil diperbarui.",
3    "data": {
4      "id": 7,
5      "nama_alat": "AP UniFi U6 Pro XXX",
6      "stok": 5,
7      "status_kondisi": "Baru",
8      "deskripsi": "Access Point WiFi 6 dual-band dengan kecepatan hingga 5.3 Gbps, cocok untuk area dengan
9        trafik padat",
10     "gambar": null,
11     "kategori": {
12       "id": 1,
13       "nama_kategori": "Jaringan & Konektivitas",
14       "created_at": "2026-06-11 10:13:32"
15     },
16     "created_at": "2026-06-25 14:07:01"
17   }

```

5. Pengujian function destroy sama seperti pengujian sebelumnya (alat).

Tugas Pengujian

Silahkan lakukan pengujian pada function katalog dengan hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.** Perhatikan pada Guru yang mengampu dikelas jika tugas sudah selesai untuk melakukan penilaian.!

Membuat Fitur User

Implementasi API Resource (User)

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:resource UserResource**

Kemudian isi/lakukan perubahan pada UserResource menjadi:

```
<?php

namespace App\Http\Resources;
use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class UserResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->email,
            'role' => $this->role,
            'no_hp' => $this->no_hp,
            'alamat' => $this->alamat,
            'foto_profile' => $this->foto_profile ? url('storage/' .
$this->foto_profile) : null,
            'bergabung_sejak' => $this->created_at?->format('Y-m-d'),
        ];
    }
}
```

Implementasi Form Requests

StoreUser

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request User/StoreUserRequest**

Kemudian isi/lakukan perubahan pada StoreUserRequest menjadi:

```
<?php
```

```

namespace App\Http\Requests\User;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rules\Password;
use Illuminate\Validation\Rule;

class StoreUserRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255',
                Rule::unique('users', 'email') // Menggunakan class Rule
agar lebih clean
            ],
            'password' => ['required', 'string',
                Password::min(8)->letters()->numbers()
            ],
            'role' => ['required',
                Rule::in(['admin', 'petugas', 'peminjam']) //input hanya
boleh dari opsi ini
            ],
            'no_hp' => ['nullable', 'string', 'max:15'],
            'alamat' => ['nullable', 'string'],
            'foto_profile' => ['nullable', 'image', 'mimes:jpeg,png,jpg',
'max:2048'],
        ];
    }
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
'email' => 'required string email max:25	'email' memastikan format teks yang dimasukkan pengguna memiliki struktur email yang benar (mengandung @ dan

5 unique:users,email'	.com/id/dll) 'unique:users,email' memerintahkan Laravel untuk memeriksa tabel users pada kolom email. Jika email tersebut sudah terdaftar oleh pengguna lain, proses pendaftaran langsung digagalkan demi mencegah duplikasi akun.
Password::min(8)->letters()->numbers()	Dibandingkan hanya menulis min:8 , menggunakan kelas bawaan <i>Illuminate\Validation\Rules>Password</i> membuat Anda bisa menerapkan standar keamanan industri secara mudah. Fungsi letters() mewajibkan adanya minimal satu huruf, dan numbers() mewajibkan adanya minimal satu angka di dalam password
'role' => 'required in:admin,petugas,p eminjam'	in:... bertindak sebagai validasi enum di layer HTTP. Ini mengunci pilihan peran pengguna. Jika peretas mencoba memanipulasi bodi request dengan mengirimkan 'role' => 'super_god_admin', Laravel akan langsung menolaknya karena string tersebut tidak terdaftar di dalam daftar opsi yang didefinisikan
'no_hp' => 'nullable string max:15'	Nomor HP dikategorikan sebagai ' string ', bukan 'integer'. Ini adalah praktik terbaik (best practice) karena nomor telepon sering kali dimulai dengan angka 0 atau simbol + (misal: +62). Jika disimpan sebagai integer, angka 0 atau simbol + di depan akan otomatis terhapus atau memicu kegagalan sistem.

UpdateUser

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request**

User/StoreUserRequest

Kemudian isi/lakukan perubahan pada UpdateUserRequest menjadi:

```
<?php

namespace App\Http\Requests\User;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules>Password;

class UpdateUserRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }
}
```

```

public function rules(): array
{
    $user = $this->route('user');
    // Memastikan kita mendapatkan ID (mengantisipasi jika parameter
route berupa objek Model)
    $userId = $user instanceof \App\Models\User ? $user->id : $user;

    return [
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'email', 'max:255',
            // Mengabaikan ID user yang sedang di-update agar tidak
memicu error "Email sudah terdaftar"
            Rule::unique('users', 'email')->ignore($userId),
        ],
        'password' => ['nullable', 'string', Password::min(8)-
>letters()->numbers()],
        'role' => ['required',
            Rule::in(['admin', 'petugas', 'peminjam'])
        ],
        'no_hp' => ['nullable', 'string', 'max:15'],
        'alamat' => ['nullable', 'string'],
        'foto_profile' => ['nullable', 'image', 'mimes:jpeg,png,jpg',
'max:2048'],
    ];
}
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
Rule::unique('users', 'email')->ignore(\$userId)	Jika Anda hanya memakai aturan unique:users,email biasa tanpa fungsi ignore() , maka saat user bernama Budi ingin memperbarui nomor HP-nya tanpa mengubah email-nya (budi@gmail.com), Laravel akan menolak request tersebut dengan pesan error "Email sudah terdaftar". Mengapa? Karena Laravel mendeteksi email budi@gmail.com tersebut sudah eksis di tabel database (milik Budi sendiri). Dengan menambahkan ignore(\$userId) , kita memberi tahu database: "Cek keunikan email ini ke semua baris data, KECUALI untuk baris milik user dengan ID ini"

'password' => ['nullable', 'string', ...]	Berbeda saat pendaftaran baru (Store) yang mewajibkan password, pada proses pembaruan data (Update), kolom password wajib diatur menjadi 'nullable'. Artinya, jika user mengosongkan kolom password, Laravel menganggap user tidak ingin mengganti password lamanya. Pengecekan kekuatan password baru akan aktif dijalankan hanya jika kolom tersebut diisi teks oleh pengguna.
---	--

Implementasi Controllers

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:controller**

API/UserController --api

Lakukan perubahan baris kode program menjadi:

```
<?php

namespace App\Http\Controllers\API;
use App\Http\Controllers\Controller;
use App\Http\Requests\User\StoreUserRequest;
use App\Http\Requests\User\UpdateUserRequest;
use App\Http\Resources\UserResource;
use App\Models\User;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Storage;

class UserController extends Controller
{
    public function index(): JsonResponse
    {
        $users = User::latest()->get();
        return response()->json([
            'message' => 'Daftar pengguna berhasil diambil.',
            'data' => UserResource::collection($users)
        ]);
    }

    public function store(StoreUserRequest $request): JsonResponse
    {
        $data = $request->validated();
        $user = DB::transaction(function () use ($request, $data) {
```

```

        $data['password'] = Hash::make($data['password']);

        if ($request->hasFile('foto_profile')) {
            $data['foto_profile'] = $request->file('foto_profile')->store('profiles', 'public');
        }

        return User::create($data);
    });

    return response()->json([
        'message' => 'Pengguna berhasil ditambahkan.',
        'data' => new UserResource($user)
    ], 201);
}

public function show(User $user): JsonResponse
{
    return response()->json([
        'data' => new UserResource($user)
    ]);
}

public function update(UpdateUserRequest $request, User $user):
JsonResponse
{
    $data = collect($request->validated());

    DB::transaction(function () use ($request, $data, $user) {
        if ($data->get('password')) {
            $data['password'] = Hash::make($data['password']);
        } else {
            $data->forget('password'); //Sama fungsinya dengan unset()
        }

        if ($request->hasFile('foto_profile')) {
            if ($user->foto_profile) {
                Storage::disk('public')->delete($user->foto_profile);
            }

            $data['foto_profile'] = $request->file('foto_profile')->store('profiles', 'public');
        }

        $user->update($data->toArray());
    });

    return response()->json([

```

```

        'message' => 'Data pengguna berhasil diperbarui.',
        'data' => new UserResource($user)
    ]);
}
public function destroy(User $user): JsonResponse
{
    DB::transaction(function () use ($user) {
        if ($user->foto_profile) {
            Storage::disk('public')->delete($user->foto_profile);
        }
        $user->delete();
    });
    return response()->json([
        'message' => 'Pengguna berhasil dihapus.'
    ]);
}
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
\$data['password'] = Hash::make(\$data['password']);	Menyimpan password dalam bentuk teks biasa (plain text) di database adalah pelanggaran keamanan berat. Hash::make() menggunakan algoritma Bcrypt (atau Argon2 tergantung konfigurasi) untuk mengubah password menjadi string acak searah yang tidak bisa didekripsi kembali. Ini memastikan data aman walau database bocor.
if (!empty(\$data['password'])) { ... } else { unset(\$data['password']); }	Ini logika krusial untuk proses update. Jika pengguna tidak mengisi input password, nilainya di array \$data akan kosong. Menggunakan unset() (atau forget()) bertugas menghapus key password beserta nilainya dari struktur array payload. Dengan begitu, metode \$user->update(\$data) hanya akan memperbarui kolom database yang tersisa di array (seperti name atau email) tanpa mengusik kolom password lama.
Storage::disk('public')->delete(\$user->foto_profile);	Pustaka (Facade) Storage dengan disk public digunakan untuk menghapus file fisik foto profil lama yang tersimpan di dalam folder <i>storage/app/public/profiles</i> . Menghapus berkas lama sebelum mengunggah berkas baru mencegah penumpukan file sampah yang tidak terpakai di kapasitas penyimpanan server.

Update model User

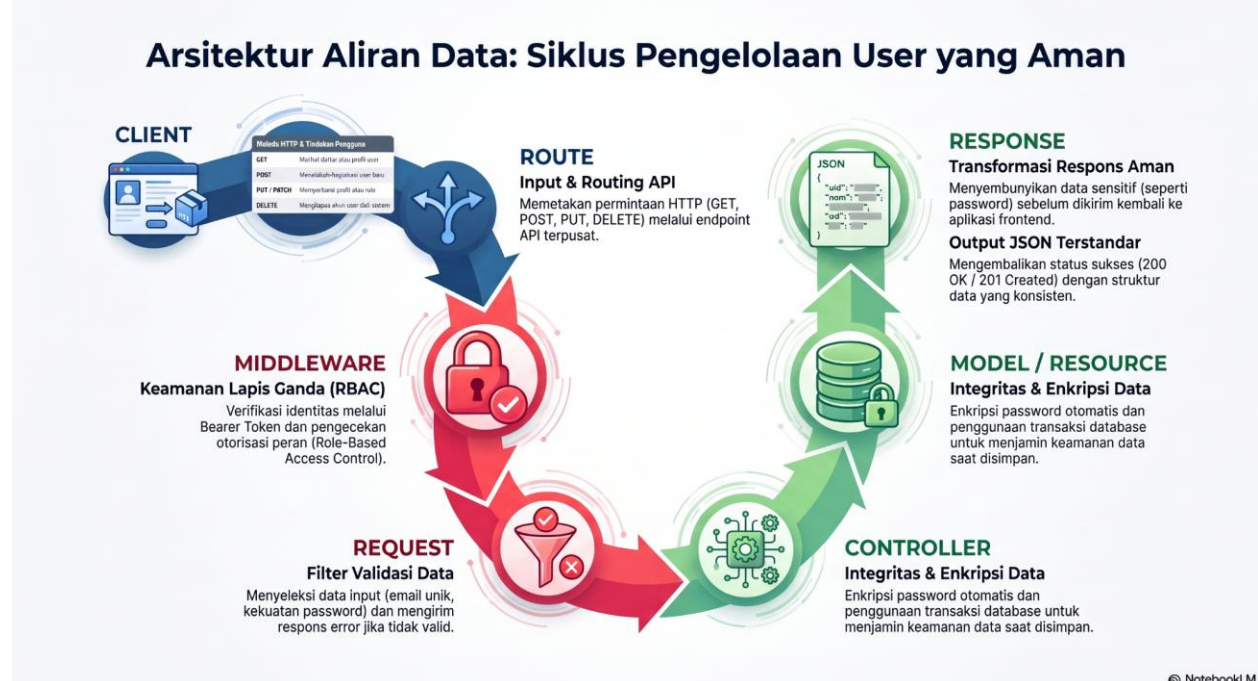
Update model user dengan melakukan penambahan function baru dengan kode program berikut:

```
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed', // Laravel otomatis meng-hash teks
        apapun yang masuk ke properti password!
    ];
}
```

Update route/api.php

Tambahkan `Route::apiResource('users', UserController::class);` pada blok middleware dengan role admin.

```
// Hanya Admin
Route::middleware('role.admin')->group(function () {
    ... Route::apiResource('kategori', KategoriController::class);
    ... Route::apiResource('alat', AlatController::class);
    ... Route::get('/katalog', [AlatController::class, 'katalog']);
    ... Route::apiResource('users', UserController::class);
});
```



Tugas Pengujian

Silahkan lakukan pengujian pada setiap function pada **UserController** dengan hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

Perlihatkan pada Guru yang mengampu dikelas jika tugas sudah selesai untuk melakukan penilaian !

Membuat Fitur Peminjaman

Implementasi API Resource

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:resource PeminjamanResource**

Kemudian isi/lakukan perubahan pada PeminjamanResource menjadi:

```
<?php

namespace App\Http\Resources;
use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class PeminjamanResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'peminjam' => $this->whenLoaded('user', fn() => $this->user?->name),
            'tgl_pinjam' => $this->tgl_pinjam?->format('Y-m-d'),
            'tgl_kembali_plan' => $this->tgl_kembali_plan?->format('Y-m-d'),
            'status' => $this->status,
            'item_dipinjam' => $this->whenLoaded('detailPinjam', function () {
                return $this->detailPinjam->map(function ($detail) {
                    return [
                        'nama_alat' => $detail->alat?->nama_alat ?? 'Alat Dihapus/Tidak Ditemukan',
                        'jumlah' => (int) $detail->jumlah,
                    ];
                });
            }),
            'info_pengembalian' => $this->whenLoaded('pengembalian', function () {
                if (!$this->pengembalian) return null;
                return [

```

```

        'tgl_kembali' => $this->pengembalian->tgl_kembali?-
>format('Y-m-d'),
        'kondisi' => $this->pengembalian->kondisi_kembali,
        'denda' => (int) $this->pengembalian->denda,
        'petugas_penerima' => $this->pengembalian->petugas?-
>name ?? 'Sistem',
    ];
    })),
];
}
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
'peminjam' => \$this->whenLoaded('user', fn() => \$this->user->name)	Ini menggunakan penulisan arrow function (fn() =>) yang diperkenalkan sejak PHP 7.4 . Ini adalah cara singkat untuk mengekstrak satu nilai dari relasi tanpa harus menulis blok fungsi penuh. Nilai nama peminjam hanya akan disisipkan ke JSON jika Controller memanggil with('user').
\$this->detailPinjam->map(function (\$detail) { ... })	Relasi detailPinjam mengembalikan sebuah Laravel Collection (kumpulan data array objek). Fungsi map() digunakan untuk mengiterasi (melakukan perulangan) setiap baris item pinjaman tersebut dan membentuk ulang (re-mapping) struktur datanya menjadi array kustom yang baru, membuang data kolom yang tidak perlu seperti created_at milik tabel pivot.
'nama_alat' => \$detail->alat->nama_alat ?? 'Alat Dihapus'	ini adalah penanganan relasi bersarang (nested relation). Struktur jalurnya adalah Peminjaman -> DetailPinjam -> Alat . Operator Null Coalescing (??) digunakan untuk mencegah aplikasi crash jika di database master data alat tersebut sudah dihapus (atau di-soft delete), sehingga API tetap mengembalikan string alternatif 'Alat Dihapus' dengan aman.

Implementasi Form Request Peminjaman

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request Peminjaman/StorePeminjamanRequest**

Kemudian isi/lakukan perubahan pada StorePeminjamanRequest menjadi:

```

<?php

namespace App\Http\Requests\Peminjaman;
use Illuminate\Foundation\Http\FormRequest;

```

```

use Illuminate\Validation\Rule;

class StorePeminjamanRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'tgl_kembali_plan' => ['required', 'date', 'date_format:Y-m-d', 'after_or_equal:today'],
            'items' => ['required', 'array', 'min:1'],
            'items.*.alat_id' => ['required', 'integer', Rule::exists('alat', 'id')],
            'items.*.jumlah' => ['required', 'integer', 'min:1'],
        ];
    }

    public function messages(): array
    {
        return [
            'tgl_kembali_plan.required' => 'Tanggal rencana pengembalian wajib diisi.',
            'tgl_kembali_plan.date_format' => 'Format tanggal harus Tahun-Bulan-Tanggal (YYYY-MM-DD).',
            'tgl_kembali_plan.after_or_equal' => 'Tanggal rencana kembali tidak boleh di masa lalu.',
            'items.required' => 'Anda harus memilih minimal satu alat untuk dipinjam.',
            'items.array' => 'Format data item yang dikirim harus berupa daftar/list.',
            'items.min' => 'Anda harus memilih minimal satu alat untuk dipinjam.',
        ];
    }

    public function attributes(): array
    {
        return [
            'items.*.alat_id' => 'Alat',
        ];
    }
}

```

```

        'items.*.jumlah' => 'Jumlah barang',
    ];
}
}

```

Penjelasan Baris Kode Program

Baris KodeProgram	Penjelasan
'tgl_kembali_plan' => 'required date after_or_equal:today'	' after_or_equal:today ' bertindak sebagai logika bisnis preventif di layer HTTP. Aturan ini memastikan pengguna tidak bisa memasukkan tanggal rencana pengembalian kemarin atau hari-hari sebelumnya, yang mana akan merusak integritas alur waktu pelaporan transaksi di database
'items' => 'required array min:1'	Sebelum memeriksa isi barangnya, Laravel wajib memastikan struktur data items yang dikirim dari client benar-benar berwujud struktur array tunggal/koleksi, dan fungsi ' min:1 ' mengunci agar array tersebut tidak kosong (bodi JSON minimal harus berisi satu objek item alat)
'items.*.alat_id' => 'required exists:alat,id'	Tanda bintang (*) disebut sebagai asterisk wildcard notation . Ini memberi tahu Laravel: "Iterasikan dan periksa setiap baris index objek yang ada di dalam array items tersebut, lalu pastikan field alat_id-nya wajib ada di tabel alat pada kolom id".
public function attributes(): array	Secara bawaan, jika input jumlah pada item pertama kosong, Laravel akan mengeluarkan pesan error: "The items.0.jumlah field is required." Tampilan ini tentu sangat buruk untuk dibaca oleh end-user di frontend. Dengan mendaftarkan array di fungsi attributes() , Laravel secara cerdas akan otomatis mengubah pesan error tersebut menjadi: "The Jumlah barang field is required." tanpa merusak pemetaan index array aslinya.

Implementasi Controllers

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:controller API/PeminjamanController --api**

Lakukan perubahan baris kode program pada blok function index menjadi:

```

<?php

namespace App\Http\Controllers\API;

```

```

use App\Http\Controllers\Controller;
use App\Http\Requests\Peminjaman\StorePeminjamanRequest;
use App\Http\Resources\PeminjamanResource;
use App\Models\Alat;
use App\Models\Peminjaman;
use App\Models\DetailPinjam;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\DB;

use Exception;

class PeminjamanController extends Controller
{
    public function index(): JsonResponse
    {
        $user = auth()->user();
        $query = Peminjaman::with(['user', 'detailPinjam.alat',
'pengembalian']);
        if ($user->role === 'peminjam') {
            $query->where('user_id', $user->id);
        }
        $peminjaman = $query->latest()->get();

        return response()->json([
            'message' => 'Daftar peminjaman berhasil diambil.',
            'data' => PeminjamanResource::collection($peminjaman
        ]);
    }
}

```

Lakukan perubahan baris kode program pada blok function store menjadi:

```

public function store(StorePeminjamanRequest $request): JsonResponse
{
    try {
        $peminjaman = DB::transaction(function () use ($request) {
            $user = auth()->user();
            $peminjaman = Peminjaman::create([
                'user_id' => $user->id,
                'tgl_pinjam' => now()->toDateString(),
                'tgl_kembali_plan' => $request->tgl_kembali_plan,
            ]);
        });
    } catch (Exception $e) {
        return response()->json([
            'message' => 'Gagal menyimpan data peminjaman.',
            'data' => null,
        ], 500);
    }

    return response()->json([
        'message' => 'Peminjaman berhasil disimpan.',
        'data' => PeminjamanResource::collection($peminjaman),
    ]);
}

```

```

        'status' => 'diajukan',
    ]);

    foreach ($request->items as $item) {
        // lockForUpdate mengunci baris data di database
        // sampai transaksi ini COMMIT
        $alat = Alat::lockForUpdate()-
>findOrFail($item['alat_id']);
        if ($alat->stok < $item['jumlah']) {
            throw new Exception("Stok alat '{$alat-
>nama_alat}' tidak mencukupi. Sisa stok: {$alat->stok}");
        }
        DetailPinjam::create([
            'peminjaman_id' => $peminjaman->id,
            'alat_id' => $item['alat_id'],
            'jumlah' => $item['jumlah'],
        ]);
    }

    return $peminjaman->load(['user', 'detailPinjam.alat']);
});

return response()->json([
    'message' => 'Peminjaman berhasil diajukan. Menunggu
persetujuan petugas.',
    'data' => new PeminjamanResource($peminjaman) //
Dioptimalkan menggunakan Resource
], 201);

} catch (Exception $e) {
    return response()->json(['message' => $e->getMessage()], 422);
}
}

```

Lakukan perubahan baris kode program pada blok function show menjadi:

```

public function show(Peminjaman $peminjaman): JsonResponse
{
    $user = auth()->user();

```

```

        if ($user->role === 'peminjam' && $peminjaman->user_id !== $user->id) {
            return response()->json(['message' => 'Akses ditolak.'], 403);
        }

        return response()->json([
            'message' => 'Detail peminjaman berhasil diambil.',
            'data' => new PeminjamanResource($peminjaman->load(['user', 'detailPinjam.alat', 'pengembalian']))
        ]);
    }
}

```

Lakukan perubahan baris kode program pada blok function update menjadi:

```

    public function update(StorePeminjamanRequest $request, Peminjaman $peminjaman): JsonResponse
    {
        $user = auth()->user();

        if ($user->role === 'peminjam' && $peminjaman->user_id !== $user->id) {
            return response()->json(['message' => 'Akses ditolak.'], 403);
        }

        if ($peminjaman->status !== 'diajukan') {
            return response()->json([
                'message' => "Peminjaman tidak dapat diubah karena status saat ini: {$peminjaman->status}."
            ], 400);
        }

        try {
            DB::transaction(function () use ($request, $peminjaman) {
                $peminjaman->update([
                    'tgl_kembali_plan' => $request->tgl_kembali_plan,
                ]);

                $peminjaman->detailPinjam()->delete();

                foreach ($request->items as $item) {

```

```

        // Ditambahkan lockForUpdate agar konsisten aman dari
        race condition saat update data draft
        $alat = Alat::lockForUpdate()-
>findOrFail($item['alat_id']);

        if ($alat->stok < $item['jumlah']) {
            throw new Exception("Stok alat '{$alat-
>nama_alat}' tidak mencukupi.");
        }

        DetailPinjam::create([
            'peminjaman_id' => $peminjaman->id,
            'alat_id' => $item['alat_id'],
            'jumlah' => $item['jumlah'],
        ]);
    }
});

return response()->json([
    'message' => 'Data permohonan peminjaman berhasil
diperbarui.',
    'data' => new PeminjamanResource($peminjaman-
>load(['user', 'detailPinjam.alat']))
]);

} catch (Exception $e) {
    return response()->json(['message' => $e->getMessage()], 422);
}
}

```

Lakukan perubahan baris kode program pada blok function destroy menjadi:

```

public function destroy(Peminjaman $peminjaman): JsonResponse
{
    $user = auth()->user();

    if ($user->role === 'peminjam' && $peminjaman->user_id !== $user-
>id) {
        return response()->json(['message' => 'Akses ditolak.'], 403);
    }
}

```

```

        if ($peminjaman->status !== 'diajukan') {
            return response()->json(['message' => 'Peminjaman tidak dapat
dibatalkan.'], 400);
        }

        DB::transaction(function () use ($peminjaman) {
            $peminjaman->detailPinjam()->delete(); // Hapus child record
terlebih dahulu
            $peminjaman->delete();
        });

        return response()->json([
            'message' => 'Permohonan peminjaman berhasil dibatalkan dan
dihapus.'
        ]);
    }
}

```

Buat function baru dengan nama approve dengan kode program seperti berikut:

```

public function approve(Peminjaman $peminjaman): JsonResponse
{
    if ($peminjaman->status !== 'diajukan') {
        return response()->json([
            'message' => "Persetujuan gagal. Status saat ini:
{$peminjaman->status}."
        ], 400);
    }

    try {
        DB::transaction(function () use ($peminjaman) {
            $peminjaman->update(['status' => 'dipinjam']);

            foreach ($peminjaman->detailPinjam as $detail) {
                // Mengunci baris alat demi validasi final sebelum
stok dikurangi
                $alat = Alat::lockForUpdate()->findOrFail($detail->alat_id);

                if ($alat->stok < $detail->jumlah) {
                    throw new Exception("Persetujuan gagal. Stok alat
'{$alat->nama_alat}' mendadak tidak mencukupi.");
                }
            }
        });
    } catch (Exception $e) {
        return response()->json(['message' => $e->getMessage()], 500);
    }

    return response()->json(['message' => 'Peminjaman berhasil dipinjam.'], 200);
}

```

```

    }

    $alat->decrement('stok', $detail->jumlah);
}

});

return response()->json([
    'message' => 'Peminjaman disetujui. Stok alat telah
otomatis dikurangi.',
    'data' => new PeminjamanResource($peminjaman-
>load(['user', 'detailPinjam.alat']))
]);

} catch (Exception $e) {
    return response()->json(['message' => $e->getMessage()], 422);
}
}

```

Buat function baru dengan nama approve dengan kode program seperti berikut:

```

public function riwayat(): JsonResponse
{
    $riwayat = Peminjaman::with(['detailPinjam.alat', 'pengembalian'])
        ->where('user_id', auth()->id())
        ->latest()
        ->get();

    return response()->json([
        'message' => 'Riwayat peminjaman Anda.',
        'data' => PeminjamanResource::collection($riwayat)
    ]);
}

```

Penjelasan Baris Kode Program pada PeminjamanController

Baris Kode Program	Pembahasan
DB::transaction(function () use (...) { ... });	Pembuatan peminjaman melibatkan manipulasi multi-tabel (peminjaman dan detail_pinjam). Jika penyimpanan data di detail_pinjam gagal di tengah jalan (misal karena stok habis), fungsi transaksi ini menjamin data master rintisan di peminjaman otomatis dibatalkan (rollback). Data database Anda dijamin tidak

	akan menggantung atau korup.
<code>Alat::lockForUpdate()->find(...)</code>	Ini adalah metode penanganan Pessimistic Locking . Saat baris data alat tersebut dibaca, Laravel menginstruksikan database (Engine InnoDB) untuk mengunci baris data tersebut secara eksklusif. Sesi atau pengguna lain tidak akan bisa mengubah atau mengurangi stok alat tersebut sampai transaksi ini selesai diproses (COMMIT atau ROLLBACK).
<code>\$alat->decrement('stok', \$detail->jumlah);</code>	Dibandingkan menulis <code>\$alat->stok = \$alat->stok - \$jumlah; \$alat->save();</code> , fungsi decrement() jauh lebih aman karena melakukan operasi pengurangan nilai matematis secara atomik langsung di dalam query level database server (UPDATE alat SET stok = stok - x).
<code>if (\$user->role === 'peminjam' && \$peminjaman->user_id !== \$user->id)</code>	Ini adalah layer otorisasi manual (Data Ownership Guard). Baris ini memastikan keamanan data tingkat pengguna agar peminjam nakal tidak bisa menebak ID URL milik orang lain (serangan IDOR - Insecure Direct Object Reference) untuk melihat, mengubah, atau menghapus transaksi peminjaman milik pengguna lain.

Update route/api.php

Tambahkan Route pada blok middleware dengan role admin:

```
Route::get('/peminjaman', [PeminjamanController::class, 'index']);
Route::get('/peminjaman/{peminjaman}', [PeminjamanController::class, 'show']);
Route::post('/peminjaman/{peminjaman}/approve', [PeminjamanController::class, 'approve']);
Route::put('/peminjaman/{peminjaman}', [PeminjamanController::class, 'update']);
Route::delete('/peminjaman/{peminjaman}', [PeminjamanController::class, 'destroy']);
```

Tambahkan Route pada blok middleware dengan role petugas:

```
Route::post('/peminjaman/{peminjaman}/approve', [PeminjamanController::class, 'approve']);
```

Tambahkan Route pada blok middleware dengan role peminjam:

```
Route::post('/peminjaman', [PeminjamanController::class, 'store']);
Route::get('/riwayat-pinjam', [PeminjamanController::class, 'riwayat']);
```

Tugas Pengujian

Silahkan lakukan pengujian pada setiap function pada **PeminjamanController** dengan hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

Perlihatkan pada Guru yang mengampu dikelas jika tugas sudah selesai untuk melakukan penilaian !

Membuat Fitur Pengembalian

Implementasi Form Request

StorePengembalianRequest

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request**

Pengembalian/StorePengembalianRequest

Kemudian isi/lakukan perubahan pada StorePengembalianRequest menjadi:

```
<?php
namespace App\Http\Requests\Pengembalian;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class StorePengembalianRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }
    public function rules(): array
    {
        return [
            'peminjaman_id' => ['required', 'integer',
                Rule::exists('peminjaman', 'id')
            ],
            'kondisi_kembali' => ['required', 'string', 'max:255'],
            'denda' => ['nullable', 'integer', 'min:0'],
        ];
    }
    public function attributes(): array
    {
        return [
            'peminjaman_id' => 'ID Peminjaman',
            'kondisi_kembali' => 'Kondisi barang kembali',
            'denda' => 'Nilai denda',
        ];
    }
}
```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
'peminjaman_id' => ['required', 'integer', Rule::exists('peminjaman', 'id')]	Kunci utama dalam proses pengembalian adalah menghubungkannya dengan data peminjaman yang valid. Aturan exists:peminjaman,id memastikan petugas tidak salah memasukkan nomor resi/ID transaksi peminjaman. Jika ID transaksi acak diinput, Laravel langsung menolaknya sebelum kueri database yang salah dieksekusi.
'kondisi_kembali' => ['required', 'string', 'max:255']	Aturan wajib (required) digunakan agar petugas memberikan laporan fisik mengenai kondisi alat saat dikembalikan (misal: "Bagus", "Lengkap", atau "Ada goresan pada lensa"). Batasan max:255 menjaga agar ukuran teks tidak terlalu panjang dan merusak struktur layout tabel database.

UpdatePengembalianRequest

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:request**

Pengembalian/UpdatePengembalianRequest

Kemudian isi/lakukan perubahan pada UpdatePengembalianRequest menjadi:

```
<?php

namespace App\Http\Requests\Pengembalian;

use Illuminate\Foundation\Http\FormRequest;

class UpdatePengembalianRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'kondisi_kembali' => ['required', 'string', 'max:255'],
            'denda' => ['nullable', 'integer', 'min:0'],
        ];
    }

    public function attributes(): array
```

```

    {
        return [
            'kondisi_kembali' => 'Kondisi barang kembali',
            'denda' => 'Nilai denda',
        ];
    }
}

```

Implementasi Controllers

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:controller API/PengembalianController --api**

Lakukan perubahan baris kode program pada blok function index menjadi:

```

<?php

namespace App\Http\Controllers\API;

use App\Http\Controllers\Controller;
use App\Http\Requests\Pengembalian\StorePengembalianRequest;
use App\Models\Alat;
use App\Models\Peminjaman;
use App\Models\Pengembalian;
use Carbon\Carbon;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\DB;
use Exception;

class PengembalianController extends Controller
{
    public function index(): JsonResponse
    {
        $user = auth()->user();
        $query = Pengembalian::with(['peminjaman.user',
        'peminjaman.detailPinjam.alat', 'petugas']);
        if ($user->role === 'peminjam') {
            $query->whereHas('peminjaman', function ($q) use ($user) {
                $q->where('user_id', $user->id);
            });
        }
    }
}

```

```

    $pengembalian = $query->latest()->get();
    return response()->json([
        'message' => 'Riwayat pengembalian berhasil diambil.',
        'data' => $pengembalian
    ]);
}
}

```

Lakukan perubahan baris kode program pada blok function show menjadi:

```

public function show(Pengembalian $pengembalian): JsonResponse
{
    $user = auth()->user();

    // Eager load relasi
    $pengembalian->load(['peminjaman.user',
        'peminjaman.detailPinjam.alat', 'petugas']);

    // Otorisasi privasi
    if ($user->role === 'peminjam' && $pengembalian->peminjaman-
        >user_id !== $user->id) {
        return response()->json(['message' => 'Akses ditolak.'], 403);
    }

    return response()->json([
        'message' => 'Detail pengembalian berhasil diambil.',
        'data' => $pengembalian
    ]);
}

```

Lakukan perubahan baris kode program pada blok function store menjadi:

```

public function store(StorePengembalianRequest $request): JsonResponse
{
    try {
        $pengembalian = DB::transaction(function () use ($request) {
            // Kunci baris peminjaman ini selama transaksi agar tidak
            dimanipulasi proses lain
            $peminjaman = Peminjaman::with('detailPinjam')-
                >lockForUpdate()->find($request->peminjaman_id);

            // Guarding: Pastikan statusnya sedang dipinjam

```

```

        if ($peminjaman->status !== 'dipinjam') {
            throw new Exception("Data ditolak. Peminjaman ini
berstatus '{$peminjaman->status}', bukan 'dipinjam'.");
        }

        // Cek keterlambatan menggunakan Carbon
        $tglKembaliPlan = Carbon::parse($peminjaman-
>tgl_kembali_plan)->startOfDay();
        $hariIni = Carbon::now()->startOfDay();

        // Jika hari ini lebih besar dari tanggal rencana kembali,
maka telat

        $statusPeminjamanBaru = $hariIni-
>greaterThan($tglKembaliPlan) ? 'telat' : 'dikembalikan';

        // 1. Insert data ke tabel pengembalian
        $pengembalian = Pengembalian::create([
            'peminjaman_id' => $peminjaman->id,
            'tgl_kembali' => now()->toDateString(),
            'kondisi_kembali' => $request->kondisi_kembali,
            'denda' => $request->denda ?? 0, // Default 0 jika
null
            'petugas_id' => auth()->id(), // Ambil ID user
(petugas) yang sedang login
        ]);

        // 2. Ubah status di tabel peminjaman utama
        $peminjaman->update(['status' => $statusPeminjamanBaru]);

        // 3. Kembalikan (tambah) stok alat berdasarkan
detail_pinjam
        foreach ($peminjaman->detailPinjam as $detail) {
            $alat = Alat::lockForUpdate()->find($detail->alat_id);
            // increment() otomatis menambah nilai pada field yang
ditentukan

            $alat->increment('stok', $detail->jumlah);
        }

        // Opsional: Catat ke log aktivitas petugas
        auth()->user()->logAktivitas()->create([

```

```

        'aktivitas' => "Memproses pengembalian peminjaman ID:
#{ $peminjaman->id } dengan status akhir: { $statusPeminjamanBaru }."
    });

    // Load relasi agar response JSON lebih informatif
    return $pengembalian->load(['peminjaman.user',
'petugas']);
});

    return response()->json([
        'message' => 'Proses pengembalian alat berhasil
diselesaikan.',
        'data' => $pengembalian
    ], 201);

} catch (Exception $e) {
    // Tangkap pesan error dari throw exception di atas (misal
status bukan 'dipinjam')
    return response()->json(['message' => $e->getMessage()], 422);
}
}

```

Lakukan perubahan baris kode program pada blok function Update menjadi:

```

public function update(UpdatePengembalianRequest $request,
Pengembalian $pengembalian): JsonResponse
{
    // Mengamankan data dengan membatasi field yang boleh dikoreksi
petugas
    $pengembalian->update([
        'kondisi_kembali' => $request->kondisi_kembali,
        'denda' => $request->denda ?? $pengembalian->denda,
    ]);

    return response()->json([
        'message' => 'Data pengembalian berhasil diperbarui.',
        'data' => $pengembalian->load(['peminjaman.user', 'petugas'])
    ]);
}

```

Lakukan perubahan baris kode program pada blok function Destroy menjadi:

```

public function destroy(Pengembalian $pengembalian): JsonResponse

```

```

{
    try {
        DB::transaction(function () use ($pengembalian) {
            $peminjaman = Peminjaman::with('detailPinjam')->lockForUpdate()->findOrFail($pengembalian->peminjaman_id);

            // Tarik kembali stok ke gudang (karena status kembali dibatalkan, stok berkurang lagi)
            foreach ($peminjaman->detailPinjam as $detail) {
                $alat = Alat::lockForUpdate()->findOrFail($detail->alat_id);

                if ($alat->stok < $detail->jumlah) {
                    throw new Exception("Gagal membatalkan pengembalian. Stok alat '{$alat->nama_alat}' saat ini tidak mencukupi untuk ditarik kembali.");
                }

                $alat->decrement('stok', $detail->jumlah);
            }

            // Kembalikan status peminjaman master menjadi dipinjam kembali
            $peminjaman->update(['status' => 'dipinjam']);

            // Log Aktivitas jika metode/relasi tersedia
            auth()->user()->logAktivitas()->create(['aktivitas' => "Membatalkan pengembalian ID: #{ $pengembalian->id }"]);

            $pengembalian->delete();
        });

        return response()->json([
            'message' => 'Data pengembalian berhasil dihapus. Stok dan status peminjaman telah dikembalikan ke kondisi semula.'
        ]);
    } catch (Exception $e) {
        return response()->json(['message' => $e->getMessage()], 422);
    }
}

```

```
}
```

Penjelasan baris kode program PengembalianController

Baris Kode Program	Penjelasan
<code>\$query->whereHas('peminjaman', function (\$q) use (\$user) { ... }) (index)</code>	Tabel pengembalian tidak memiliki kolom <code>user_id</code> secara langsung (karena hubungannya terikat ke rujukan transaksi lewat <code>peminjaman_id</code>). Fungsi whereHas() digunakan untuk melakukan kueri filter lintas tabel (relationship query filtering). Laravel secara cerdas melakukan operasi EXISTS SQL ke tabel <code>peminjaman</code> untuk menyaring nota kembali yang hanya dimiliki oleh ID user peminjam yang sedang aktif.
<code>Carbon::parse(\$peminjaman->tgl_kembali_plan)->startOfDay();</code>	Membandingkan dua objek tanggal mentah di PHP sering kali tidak akurat akibat adanya perbedaan komponen jam, menit, dan detik berjalan. Fungsi startOfDay() mereset waktu jam ke pukul 00:00:00. Hal ini menjamin kalkulasi tanggal murni (contoh membandingkan Tanggal 27 dengan Tanggal 26) menghasilkan keputusan keterlambatan yang akurat 100% tanpa terganggu jam pengerjaan sistem.
<code>\$hariIni->greaterThan(\$tglKembaliPlan)</code>	Ini adalah metode ekspresif bodi mutasi waktu dari Carbon untuk menggantikan operator matematika pembandingan klasik (<code>></code>). Jika hari eksekusi pengembalian melewati batas rencana janji, status otomatis diset menjadi 'telat', yang mempermudah kalkulasi data denda berkala di masa depan.
<code>\$alat->increment('stok', \$detail->jumlah);</code>	Kebalikan dari metode decrement() , fungsi increment() mengirimkan perintah langsung ke mesin database untuk menaikkan nilai integer pada baris <code>stok</code> secara atomik. Ini mengeliminasi risiko kegagalan pembaruan data jika ratusan barang dikembalikan bersamaan dalam satu waktu.

Update route/api.php

Tambahkan Route pada blok middleware dengan role admin:

```
Route::get('/pengembalian', [PengembalianController::class, 'index']);
Route::get('/pengembalian/{pengembalian}', [PengembalianController::class, 'show']);
Route::put('/pengembalian/{pengembalian}', [PengembalianController::class, 'update']);
Route::delete('/pengembalian/{pengembalian}', [PengembalianController::class, 'destroy']);
```

Tambahkan Route pada blok middleware dengan role petugas:

```
Route::post('/pengembalian', [PengembalianController::class, 'store']);
```

Tugas Pengujian

Silahkan lakukan pengujian pada setiap function pada **PengembalianController** dengan hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

Perlihatkan pada Guru yang mengampu dikelas jika tugas sudah selesai untuk melakukan penilaian !

Log Aktivitas

Materi Singkat

Observer Pattern dalam Laravel adalah sebuah design pattern yang memungkinkan Anda untuk berlangganan dan mendengarkan berbagai peristiwa (events) yang terjadi pada model Eloquent. Pola ini merupakan bagian penting dari penerapan prinsip **Clean Code** dan **Separation of Concerns** (SoC), di mana Controller tetap fokus pada logika bisnis HTTP, sementara Observer menangani efek samping (side-effects) seperti pencatatan log aktivitas di latar belakang.

Fungsi dan Tujuan Utama

1. **Audit Trail** (Log Aktivitas): Secara otomatis merekam jejak audit setiap kali ada perubahan data (seperti saat alat ditambah, dipinjam, atau dikembalikan) tanpa harus menulis kode log manual di setiap fungsi Controller.
2. **Skinny Controller**: Menjaga agar Controller tetap "tipis" atau ringkas karena logika pencatatan log dipindahkan ke kelas tersendiri.
3. **Otomatisasi**: Perubahan pada model secara otomatis memicu tindakan tambahan di balik layar.

Separation of Concerns (SoC) adalah prinsip desain dalam rekayasa perangkat lunak untuk memisahkan program ke dalam bagian-bagian yang berbeda, di mana setiap bagian memiliki tanggung jawab atau "perhatian" (concern) yang spesifik. Dalam ekosistem Laravel 12, penerapan prinsip ini sangat krusial untuk menjaga kode tetap bersih (Clean Code), mudah dikelola, dan memiliki skalabilitas yang tinggi.

Penerapan SoC dalam pengembangan API Laravel:

Laravel menerapkan SoC dengan memecah logika yang biasanya menumpuk di satu tempat (seperti Controller) ke dalam beberapa komponen khusus:

- **Form Request** (Logika Validasi & Otorisasi): Berfungsi sebagai "tameng" di pintu masuk permintaan HTTP. Alih-alih melakukan pengecekan data di dalam Controller, logika validasi (seperti aturan email unik) dan otorisasi (apakah user boleh melakukan aksi tersebut) dipindahkan ke kelas Form Request mandiri. Ini membuat Controller tidak terganggu oleh rantai aturan validasi yang panjang.
- **Controller** (Orkestrator Logika): Dalam arsitektur SoC, Controller harus tetap "tipis" (Skinny Controller). Perannya terbatas sebagai "dirigen" yang menerima input, memanggil model atau layanan yang diperlukan, dan memerintahkan pengembalian respons.
- **Eloquent Model** (Interaksi Basis Data): Model bertanggung jawab penuh atas pengelolaan data dan interaksi langsung dengan basis data (seperti kueri, penyimpanan, dan penghapusan).
- **API Resource / JsonResource** (Presentasi Data): Komponen ini memisahkan logika format keluaran dari logika bisnis. Resource bertugas mentransformasi data model

menjadi JSON yang konsisten, menyembunyikan informasi sensitif (seperti password), dan menangani pemuatan relasi data secara efisien.

- **Observer** (Efek Samping/Side-Effects): Menggunakan Observer Pattern memungkinkan Anda menangani tugas tambahan yang dipicu oleh perubahan data (seperti mencatat log aktivitas) di latar belakang. Dengan cara ini, Controller tidak perlu memproses penulisan log secara manual setiap kali ada data yang berubah.
- **Middleware** (Penyaringan): Menangani masalah lintas-sektoral (cross-cutting concerns) seperti autentikasi, pengecekan peran (RBAC), dan pembatasan akses (rate limiting) sebelum permintaan menyentuh logika aplikasi utama.

Kegiatan Praktikum

1. Membuat File Observer Alat

Kelas ini bertugas untuk mendengarkan (listen) secara otomatis kejadian-kejadian penting (lifecycle events) yang terjadi pada model Alat—seperti saat data baru ditambahkan (created), diperbarui (updated), atau dihapus (deleted)—lalu mencatat jejak audit/riwayatnya ke dalam tabel LogAktivitas. untuk membuat kelas observer alat seperti berikut:

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:observer**

AlatObserver --model=Alat

Kemudian isi/lakukan perubahan pada AlatObserver menjadi:

```
<?php

namespace App\Observers;

use App\Models\Alat;
use App\Models\LogAktivitas;
use Illuminate\Support\Facades\Auth;

class AlatObserver
{
    private function catatLog(string $pesan): void
    {
        if (Auth::check()) {
            LogAktivitas::create([
                'user_id' => Auth::id(),
                'aktivitas' => $pesan,
            ]);
        }
    }

    public function created(Alat $alat): void
```

```

    {
        $this->catatLog("Menambahkan master data alat baru: {$alat->nama_alat}
(ID: {$alat->id})");
    }

    public function updated(Alat $alat): void
    {
        $perubahanArray = array_diff(array_keys($alat->getChanges()),
['updated_at']);

        if (!empty($perubahanArray)) {
            $perubahan = implode(', ', $perubahanArray);
            $this->catatLog("Memperbarui data alat '{$alat->nama_alat}'
(Kolom yang diubah: {$perubahan})");
        }
    }

    public function deleted(Alat $alat): void
    {
        $this->catatLog("Menghapus master data alat: {$alat->nama_alat}
(ID: {$alat->id})");
    }
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
class AlatObserver	Observer adalah kelas khusus yang memisahkan logika bisnis pencatatan log (sampingan) dari Controller utama. Jadi Controller Anda tetap bersih karena tidak perlu memanggil fungsi LogAktivitas::create() secara manual di setiap CRUD.
if (Auth::check())	Kondisi ini bertindak sebagai pengaman. Jika di masa depan Anda memasukkan data alat lewat file migrasi, database seeder, atau perintah terminal (CLI/Artisan), fungsi Auth::check() akan bernilai false. Ini mencegah aplikasi error akibat mencoba mengambil Auth::id() dari sesi yang kosong.
public function created(Alat \$alat)	Event created otomatis dipicu oleh Laravel sesaat setelah sebuah baris data sukses masuk ke dalam tabel database. Laravel melempar objek \$alat yang baru dibuat ke fungsi ini, sehingga kita bisa membaca properti seperti \$alat->id atau

	\$alat->nama_alat.
\$alat->getChanges()	Laravel memiliki dua fungsi pelacak perubahan. \$alat->getDirty() digunakan pada event berawalan updating/saving (sebelum masuk DB). Sedangkan \$alat->getChanges() digunakan pada event berawalan updated/saved (setelah masuk DB). Fungsi ini mengembalikan array berupa kolom apa saja yang nilainya berubah dari nilai aslinya.
array_diff(..., ['updated_at'])	Setiap kali data diperbarui, kolom updated_at otomatis ikut berubah. Agar isi catatan log Anda bersih dan hanya fokus pada data esensial (seperti nama, stok, kondisi), kita membuang kata 'updated_at' dari hasil pelacakan.

2. Membuat File Observer Peminjaman

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:observer PeminjamanObserver --model=Peminjaman**

Kemudian isi/lakukan perubahan pada PeminjamanObserver menjadi:

```
<?php

namespace App\Observers;

use App\Models\Peminjaman;
use App\Models\LogAktivitas;
use Illuminate\Support\Facades\Auth;

class PeminjamanObserver
{
    private function catatLog(string $pesan): void
    {
        if (Auth::check()) {
            LogAktivitas::create([
                'user_id' => Auth::id(),
                'aktivitas' => $pesan,
            ]);
        }
    }

    public function created(Peminjaman $peminjaman): void
    {
        $namaPeminjam = $peminjaman->user?->name ?? 'User';
```

```

        $this->catatLog("Peminjam ({$namaPeminjam}) membuat permohonan
peminjaman baru (ID: #{$peminjaman->id})");
    }

    public function updated(Peminjaman $peminjaman): void
    {
        if ($peminjaman->wasChanged('status')) {
            $this->catatLog("Status peminjaman (ID: #{$peminjaman->id})
berubah menjadi: '{$peminjaman->status}'");
        } else {
            if (!empty($peminjaman->getChanges())) {
                $this->catatLog("Memperbarui detail data peminjaman (ID:
#{$peminjaman->id})");
            }
        }
    }

    public function deleted(Peminjaman $peminjaman): void
    {
        $this->catatLog("Membatalkan/menghapus permohonan peminjaman (ID:
#{$peminjaman->id})");
    }
}

```

3. Membuat File Observer Pengembalian

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:observer**

PengembalianObserver --model=Pengembalian

Kemudian isi/lakukan perubahan pada PengembalianObserver menjadi:

```

<?php

namespace App\Observers;

use App\Models\Pengembalian;
use App\Models\LogAktivitas;
use Illuminate\Support\Facades\Auth;

class PengembalianObserver
{
    private function catatLog(string $pesan): void

```

```

{
    if (Auth::check()) {
        LogAktivitas::create([
            'user_id' => Auth::id(),
            'aktivitas' => $pesan,
        ]);
    }
}

public function created(Pengembalian $pengembalian): void
{
    $this->catatLog("Memproses pengembalian alat untuk Peminjaman ID:
#{ $pengembalian->peminjaman_id}");
}

public function updated(Pengembalian $pengembalian): void
{
    $perubahan = array_diff(array_keys($pengembalian->getChanges()),
['updated_at']);

    if (!empty($perubahan)) {
        $kolom = implode(' ', $perubahan);
        $this->catatLog("Merevisi data pengembalian (ID Kembali:
#{ $pengembalian->id}, Peminjaman ID: #{ $pengembalian->peminjaman_id},
Kolom diubah: { $kolom})");
    }
}

public function deleted(Pengembalian $pengembalian): void
{
    $this->catatLog("Membatalkan/menghapus riwayat pengembalian
(Peminjaman ID: #{ $pengembalian->peminjaman_id})");
}
}

```

Membuat API Resource untuk Log

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:resource LogAktivitasResource**

Kemudian isi/lakukan perubahan pada LogAktivitasResource menjadi:

```

<?php
namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class LogAktivitasResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'user' => [
                'id' => $this->user->id ?? null,
                'name' => $this->user->name ?? 'Sistem',
                'role' => $this->user->role ?? null,
            ],
            'aktivitas' => $this->aktivitas,
            'waktu' => $this->created_at->format('Y-m-d H:i:s'),
        ];
    }
}

```

Membuat LogAktivitasController

Buka terminal kemudian ketikan **docker exec -it laravel-api php artisan make:controller API/LogAktivitasController**

Kemudian isi/lakukan perubahan pada LogAktivitasController menjadi:

```

<?php
namespace App\Http\Controllers\API;

use App\Http\Controllers\Controller;
use App\Http\Resources\LogAktivitasResource;
use App\Models\LogAktivitas;
use Illuminate\Http\JsonResponse;

class LogAktivitasController extends Controller
{
    public function index(): JsonResponse
    {

```

```

        $logs = LogAktivitas::with('user')->latest()->get();

        return response()->json([
            'message' => 'Seluruh catatan log aktivitas berhasil
diambil.',
            'total_data' => $logs->count(),
            'data' => LogAktivitasResource::collection($logs)
        ]);
    }
}

```

Update Route/api.php

Tambahkan Route pada blok middleware dengan role admin: **Route::get('/log-aktivitas', [LogAktivitasController::class, 'index']);**

Tugas Pengujian

Silahkan lakukan pengujian pada setiap function pada **LogAktivitasController** dengan hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

Perlihatkan pada Guru yang mengampu dikelas jika tugas sudah selesai untuk melakukan penilaian !

Membuat Fitur Laporan

LaporanController, sebuah API Controller yang dirancang khusus untuk menghasilkan ringkasan atau log data laporan transaksi peminjaman. Tujuan utamanya adalah menyediakan fitur Filter Dinamis (berdasarkan rentang tanggal dan status) sehingga pihak manajemen atau admin dapat menarik data laporan secara fleksifik, terstruktur, dan efisien tanpa membebani performa database (bebas dari N+1 query).

Kegiatan Praktikum

Buka terminal kemudian ketikkan **docker exec -it laravel-api php artisan make:controller API/LaporanController --api**

Lakukan perubahan baris kode program menjadi :

```
<?php

namespace App\Http\Controllers\API;

use App\Http\Controllers\Controller;
use App\Http\Resources\PeminjamanResource;
use App\Models\Peminjaman;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class LaporanController extends Controller
{
    public function index(Request $request): JsonResponse
    {
        // 1. Validasi Input Parameter
        $validator = Validator::make($request->all(), [
            'start_date' => ['nullable', 'date', 'date_format:Y-m-d'],
            'end_date'    => ['nullable', 'date', 'date_format:Y-m-d',
            'after_or_equal:start_date'],
            'status'      => ['nullable', 'string',
            'in:diajukan,dipinjam,dikembangkan,telat'],
            'per_page'    => ['nullable', 'integer', 'min:1', 'max:100'],
        ]);
        if ($validator->fails()) {
            return response()->json([
                'message' => 'Parameter filter tidak valid.',
            ]);
        }
    }
}
```

```

        'errors' => $validator->errors()
    ], 422);
}
// 2. Eager loading untuk mencegah masalah N+1 Query
$query = Peminjaman::with(['user', 'detailPinjam.alat',
'pengembalian.petugas']);
// Filter: Rentang Tanggal Pinjam
$query->when($request->filled('start_date') && $request-
>filled('end_date'), function ($q) use ($request) {
    $q->whereBetween('tgl_pinjam', [$request->start_date,
$request->end_date]);
});
// Filter: Status Peminjaman
$query->when($request->filled('status'), function ($q) use
($request) {
    $q->where('status', $request->status);
});
// (Pagination)
$perPage = $request->input('per_page', 15);
$laporan = $query->latest()->paginate($perPage);
//API Resource khusus untuk instance Paginate
return PeminjamanResource::collection($laporan)
    ->additional([
        'message' => 'Laporan peminjaman berhasil ditarik.'
    ])
    ->response();
}
}

```

Penjelasan Baris Kode Program

Baris Kode Program	Penjelasan
\$query->when(\$request->filled('status'), function (\$q) use (\$request) { ... })	Fungsi when() adalah mutator kueri kondisional di Laravel. Cara kerjanya: jika argumen pertama bernilai true (artinya parameter status diisi oleh user), maka blok fungsi closure di argumen kedua akan dieksekusi untuk menyuntikkan klausa where('status', ...) . Jika parameter kosong, Laravel akan melewati blok tersebut secara elegan tanpa kita perlu menulis struktur if-else bercabang yang mengotori Controller.
\$q->	Klausa whereBetween digunakan untuk menyaring data numerik

>whereBetween('tgl_pinjam', [\$start, \$end]);	atau waktu di dalam batas rentang tertentu. Kueri ini akan menghasilkan perintah SQL WHERE tgl_pinjam BETWEEN 'start_date' AND 'end_date', yang sangat efisien jika kolom tgl_pinjam sudah diberi indeks (indexed) di struktur tabel database Anda.
'after_or_equal:start_date'	Aturan ini memastikan bahwa pengguna tidak bisa sengaja atau tidak sengaja memasukkan tanggal akhir laporan (end_date) yang mendahului tanggal awal laporan (start_date), yang mana akan mengakibatkan kueri SQL menghasilkan data kosong (zero result).
\$query->latest()->paginate(\$perPage);	fungsi paginate() akan memotong data di level database server menggunakan klausa LIMIT dan OFFSET SQL. Server tidak perlu bekerja keras memproses ribuan baris data, melainkan hanya mengambil jumlah berkas yang diminta per halaman secara berkala.

Update Route/api.php

Tambahkan Route pada blok middleware dengan role admin dan petugas :

Route::get('/laporan-peminjaman', [LaporanController::class, 'index']);

Tugas Pengujian

Silahkan lakukan pengujian pada setiap function pada controller Laporan kondisi(tanpa filer, dengan filter tanggal, dengan filter status) dengan hasil yang diharapkan: **Anda menerima HTTP Status 200 OK.**

Perlihatkan pada Guru yang mengampu dikelas jika tugas sudah selesai untuk melakukan penilaian !